Laborator 1. Introducere în Rețele de Calculatoare

Cunoștințe și abilități ce vor fi dobândite

- Cunoștințe generale legate de stiva OSI-TCP/IP
- Sertizarea unui cablu UTP
- Recunoașterea echipamentelor de conectare între dispozitivele fizice ale unei rețele
- Codificări folosite de standardele de comunicație în rețea

Navigare

Laboratorul 1

- 00. Prezentare RL, asistent, studenți
- 01. Stiva OSI
- 02. Sertizare de cablu UTP
- 03. Recunoaștere de echipamente
- 04. Medii de transmisie
- 05. Exerciții de codificare

Exerciții

00. Prezentare RL, asistent, studenți

Pentru început, asistentul vă va prezenta, pe scurt:

1. Obiectivele materiei
2. Subiectele centrale ale materiei
3. Cunoștințe și competențe pe care le veți dobândi
4. Resursele disponibile
5. Sistemul de evaluare, punctarea laboratorului

01. Stiva OSI

Stiva OSI este stiva de jure din Internet, cuprinzând 7 niveluri de protocoale:

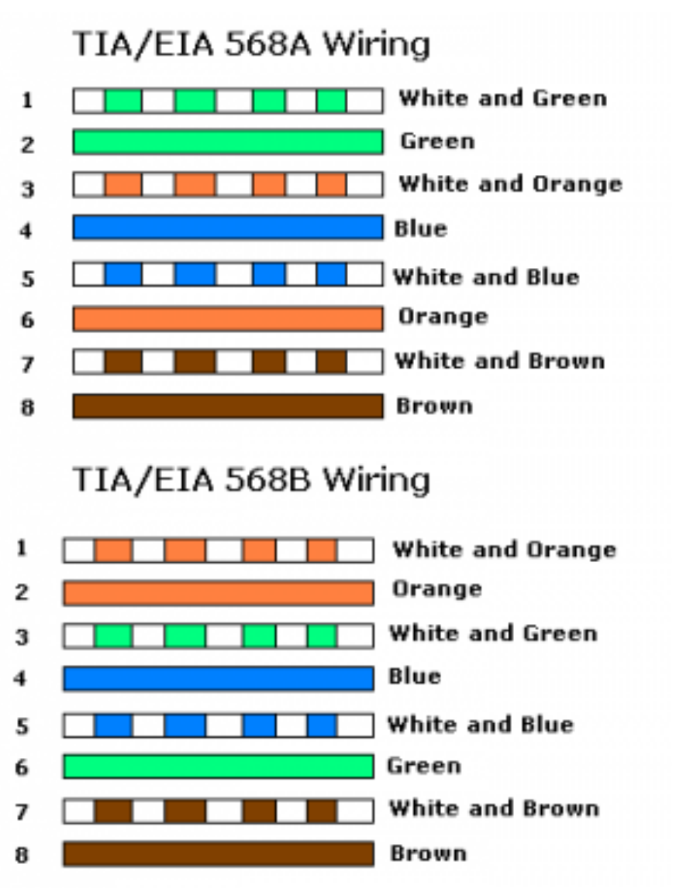
1. Fizic
2. Legătură de date
3. Rețea
4. Transport
5. Sesiune
6. Prezentare
7. Aplicație

Veți participa la un concurs pe echipe pentru recunoașterea termenilor specifici fiecărui nivel din stiva OSI.

Pentru început, împărțiți-vă în echipe de câte 3 persoane. Acolo unde este cazul puteți fi 4 persoane. Fiecare echipă primește o foaie cu 28 de termeni cărora trebuie să le asociați nivelul din stiva OSI. În celula liberă din dreptul fiecărui termen scrieți nivelul din stiva OSI (un număr între 1 și 7).

După 5 minute, veți aduce foile înapoi la asistent. Echipa care a obținut cele mai multe potriviri este declarată câștigătoare.

02. Sertizare de cablu UTP



Ne propunem să câștigați expertiză în sertizarea cablurilor UTP (*Unshielded Twisted Pair*). Scopul torsadării firelor dintr-un cablu UTP sau a ecranarii acestui cablu (STP/FTP) este acela de a diminua efectele interferențelor electromagnetice și a efectului crosstalk . Pentru aceasta veți realiza întâi un cablu *straight through* și apoi un cablu *cross over*.

Pentru aflarea pașilor de sertizare a cablului UTP discutați cu asistentul sau puteți urmări pagina de aici sau acest clip video.

Vă grupați în echipe de câte două persoane și veți primi o cutie ce conține un clește, mufe, două cabluri și un tester. Creați un cablu straight through și celălalt cross over. Fiecare dintre voi va lucra la un capăt al cablului. După ce terminați cablurile, testați-le. Dacă nu merg, refaceți-le.

Vă rugăm lucrați cu grijă și nu faceți mizerie pe jos. Bucațile de cablu și de fire strângeți-le într-un loc și apoi aruncați-le la gunoi.

Un cablu *straight through* va avea ambele capete realizate după același standard: fie T568A, fie T568B. Un cablu *cross over* va avea un capăt după un standard, iar celălalt după standardul complementar. O imagine cu cele doua standarde este cea din partea dreaptă.

03. Recunoaștere de echipamente

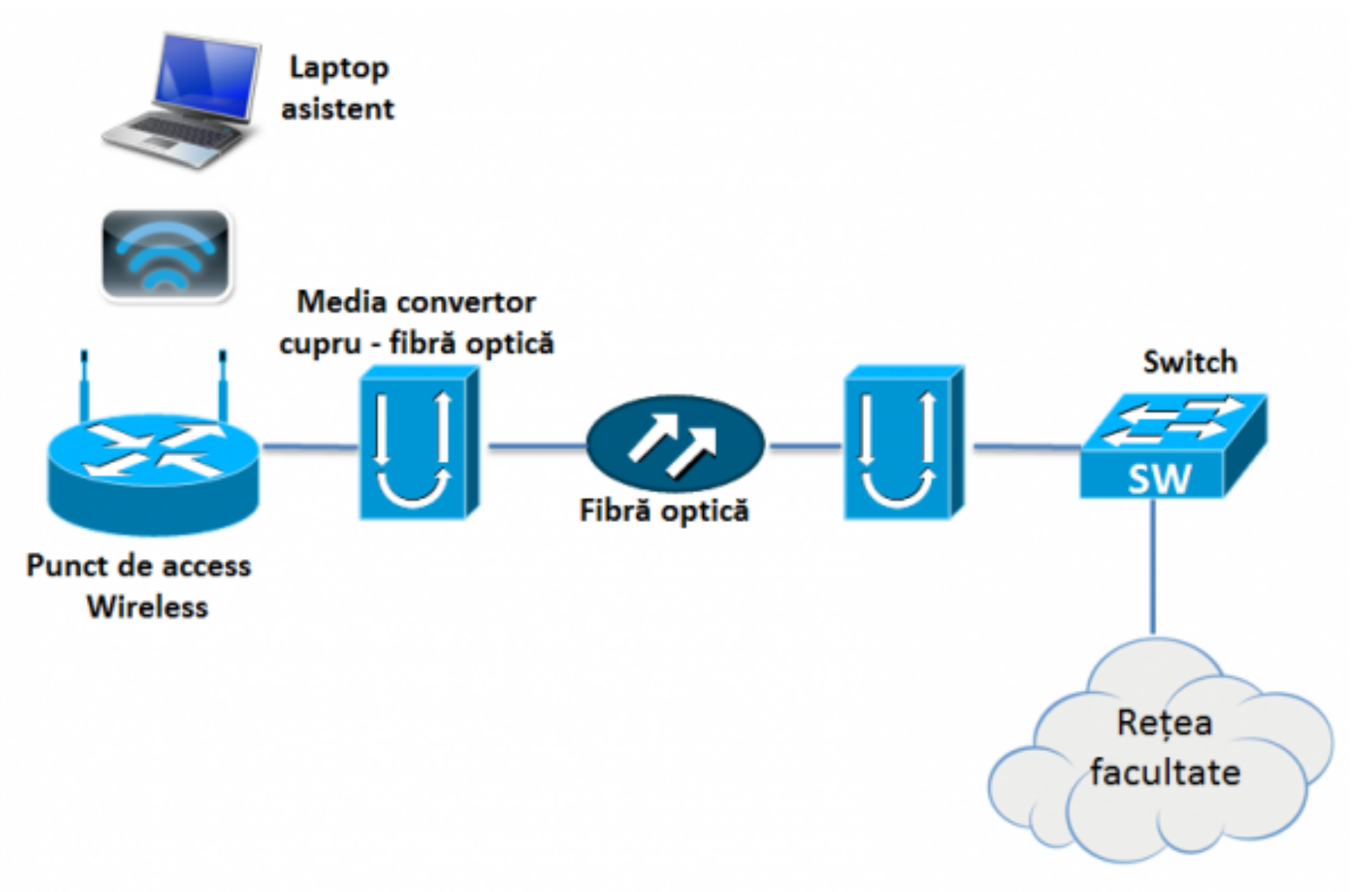
Ne propunem să recunoaștem echipamente de rețea: fie dispozitive, fie medii de interconectare, fie accesorii ale acestora. Echipamentele se găsesc în zona descrisă de asistent.

Vă împărțiți în două echipe și organizăm un concurs de recunoaștere de echipamente. Concursul se va desfășura astfel:

1. Se alege aleator sau prin tragere la sorți echipa de start.
2. Un membru A al unei echipei 1 alege un echipament.
3. Transmite acest echipament ca provocare unui membru B al echipei 2.
4. B spune ce este acel echipament și dă cât mai multe detalii.
5. După ce B încheie "plodoaria", A poate adăuga alte informații.
6. Asistentul distribuie 2 puncte lui B și, apoi lui A astfel:
   - Pentru răspuns greșit/nesatisfăcător/deloc: 0 puncte
   - Pentru răspuns parțial: 1 punct
   - Pentru răspuns cvasi-complet: 2 puncte
   - A poate câștiga puncte doar dacă B a răspuns greșit sau parțial.
7. După această rundă, B alege un echipament și un membru C al echipei 1 și continuă pașii de mai sus.
8. Un membru al unei echipe poate primi și poate alege echipament o singură dată.

La final, se adună punctele și se declară echipa câștigătoare.

04. Medii de transmisie



În topologia prezentată mai sus se observă prezența tuturor mediilor de transmisie folosite în mod curent:

- aerul - wireless
- cupru - UTP
- fibra optică

Semnalul este convertit de la un mediu de transmisie la altul folosind echipamente specifice:

- punct de acces wireless (*wireless access point*) - conversie aer-cupru
- convertor de mediu (*media converter*) - conversie cupru-fibră optică

Identificați fiecare echipament din figura anterioară în topologia fizică (reală) prezentă în laborator.

În topologie observați folosirea unui switch (SW) pentru conectarea la rețeaua facultății. Acesta este un echipament activ al rețelei cu rolul de a interconecta segmente de rețea. Detalii despre funcționarea unui switch veți afla în laboratorul următor.

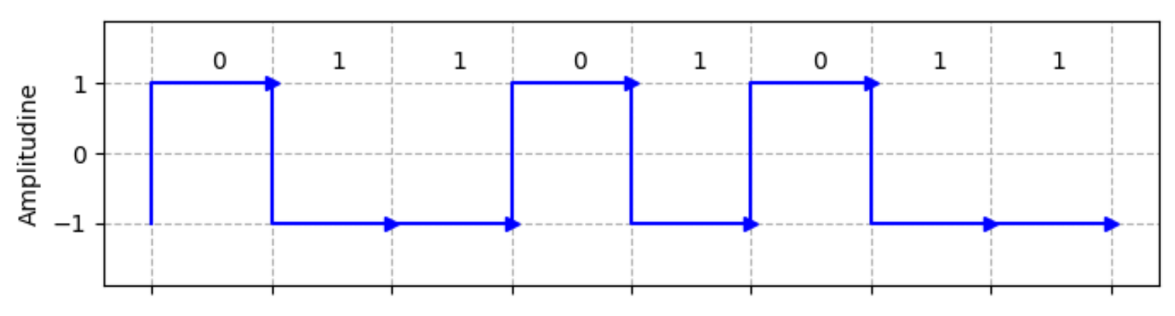
05. Exerciții de codificare

Codificarea unui semnal este esențială în transmisia de date întrucât aceasta permite transferul în mod eficient, cu cât mai puține erori, a informațiilor peste mediile de transmisie. În cadrul acestui exercițiu, veți realiza 4 codificări de bază din transmisia de date:

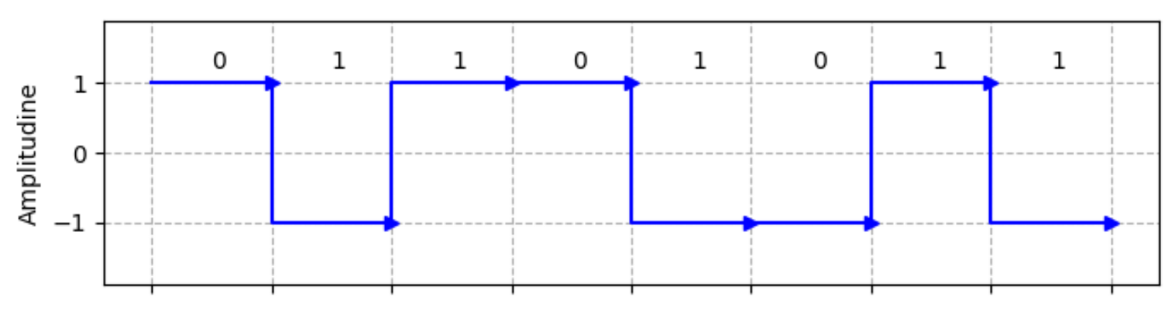
- NRZ-L (Non Return to Zero - Level)
- NRZ-I (Non Return to Zero - Inverted)
- Manchester
- Manchester Diferențial

În figura următoare este reprezentată codificarea literei `k` (ASCII `0x6B` = 0110 1011) folosind toate cele 4 metode. Urmăriți explicațiile asistentului legate de procedeul prin care obțineți fiecare din codificări.

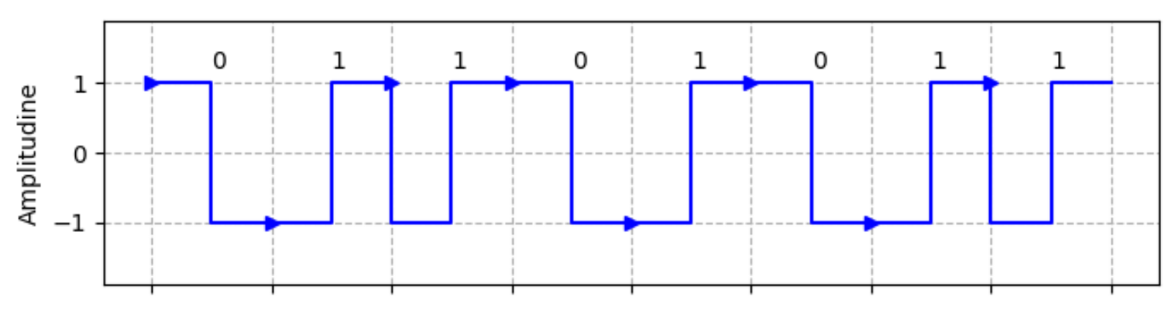
- NRZ-L



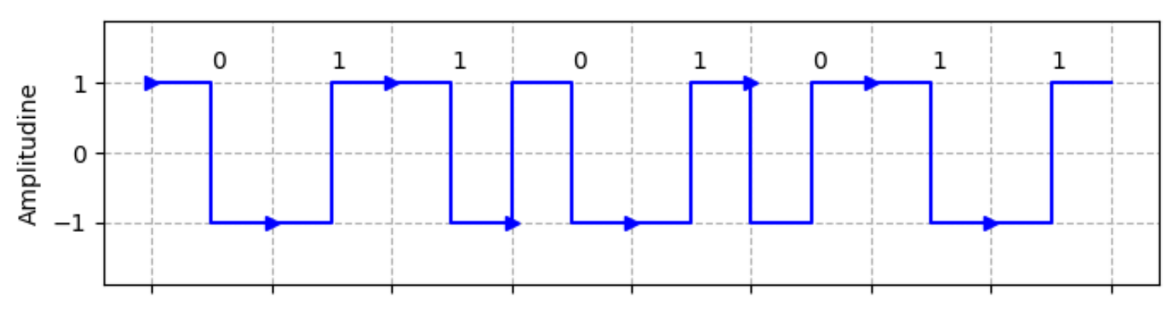
- NRZ-I



- Manchester



- Manchester diferential



Realizați codificarea inițialelor voastre (inițiala prenumelui și inițiala numelui) în format numeric ASCII, folosind cele 4 metode exemplificate mai sus .

- De exemplu, pentru Gabriel Ivan codificați `G` (ASCII `0x47`) și `I` (ASCII `0x49`).

Navigare

Laboratorul 1

- 00. Prezentare RL, asistent, studenți
- 01. Stiva OSI
- 02. Sertizare de cablu UTP
- 03. Recunoaștere de echipamente
- 04. Medii de transmisie
- 05. Exerciții de codificare

Search

Laborator

Laboratorul 1

- 00. Prezentare RL, asistent, studenți
- 01. Stiva OSI
- 02. Sertizare de cablu UTP
- 03. Recunoaștere de echipamente
- 04. Medii de transmisie
- 05. Exerciții de codificare

Curs

Informații generale RL

- **Regulament**
  - Notare Seria CA
  - Notare Seriile CB/CC
- **Resurse**
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

rl/labs/01.txt · Last modified: 2014/04/21 22:54 by mihai.carabas

CC BY-SA CHIMERIC DE W3C CSS DOKUWIKI GET FIREFOX RSS XML FEED W3C XHTML 1.0

Laborator 2. Comutarea pachetelor

Cunoștințe și abilități ce vor fi dobândite

- Procesul de comutare al switch-urilor
- Popularea și conținutul tabelei de comutare
- Spanning Tree Protocol (STP)

Cheat sheet

- Cheat Sheet

Packet Tracer

Descărcați arhiva de instalare Packet Tracer 6.2 **de aici**.

E posibil sa nu mearga download-ul executabilului de Cisco PKT de pe Google Chrome! Incercati cu alt browser in acest caz.

Tutoriale (old):

- Packet Tracer 6.2 Windows
- Packet Tracer 6.2 Linux

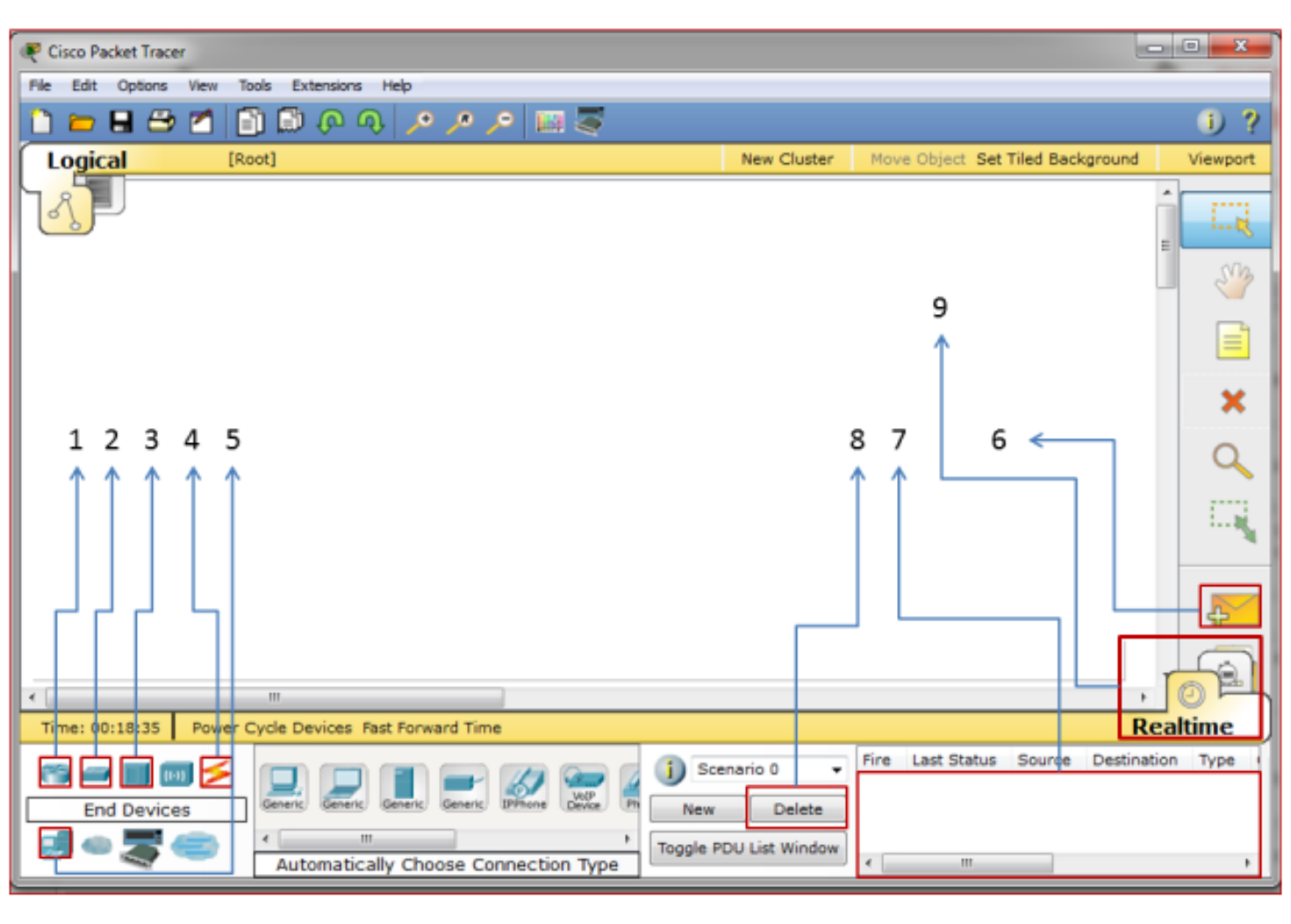
Navigare

Laboratorul 2

- 00. [M][5p] Introducere în Packet Tracer
- 01. [M][10p] Switch vs. Hub
- 02. [M][15p] Popularea tabelei de comutare
- 03. [M][10p] Conținutul tabelei de comutare
- 04. [M][20p] Intrări statice în tabela de comutare
- 05. [M][10p] Configurație cu STP deja rulat
- 06. [M][10p] Rulare STP cu pornire switch-uri
- 07. [A][20p] Schimbare root bridge

Exerciții

00. [M][5p] Introducere în Packet Tracer



Packet Tracer este un utilitar cu ajutorul căruia se pot simula topologii de rețea folosind hub-uri, rutere, switch-uri și calculatoare. Vom prezenta principalele componente din spațiul de lucru:

1. La apăsare vor apărea mai multe modele de **rutere** în dreapta acestuia. Este suficient să alegeți modelul **Generic** → **Router-PT** și să îl plasați în zona de lucru (partea albă din mijloc) oriunde doriți.
2. La apăsare vor apărea mai multe modele de **switch-uri** în dreapta acestuia. Este suficient să alegeți modelul **Generic** → **Switch-PT** și să îl plasați în zona de lucru (partea albă din mijloc) oriunde doriți.
3. La apăsare vor apărea mai multe modele de **hub-uri** în dreapta acestuia. Este suficient să alegeți modelul **Generic** și să îl plasați în zona de lucru (partea albă din mijloc) oriunde doriți.
4. Se alege **tipul legăturii** (UTP straight, UTP cross-over, conexiune serială, etc.). Este suficient să alegeți primul tip de cablu (cel marcat **Automatically Choose Connection Type**, cu iconița unui fulger portocaliu), aceasta punând automat tipul legăturii dintre două echipamente. După selecția tipului de cablu apăsați pe echipamentele care se doresc a fi unite. În unele cazuri veți avea de ales portul echipamentului (ex. **FastEthernet 0/1**, **FastEthernet 0/2**, etc.)
5. La apăsare vor apărea mai multe modele **calculatoare** în dreapta acestuia. Este suficient să alegeți modelul **Generic** și să îl plasați în zona de lucru (partea albă din mijloc) oriunde doriți.
6. Se poate trimite un pachet între două entități de nivel 3 (calculator, router). După selectare, apăsați pe echipamentele între care vreți să trimiteți pachete.
7. În chenarul din dreapta jos va apărea pachetul trimis și status-ul recepției (**Last Status**).
8. Puteți folosi butonul **Delete** pentru a șterge istoria pachetelor trimise.
9. Se poate selecta modul de operare: **Realtime** (modul implicit) sau **Simulation** (se apasă pe tab-ul din spatele **Realtime** pentru a-l activa). În modul **Simulation** se poate vedea pas cu pas cum circulă pachetele între echipamentele active. În modul **Simulation** trimiteți un pachet și apoi apăsați pe butonul **Auto Capture/Play**.

Testare conectivitate

- Creați o topologie cu 2 switch-uri legate între ele și câte 2 calculatoare legate la fiecare din switch-uri.
- Înainte de a putea trimite pachete, trebuie configurate adresele IP pe fiecare din calculatoare. La apăsarea unui click pe calculator, va apărea o fereastră prin care puteți configura calculatoarele. Mergeți pe tab-ul **Desktop** și apăsați pe iconița **IP Configuration**.
- Completați câmpul **IP Address** cu o adresă IP din intervalul **192.168.1.1 - 192.168.1.5** și câmpul **Subnet Mask** cu valoarea **255.255.255.0**. Restul câmpurilor rămân goale.
- Verificați conectivitatea între fiecare 2 calculatoare folosind facilitatea **6** prezentată mai sus.

Dacă ați introdus o **comandă inexistentă** în **CLI** este foarte posibil să se blocheze pentru o perioadă scurtă de timp terminalul și să vă afișeze un mesaj asemănător celui de mai jos:

```
R1# end
Translating "end"...domain server (255.255.255.255)
```

În acest caz folosiți combinația de taste: **CTRL + SHIFT + 6** sau **CTRL + ALT + 6**

01. [M][10p] Switch vs. Hub

Topologie: download.

- Vrem să urmărim diferențele între switch și hub. Vom simula transmiterea de pachete între stații folosind o topologie PacketTracer în care se află 2 rețele cu câte 4 calculatoare fiecare. Diferența între topologii este dispozitivul de interconectare: una este cu un **switch**, iar cealaltă este cu un **hub**.
- Așteptați ca porturile din switch să lumineze cu verde (**nu** cu portocaliu) și apoi trimiteți pachete între oricare 2 calculatoare în cele 2 topologii pentru a vă asigura că există conectivitate. (luminează cu portocaliu timp de circa 30 de secunde până când se definitivează rularea **STP** - *Spanning Tree Protocol*)
- După ce ati verificat existența conectivității, intrați în modul **Simulation** și trimiteți câte un pachet între 2 calculatoare. Faceți acest lucru pentru ambele rețele. Observați ce pachete sunt generate în rețea în fiecare caz.

- Ce puteți spune despre diferența dintre switch și hub?
- La ce nivel din stiva OSI rulează fiecare?

02. [M][15p] Popularea tabelei de comutare

Topologie: download

- Dorim să urmărim modul în care se populează tabela de comutare a unui switch. Topologia este formată dintr-un switch și 4 stații. Încărcați fișierul de topologie în Packet Tracer și așteptați ca porturile switch-ului să lumineze verde în loc de portocaliu (să fi rulat STP).
- După cum ați observat de la exercițiul anterior, switch-ul comută pachetele doar către destinație, făcând acest lucru pe baza unei tabele de comutare (**MAC Address Table**).

- Pentru configurarea unui switch, apăsăm pe iconița sa din fereastră albă din Packet Tracer. Va apărea o fereastră prin care putem configura switch-ul. Singurul **tab** de interes este cel denumit **CLI** (*Command Line Interface*). Apăsați tasta **Enter** cel puțin o dată ca să vedeți unde este cursorul. Acest CLI este specific Cisco si este format după cum urmează:
  - literele până la caracterul ">" (mai mare) reprezintă hostname-ul echipamentului
  - caracterul ">" semnifică faptul că sunteți autentificat ca un utilizator neprivilegiat (fără drepturi administrative)
- Obtineti drepturi administative pentru a afisa tabela de comutare/MAC a switch-ului si observati ca este goala.

Tabela de comutare

- Tabela de comutare conține mai multe coloane dintre care două sunt de interes:
  - coloana **Mac Address** specifică adresa MAC
  - coloana **Ports** specifică portului switch-ului
- Tabela este, așadar, o **asociere** între o **adresă MAC** și un **port**. Spunem că tabelele au, pentru fiecare intrare:
  - o componentă de potrivire și de căutare (**match**) - se caută o adresă MAC
  - o componentă de acțiune - pe baza rezultatului obtinut in urma operatiei de match se trimite pachetul pe portul aferent.
- Partea de căutare poate apărea o singură dată, partea de acțiune de mai multe ori:
  - putem avea mai multe adrese MAC aferente unui port
  - o adresă MAC poate apărea cel mult o dată în tabela de comutare.

- Aflati numele/numărul fiecărui port al unui switch si adresa MAC a unei statii.
- Trimiteți un pachet de la `PC0` la `PC1`. Afișați din nou tabela de comutare.
- Observați adăugarea a două intrări:
  - O intrare este generată la trimiterea pachetului `PC0` la `PC1` - conține adresa MAC a stației `PC0` și portul pe care aceasta este legată la switch
  - A doua este generată la întoarcerea pachetului de la `PC1` la `PC0` - conține adresa MAC a stației `PC1` și portul pe care aceasta este, la rândul ei, legată la switch.

- Trimiteți un pachet de la `PC1` la `PC2`. Afișați din nou tabela de comutare. De ce apare doar o singură intrare, nu două?
- Trimiteți un pachet de la `PC2` la `PC3`. Afișați din nou tabela de comutare. Ce observați?

- Curățați tabela de comutare.
- Intrați în modul **Simulation** și trimiteți un pachet de la `PC1` la `PC3`. Ce observați? Urmăriți în fereastra **Event list** un sumar al pachetelor trimise.
- Mai trimiteți încă un pachet de la `PC1` la `PC3`. De ce s-a comportat diferit de data aceasta? Afișați din nou tabela de comutare.

- Curățați tabela de comutare.
- Intrați în modul **Simulation** și trimiteți un pachet de la `PC1` la `PC3`. La fiecare pas al simulării vizualizați tabela de comutare a switch-ului. Discutați cu asistentul dacă aveți întrebări legate de popularea tabelei de comutare.

03. [M][10p] Conținutul tabelei de comutare

Topologie: download

Analiza tabela de comutare

- Ne propunem să urmărim conținutul tabelei de comutare într-un scenariu cu 2 switch-uri.
- La fiecare switch sunt conectate câte 2 calculatoare. Încărcați fișierul topologie în PacketTracer și așteptați ca switch-urile să ruleze STP. după rularea STP, porturile switch-urilor vor avea culoarea verde.
- Trimiteți pachete între orice 2 calculatoare din rețea și vizualizați tabela de comutare pe ambele switch-uri.
- Observați că un port poate avea asociate mai multe adrese MAC. Portul unui switch care are legătura cu celălalt switch cuprinde trei adrese MAC:
  - adresa MAC a portului celuilalt switch
  - adresele MAC ale stațiilor conectate la celălalt switch

Analiza tabela de comutare dupa decuplarea unui echipament

- Afișați tabela de comutare a switch-ului `Switch0` și rețineți portul asociat adresei MAC a stației `PC1`.
- Decuplați stația `PC1` de la rețea apăsând pe beculețul verde de pe legătura la `Switch0` a acestuia urmată de tasta Escape (un alt mod de a șterge o legătură se poate obține prin folosirea butonului în formă de "X" din dreapta, urmată de apăsarea pe elementul care se dorește a fi șters).
- După decuplarea calculatorului, urmăriți tabela de comutare și observați pierderea intrării aferente.

Analiza tabela de comutare dupa reconectarea unui echipament

- Folosiți un cablu **Copper Straight-Through** pentru a conecta stația `PC1` la celălalt port disponibil al switch-ului `Switch0` (diferit de cel inițial).
- După rularea STP pe switch, trimiteți un pachet între `PC1` și `PC0`.
- Observați popularea tabelei de comutare cu o nouă intrare corespunzătoare adresei MAC a stației `PC1`. O adresă MAC nu poate apărea de două ori în tabela de comutare a unui switch.

04. [M][20p] Intrări statice în tabela de comutare

Topologie: download

- Până în acest punct am observat cum se poate popula dinamic o tabela de comutare a unui switch. Ați văzut că în tabela de comutare intrările erau marcate cu `DYNAMIC` în dreptul coloanei `Type`.
- Tabela de comutare se poate popula și static cu intrări predefinite pentru un anumit port. Dorim să configurăm intrări statice în tabela de comutare a unui switch.

- Pentru a putea configura un echipament Cisco, trebuie să intrăm în modul de configurare.
- Acest mod îl veți recunoaște prin adăugarea sufixului `(config)` la prompt-ul CLI-ului.
- În acest mod puteți doar să realizați configurări, nu puteți vizualiza parametrii switch-ului (ex. afișarea tabelei de comutare).

Configurare statică în tabela de comutare

- Topologia conține un switch și 4 calculatoare. Încărcați fișierul conținând topologia în PacketTracer și așteptați rularea STP.
- Dorim să configurăm o intrare statică pentru stația `PC1`, stație conectată pe portul `FastEthernet3/1` (sau `Fa3/1`) din switch și care are adresa MAC `0090.0C1A.03DE`.
- Trimiteți un pachet de la stația `PC0` la stația `PC1`. Afișați tabela de comutare.
- Se observă tipul `STATIC` pentru intrarea cu adresa MAC a stației `PC1` și intrarea de tipul `DYNAMIC` pentru adresa MAC a stației `PC0`.
- Creați încă o intrare statică pentru stația `PC2` după modelul de mai sus. Verificați conectivitatea cu toate celelalte stații.

- Goliți tabela de comutare.
- Creați o intrare statică pentru portul din switch aferent stației `PC3`, dar în loc sa puneți adresa MAC a stației `PC3`, puneți o adresă MAC care nu există în rețeaua locală.
- Afișați tabela de comutare. Testați conectivitatea stației `PC3` cu celelalte stații și afișați din nou tabela de comuntare.
- Observați prezența a două intrări MAC pe același port, una statică (introdusă mai devreme) și alta dinamică (adresa MAC a stației `PC3`).

- O intrare statică are avantajul permanenței. În absența traficului, o intrare dinamică este ștearsă din tabela de comuntare după câteva minute. O intrare statică poate fi ștearsă doar explicit, prin folosirea comenzii `clear mac-address-table`.
- Dacă dorim să configurăm acces exclusiv al stației `PC3` în tabela de comutare, trebuie folosite comenzi de port security.

05. [M][10p] Configurație cu STP deja rulat

Topologie: download

- STP (*Spanning Tree Protocol*) reprezintă un protocol prin care switch-urile evită buclele în rețea (în special *broadcast storms*). Se creează un arbore de acoperire între switch-uri (un switch este *root bridge*, rădăcina arborelui) care stabilește o cale unică de a ajunge de la un switch la alt switch.
- Pentru început, ne propunem să investigăm o rețea funcțională cu STP rulat. Pentru acesta încărcați topologia în PacketTracer și așteptați circa 30 de secunde rularea STP: unele porturi nu vor mai avea lumini portocalii, ci vor avea lumini verzi.
- Observați că unul dintre porturile switch-ului `Switch1` (`Fa0/1`) este blocat din perspectiva STP, adică nu trimite pachete de date.

- Pentru a popula tabelele de comutare ale switch-urilor trimiteți pachete de verificare a conexiunii între toate stațiile.
- În modul **Simulation** trimiteți un pachet de la stația `PC1` la stația `PC2`. Observați calea urmată de pachet: deși ar fi un drum mai scurt între `Switch1` și `Switch2`, pachetul urmează calea `Switch1` → `Switch0` → `Switch2`. Asta întrucât legătura `Switch1` – `Switch2` este dezactivată.
- În topologia curentă, `Switch0` este **root bridge** și este rădăcina arborelui de acoperire.

- Pe fiecare switch deschideți consola si afisati arborele de acoperire.
- Veți obține o informație precum cea de mai jos (pentru `Switch1`):

```
  Spanning tree enabled protocol ieee
  Root ID    Priority    32769
             Address     0007.EC9C.D291
             Cost        19
             Port        2(FastEthernet1/1)
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID  Priority    32769  (priority 32768 sys-id-ext 1)
             Address     0060.5C08.3EDC
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
             Aging Time  20

Interface        Role Sts Cost      Prio.Nbr Type
---------------- ---- --- --------- -------- --------------------------------
Fa1/1            Root FWD 19        128.2    P2p
Fa0/1            Altn BLK 19        128.1    P2p
Fa2/1            Desg FWD 19        128.3    P2p
```

- Informațiile care pot fi extrase din ieșirea de mai sus a comenzii sunt:
  - Adresa root bridge-ului (*Root ID*) este `0007.EC9C.D291`, adică adresa unui port al `Switch0`.
  - ID-ul curent (al `Switch1`) este `0060.5C08.3EDC`.
  - Portul `Fa1/1` este în starea `FORWARDING` și este legătura către root bridge.
  - Portul `Fa0/1` este în starea `BLOCKED`: parte a legăturii dezactivate `Switch1` – `Switch2`.

06. [M][10p] Rulare STP cu pornire switch-uri

- Ne propunem să urmărim evoluția STP prin pornirea pas cu pas a switch-urilor.
- Pentru început opriți toate switch-urile din topologia anterioară. Pentru aceasta, dați click pe switch și în vederea fizică (*Physical Device View*) din interfață dați click pe butonul de On-Off (e în partea dreaptă a imaginii cu porturi a switch-ului).
- Pentru început, reporniți `Switch1`. Afisati arborele de acoperire.
- Veți obține un output de genul

```
  Spanning tree enabled protocol ieee
  Root ID    Priority    32769
             Address     0060.5C08.3EDC
             This bridge is the root
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec

  Bridge ID  Priority    32769  (priority 32768 sys-id-ext 1)
             Address     0060.5C08.3EDC
             Hello Time  2 sec  Max Age 20 sec  Forward Delay 15 sec
             Aging Time  20

Interface        Role Sts Cost      Prio.Nbr Type
---------------- ---- --- --------- -------- --------------------------------
Fa2/1            Desg FWD 19        128.3    P2p
```

- Este vorba de o singură legătură activă, cu stația `PC1`, legătura în care `Switch1` este root bridge.

- Porniți apoi `Switch2` și observați informațiile legate de STP pe `Switch1` și `Switch2`. Observați că acum rolul de root bridge este deținut de `Switch2`, pentru că cele două switch-uri au priorități identice (`32769`), dar `Switch2` are adresa MAC mai mică.
- La fel, după pornirea `Switch0` acesta va fi root bridge pentru că are adresa MAC mai mică.

- Opriți `Switch1` și observați schimbarea informațiilor despre STP și starea porturilor.
- Reporniți `Switch1`. După ce `Switch1` bootează și în timpul cât rulează STP (sunt porturi cu lumină portocalie), rulați `show spanning-tree` pe switch-uri și observați trecerea unor porturi prin starea `LEARNING` în care se stabilesc legăturile active din cadrul arborelui de acoperire.

07. [A][20p] Schimbare root bridge

- Realizați o configurație în așa fel încât root bridge-ul topologiei anterioare să fie `Switch1`.

Navigare

Laboratorul 2

- 00. [M][5p] Introducere în Packet Tracer
- 01. [M][10p] Switch vs. Hub
- 02. [M][15p] Popularea tabelei de comutare
- 03. [M][10p] Conținutul tabelei de comutare
- 04. [M][20p] Intrări statice în tabela de comutare
- 05. [M][10p] Configurație cu STP deja rulat
- 06. [M][10p] Rulare STP cu pornire switch-uri
- 07. [A][20p] Schimbare root bridge

Laborator

Laboratorul 2

- 00. [M][5p] Introducere în Packet Tracer
- 01. [M][10p] Switch vs. Hub
- 02. [M][15p] Popularea tabelei de comutare
- 03. [M][10p] Conținutul tabelei de comutare
- 04. [M][20p] Intrări statice în tabela de comutare
- 05. [M][10p] Configurație cu STP deja rulat
- 06. [M][10p] Rulare STP cu pornire switch-uri
- 07. [A][20p] Schimbare root bridge

Curs

Informații generale RL

- **Regulament**
  - Notare Seria CA
  - Notare Seriile CB/CC
- **Resurse**
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

Laborator 3. Adresarea IP

Cunoștințe și abilități ce vor fi dobândite

- Cunoștințe legate de adresare IP, mască de rețea, adresă de rețea, adresă de broadcast
- Împărțirea unui spațiu de adresă (subnetare, VLSM – *Variable Length Subnet Mask*)
- Configurarea adreselor IP pe dispozitive dedicate (Cisco, Packet Tracer)

Cheat sheet

- Cheat Sheet

Navigare

Laboratorul 3

- 00. [M][5p] Lucru cu biți
- 01. [M][5p] Adresă IP și mască de rețea
- 02. [M][10p] Calcul adresă de rețea și adresă de broadcast
- 03. [M][5p] Utilitate mască de rețea
- 04. [M][5p] Număr de stații în rețea
- 05. [M][5p] Prezentare subnetare
- 06. [M][10p] Întrebări de subnetare
- 07. [M][10p] Subnetare
- 08. [M][10p] Subnetare avansată
- 09. [M][5p] Prezentare VLSM
- 10. [M][10p] VLSM
- 11. [M][10p] VLSM avansat
- 12. [A][10p] Configurare adrese IP și ruter în Packet Tracer
- 13. [A][BONUS - 10p] Configurare VLSM în Packet Tracer

Exerciții

00. [M][5p] Lucru cu biți

- Reprezentarea adreselor IP se face în grupuri de câte 8 biți. Drept pentru care, vă sunt necesare cunoștințe bune de lucru în binar.
- Atunci când faceți reprezentare în binar a unui număr cel mai bine este să îl descompuneți în puteri ale lui 2. De exemplu:
  - $17 = 16 + 1 = 2^4 + 2^0$; rezultă reprezentarea (în binar pe 8 biți) `00010001`
  - $173 = 128 + 32 + 8 + 4 + 1 = 2^7 + 2^5 + 2^3 + 2^2 + 2^0$; rezultă reprezentarea (în binar pe 8 biți) `10101101`
- Sunt utile abilitățile de a face rapid reprezentarea rapidă a numerelor care se găsesc în jurul puterilor lui 2. De exemplu:
  - pentru `64 = 01000000`
    - 63 este 64-1, un număr cu mulți biți de 1 (6 biți): `00111111`
    - 62 este 63-1, un număr cu mulți biți de 1 mai puțin ultimul (5 biți de 1 și un bit de 0): `00111110`
    - 65 este 64+1, se adaugă ultimul bit de 1 la reprezentarea lui 64: `01000001`

Exerciții

- Realizați reprezentarea binară pe 8 biți a următoarelor numere:
  - 127
  - 80
  - 72
  - 254

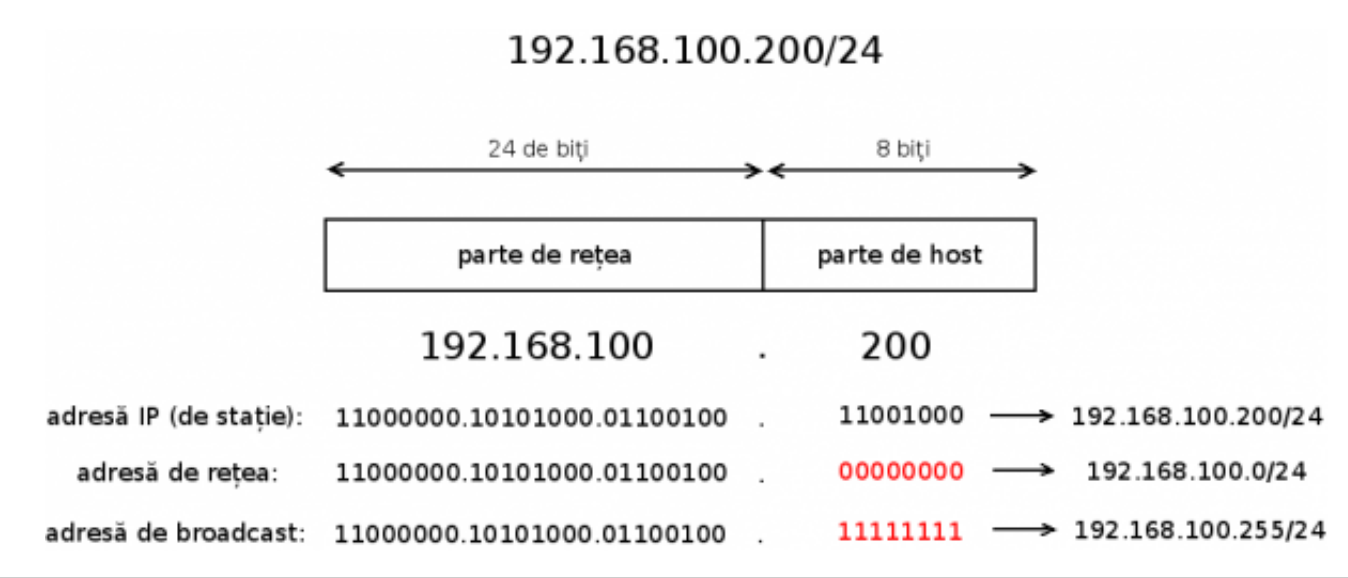
01. [M][5p] Adresă IP și mască de rețea

- Ne propunem să sistematizăm noțiunile de adresă IP, mască de rețea, adresă de rețea și adresă de broadcast.
- Pentru asigurarea conectivității avem nevoie de o adresă de nivel 3, numită și adresă IP . Pe fiecare echipament cu suport de nivel 3 (stații, rutere, switch-uri layer 3) avem nevoie de adresă.

- În cazul unei adrese IP, vom configura, tot timpul, următoarele:
  - **adresa IP** - 4 grupuri a câte 8 biți[1]. Exemplu: `192.168.100.200`[2]
  - **masca de rețea** (subnet mask) - 4 grupuri a câte 8 biți, cu proprietatea că se începe cu bitul 1, iar toți biții de 1 sunt consecutivi, alternanța 0/1 fiind interzisă. De exemplu `11111111.00000000.00000000.00000000` este o mască de rețea validă, iar `11000001.00000000.00000000.0000000` este o mască nevalidă. Pentru a ușura citirea măștii acestea se scrie în zecimal, similar adresei IP: `11111111.00000000.00000000.00000000 = 255.0.0.0`. Datorită proprietății speciale în care biții de 1 sunt consecutivi o altă formă în care veți mai găsi specificată masca de rețea este forma prefixată: `/X`, unde X reprezintă numărul de biți de 1: `11111111.00000000.00000000.00000000` = `255.0.0.0` = `/8`.

- Pornind de la adresa IP și masca de rețea putem identifica două alte proprietăți ale unei rețele (pentru exemplificare vom folosi adresa IP 192.168.100.200/255.255.255.0):
  - **adresa de rețea** - se obține făcând **ȘI-logic** între biții adresei IP și biții măștii de rețea
    - `192.168.100.200 & 255.255.255.0 =` **`192.168.100.0`**
  - **adresa de broadcast** - se obține făcând **SAU-logic** între biții adresei IP și biții din complementul măștii de rețea (complementul se obține inversând valoarea biților de pe fiecare poziție)
    - `192.168.100.200 | 0.0.0.255 =` **`192.168.100.255`**

- Atunci când cunoaștem adresa IP și masca de rețea și vrem să obținem adresa de rețea și adresa de broadcast, este util să folosim masca de rețea pentru a împărți adresa IP în două:
  - O **parte de subrețea**, care se întinde pe câți biți de 1 are masca de rețea. E vorba de 24 de biți, pentru o mască /24 (sau 255.255.255.0) sau 16 biți pentru o mască /16 (sau 255.255.0.0) sau 20 de biți pentru o mască /20 (sau 255.255.240.0).
  - O **parte de stație** (sau parte de host) care se întinde pe restul spațiului (32 - numărul de biți de 1 ai măștii de rețea). E vorba de 8 biți pentru o mască /24 (32-24 = 8) sau de 16 biți pentru o mască /16 (32-16 = 16) sau de 12 biți pentru o mască /20 (32-20 = 12).
- Pe această împărțire vom obține aceeași pentru adresa 192.168.100.200/24 aceleași valori precum cele calculate mai sus, lucru reflectat și în figura de mai jos.


192.168.100.200/24
24 de biți
8 biți
parte de rețea
parte de host
192.168.100
.
200
adresă IP (de stație): 11000000.10101000.01100100 . 11001000 → 192.168.100.200/24
adresă de rețea: 11000000.10101000.01100100 . 00000000 → 192.168.100.0/24
adresă de broadcast: 11000000.10101000.01100100 . 11111111 → 192.168.100.255/24

- Să obținem adresa de rețea și adresa de broadcast pentru adresa `172.16.200.100/20`.
- Transformăm adresa într-o adresă hibridă punând biți de 0 acolo unde se găsește masca de rețea: al treilea octet din cei patru ai adresei IP: `172.16.1100|1000.xxxxxxxx`.
- Am folosit operatorul | (pipe) pentru a separa **partea de rețea** (primii 20 de biți, aferenți rețelei) de **partea de stație (*host*)** (ceilalți biți (32-20 = 12 biți) aferenți stației). Nu sunt relevanți pentru calculul nostru biții ultimului octet așa că am pus `xxxxxxxx` în locul lor.

- Adresa de rețea are **toți biții de stație puși pe 0**, deci va fi `172.16.1100|0000.00000000`. Rezultă adresa de rețea `172.16.192.0/20`.
- Adresa de broadcast are **toți biții de stație puși pe 1**, deci va fi `172.16.1100|1111.11111111`. Rezultă adresa de broadcast `172.16.207.255/20`.

02. [M][10p] Calcul adresă de rețea și adresă de broadcast

- Găsiți adresa de rețea și adresa de broadcast pentru următoarele tupluri de adrese IP și măști de rețea:
  1. `192.168.5.14/24`
  2. `192.168.5.14/25`
  3. `10.10.10.0/8`
  4. `172.16.4.254/22`

03. [M][5p] Utilitate mască de rețea

Topologie: download

- Topologia conține 3 calculatoare legate printr-un switch și au configurate adresele IP. După ce toate legăturile devin active (toate beculețele devin verzi), trimiteți câte un pachet între oricare două stații.
- Observați că nu există conectivitate între stația `PC1` și celelalte două stații (`PC0` si `PC2`).

- Vizualizați adresele IP și măștile de rețea configurate pe fiecare din stații.
- Motivul pentru care nu funcționează transmiterea pachetelor de la `PC1` este pentru că stația `PC1` se află în altă rețea față de stațiile `PC0` și `PC2`.
- Deși aparent este în aceeași rețea, diferă masca de rețea. Vom demonstra acest lucru calculând adresele de rețea:
  - `PC0`: `172.16.10.10 & 255.255.0.0 (/16)` = `172.16.0.0`
  - `PC1`: `172.16.20.20 & 255.255.255.0 (/24)` = `172.16.20.0`
  - `PC2`: `172.16.30.30 & 255.255.0.0 (/16)` = `172.16.0.0`
- Se observă că stațiile `PC0` și `PC2` fac parte din aceeași rețea, pe când `PC1` se află în altă rețea, lucru indicat de masca de rețea.

> Apartenența unei adrese IP la o rețea se poate decide doar folosind atât adresa IP cât și masca de rețea. O configurație fără mască de rețea este incompletă.

04. [M][5p] Număr de stații în rețea

- Masca de rețea face separația între partea de rețea și partea de stație. Numărul de biți pentru partea de stație reflectă adresele IP posibile pentru stații.
- Două dintre aceste adrese, însă, nu sunt asignabile stațiilor (nu pot fi configurate pe stații). Este vorba de prima adresă IP a spațiului și ultima adresă IP a spațiului posibil.
- Prima adresă IP a spațiului de adrese conține doar biți de 0 pentru partea de host și este **adresa de rețea**.
- Ultima adresă IP a spațiului de adrese conține doar biți de 1 pentru partea de host și este **adresa de broadcast**.

- Dacă avem masca de rețea /24, adică 8 biți (32-24) pentru partea de stație, vom avea un spațiul posibil de $2^8 = 256$ adrese. Din aceste adrese 2 nu sunt folosibile (adresa de rețea și adresa de broadcast) și vom avea, așadar 254 de adrese asignabile.
- În general, dacă avem `N` biți alocați pentru partea de stație, vom avea 2^N-2 adrese asignabile.

- Răspundeți la întrebările de mai jos:
  1. Câte adrese asignabile se găsesc în rețeaua `10.10.0.0/16`?
  2. Câte adrese asignabile se găsesc în rețeaua `15.16.192.0/20`?
  3. Câte adrese asignabile se găsesc în rețeaua `1.2.3.4/30`?
  4. Care este masca de rețea a celei mai mici rețele care să cuprindă 25 de adrese asignabile?
  5. Care este masca de rețea a celei mai mici rețele care să cuprindă 62 de adrese asignabile?
  6. Care este masca de rețea a celei mai mici rețele care să cuprindă 127 de adrese asignabile?

05. [M][5p] Prezentare subnetare

- În rețele mari, se pornește de la un spațiu de adresă oferit de o autoritate și apoi se fac subrețele la acest spațiu. Subrețelele vor acoperi zone/departamente diferite. Procesul de împărțire în subrețele se cheamă *subnetting* sau subnetare.
- În forma simplă, se împart în subrețele egale, de dimensiune fixă.

Tutorial

- Vom realiza subnetarea rețelei `78.78.32.0/22` în 4 subrețele
- În forma binară hibridă, adresa rețelei mari este `78.78.001000|00.00000000/22`. Am folosit caracterul | (pipe) pentru a separa partea de rețea de partea de host.
- Avem 4 subrețele deci avem nevoie de 2 biți suplimentari pentru indicarea rețelei: 00, 01, 10, 11. Înseamnă că subrețelele vor avea masca /22 + 2 biți =/24. Adică vor avea $2^8-2 = 254$ stații fiecare.

- Cele 4 subrețele vor avea, respectiv adresele:
  - `78.78.00100`**`00`**`.00000000/24` = `78.78.32.0/24`
  - `78.78.00100`**`01`**`.00000000/24` = `78.78.33.0/24`
  - `78.78.00100`**`10`**`.00000000/24` = `78.78.34.0/24`
  - `78.78.00100`**`11`**`.00000000/24` = `78.78.35.0/24`

- Adresa de broadcast a fiecărei subrețele va fi, respectiv:
  - `78.78.00100000.`**`11111111`**`/24` = `78.78.32.255/24`
  - `78.78.00100001.`**`11111111`**`/24` = `78.78.33.255/24`
  - `78.78.00100010.`**`11111111`**`/24` = `78.78.34.255/24`
  - `78.78.00100011.`**`11111111`**`/24` = `78.78.35.255/24`

- Prima adresă asignabilă din cadrul fiecărei subrețele va fi, respectiv:
  - `78.78.00100000.`**`00000001`**`/24` = `78.78.32.1/24`
  - `78.78.00100001.`**`00000001`**`/24` = `78.78.33.1/24`
  - `78.78.00100010.`**`00000001`**`/24` = `78.78.34.1/24`
  - `78.78.00100011.`**`00000001`**`/24` = `78.78.35.1/24`

- Ultima adresă asignabilă din cadrul fiecărei subrețele va fi, respectiv:
  - `78.78.00100000.`**`11111110`**`/24` = `78.78.32.254/24`
  - `78.78.00100001.`**`11111110`**`/24` = `78.78.33.254/24`
  - `78.78.00100010.`**`11111110`**`/24` = `78.78.34.254/24`
  - `78.78.00100011.`**`11111110`**`/24` = `78.78.35.254/24`

06. [M][10p] Întrebări de subnetare

- Câte adrese asignabile (care pot fi asociate unei stații) se găsesc într-o rețea cu masca /23?
- Câți biți sunt necesari pentru partea de subrețea dacă dorim să creăm 7 subrețele cu cât mai multe stații?
- Câți biți sunt necesari pentru partea de stație dacă dorim să creăm cât mai multe rețele cu 7 stații?
- Pornim de la o rețea /22. Care va fi masca noilor subrețele dacă dorim să creăm 6 subrețele de dimensiune egală cu cât mai multe stații?
- Pornim de la o rețea /22. Care va fi masca noilor subrețele dacă dorim să creăm cât mai multe subrețele cu 27 de stații?
- Dați exemplu de două măști de (sub)rețea pentru care adresa `78.78.78.159` este adresă de broadcast și două măști de (sub)rețea pentru care adresa `78.78.78.159` este adresă de stație.

07. [M][10p] Subnetare

- O firmă are la dispoziție un spațiu de adrese `/24` și un ruter cu mai multe interfețe.
- Ruterul distribuie spațiul de adrese în 4 subrețele cu număr egal de stații.
- Stabiliți adresa de rețea și adresa de broadcast a fiecărei subrețele, precum și prima adresă asignabilă și ultima adresă asignabilă pentru fiecare subrețea:
  1. `17.18.19.0/24`

08. [M][10p] Subnetare avansată

- O firmă are la dispoziție un spațiu de adrese `/22` și un switch layer 3 cu mai multe interfețe.
- Ruterul distribuie spațiul de adrese în 6 subrețele cu număr egal de stații.
- Stabiliți adresa de rețea a fiecărei subrețele și adresa ultimei adrese asignabile pentru fiecare subrețea.
  1. `17.18.16.0/22`

09. [M][5p] Prezentare VLSM

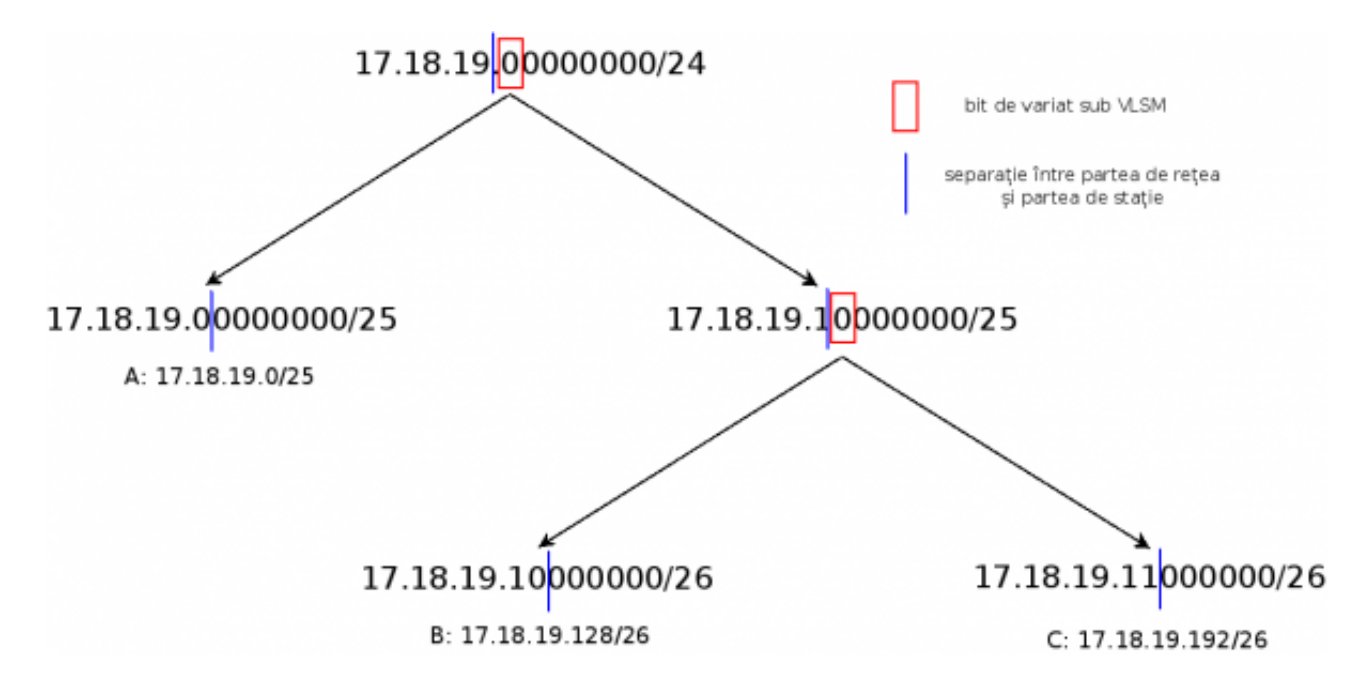
- Să presupunem că un institut primește un spațiu `17.18.19.0/24`. Decizia organizatorică dictează ca acel spațiu să fie împărțit în trei subrețele corespunzătoare pentru trei departamente:

1. A: o subrețea cu un număr de 100 de stații
2. B: o subrețea cu 60 de stații
3. C: o subrețea cu 50 de stații

- Când facem subnetting folosind VLSM, sortăm în ordine descrescătoare subrețelele după numărul de stații. Pornim de la subrețeaua cea mai mare și apoi continuăm, pas cu pas, la rețelele mai mici.
- Prima subrețea are 100 de stații, deci are nevoie de 7 biți pentru partea de host, adică de o mască `/25`. Împărțim rețeaua mare în două subrețele:
  1. `17.18.19.0/25`
  2. `17.18.19.128/25`
- Prima subrețea o alocăm primul departament: `A` - `17.18.19.0/25`.

- A doua subrețea (`17.18.19.128/25`) o împărțim în continuare.
- Trebuie să folosim spațiul rămas pentru celelalte două subrețele (B și C) cu câte 60, respectiv 50 de stații.
- Avem nevoie de 6 biți pentru partea de host a fiecărei subrețele, adică de o mască `/26`. Împărțim noua subrețea în două subrețele:
  1. `17.18.19.128/26`
  2. `17.18.19.192/26`
- Alocăm fiecare dintre cele două subrețele rezultate celor două departamente: `B` - `17.18.19.128/26` și `C` - `17.18.19.192/26`.
- Întrucât spațiul inițial a fost împărțit în subrețele cu mască diferită, numim acest process *Variable Length Subnet Mask* (VLSM).

- Procesul este reprezentat grafic în imaginea de mai jos:

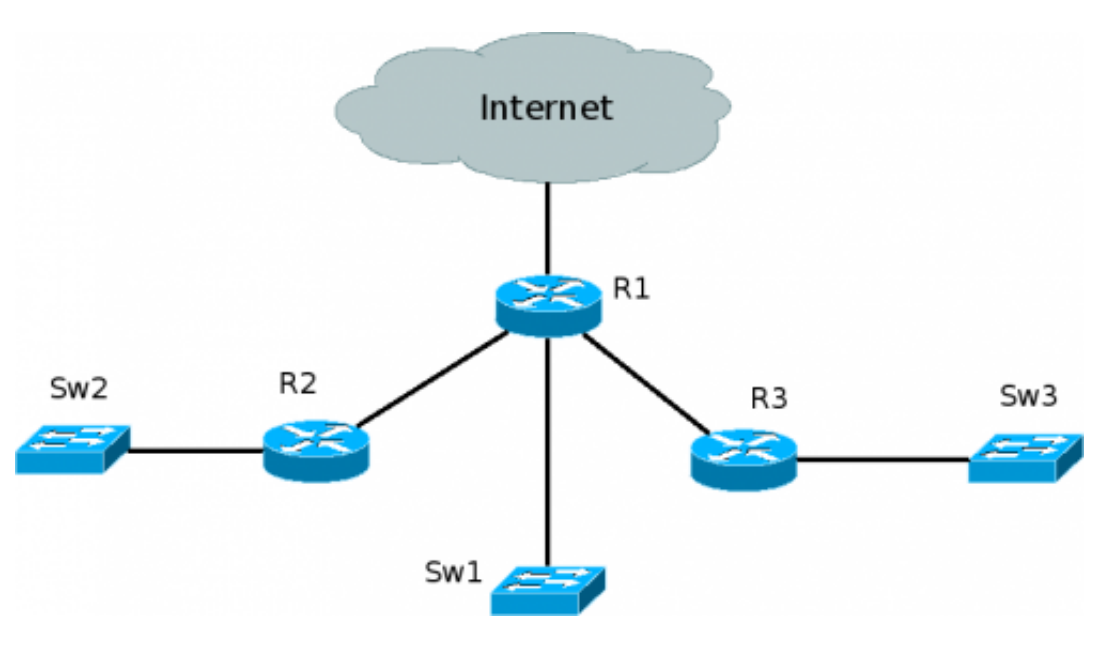


10. [M][10p] VLSM

- O firmă are la dispoziție un spațiu de adrese `/25` și un ruter cu mai multe interfețe.
- Ruterul distribuie spațiul de adrese în 3 subrețele cu următorul număr de stații: 54, 27, 19. Determinați aceste subrețele pentru următoarele spații de adrese:
  1. `12.13.14.128/25`

11. [M][10p] VLSM avansat

- O firmă primește spațiul de adrese 45.67.89.0/24 pe care să îl folosească intern. Firma își definește topologia din imaginea de mai jos.



- Topologia conține 5 rețele:
  - Rețeaua A: între ruterele R1 și R2 (două adrese)
  - Rețeaua B: între ruterele R1 și R3 (două adrese)
  - Rețeaua C: creată în jurul switch-ului Sw1, conectat la ruterul R1, conținând 45 de stații
  - Rețeaua D: creată în jurul switch-ului Sw2, conectat la ruterul R2, conținând 45 de stații
  - Rețeaua E: creată în jurul switch-ului Sw3, conectat la ruterul R3, conținând 45 de stații

- Realizați o distribuție a spațiului de adresă folosind VLSM în cadrul celor cinci rețele.
- Precizați care vor fi adresele pentru rutere. În cazul rețelelor C, D, E, alocați ruterului prima adresă din rețea.

12. [A][10p] Configurare adrese IP și ruter în Packet Tracer

Topologie: download

- Topologia este formată dintr-un ruter și 2 stații conectate la 2 interfețe diferite ale ruterului prin switch-uri.
- Pentru a avea conectivitate între stații și ruter trebuie sa configurăm adrese IP pe fiecare din ele.
- Vom folosi 2 clase, câte una pentru fiecare rețea reprezentată de cele 2 interfețe ale ruterului:
  - `192.168.0.0/24`
  - `10.0.0.0/24`

- Configurați pe stația `PC0` adresa IP `192.168.0.10` cu masca `255.255.255.0`, iar pe `PC1` adresa IP `10.0.0.10` cu masca `255.255.255.0`.
- Pe interfața `Fa0/0` a ruterului vom configura adresa IP `192.168.0.1` cu masca `255.255.255.0`:

```
Router>enable
Router#configure terminal
Router(config)#interface fa0/0
Router(config-if)#no shutdown
Router(config-if)#ip address 192.168.0.1 255.255.255.0
Router(config-if)#exit
Router(config)#exit
Router# ping 192.168.0.10
```

- Configurați interfața `Fa1/0` a ruterului asemănător interfeței `Fa0/0` folosind adresa IP `10.0.0.1` cu masca de rețea `255.255.255.0`.

> Pentru a ridica interfața, folosiți comanda `no shutdown`.
>
> De pe ruter, verificați conectivitatea la stația PC1 folosind comanda `ping`.
>
> Puteți realiza verificarea folosind și mesajele din Packet Tracer.

13. [A][BONUS - 10p] Configurare VLSM în Packet Tracer

Topologie: download

- Topologia conține un ruter care interconectează 3 rețele, fiecare rețea având câte două stații conectate la un switch.
- Folosind clasele obținute la exercițiul 9, configurați cele 3 rețele astfel:
  - reteaua cu statiile `PC0`, `PC1` si interfata aferenta de pe ruter sa corespunda primei clase de adrese
  - reteaua cu statiile `PC2`, `PC3` si interfata aferenta de pe ruter sa corespunda celei de-a doua clase de adrese
  - reteaua cu statiile `PC4`, `PC5` si interfata aferenta de pe ruter sa corespunda celei de-a treia clase de adrese

- Configurarea se va realiza după următoarea regulă:
  - prima adresă alocabilă din clasă va fi configurată pe interfața ruterului
  - alte două adrese la alegere din clasă vor fi configurate pe cele 2 stații

- Verificați conectivitate între stațiile din aceeași rețea și ruter folosind comanda `ping` de pe ruter sau pachetele puse la dispozitie de Packet Tracer.

> Interfețele ruterului trebuie să fie pornite; folosiți comanda `no shutdown`. Beculețele trebuie să fie verzi.
>
> Pentru a afla pe ce interfață a ruterului se conectează o legătură fizică (`Fa1/0`, `Fa4/0` etc.) plasați mouse-ul (*hover*) peste beculețul aferent legăturii și interfeței.

Navigare

Laboratorul 3

- 00. [M][5p] Lucru cu biți
- 01. [M][5p] Adresă IP și mască de rețea
- 02. [M][10p] Calcul adresă de rețea și adresă de broadcast
- 03. [M][5p] Utilitate mască de rețea
- 04. [M][5p] Număr de stații în rețea
- 05. [M][5p] Prezentare subnetare
- 06. [M][10p] Întrebări de subnetare
- 07. [M][10p] Subnetare
- 08. [M][10p] Subnetare avansată
- 09. [M][5p] Prezentare VLSM
- 10. [M][10p] VLSM
- 11. [M][10p] VLSM avansat
- 12. [A][10p] Configurare adrese IP și ruter în Packet Tracer
- 13. [A][BONUS - 10p] Configurare VLSM în Packet Tracer

---

[1] pentru IPv4

[2] valoarea maximă pentru fiecare grup este 255 = 2^8-1

Laborator

Laboratorul 3

- 00. [M][5p] Lucru cu biți
- 01. [M][5p] Adresă IP și mască de rețea
- 02. [M][10p] Calcul adresă de rețea și adresă de broadcast
- 03. [M][5p] Utilitate mască de rețea
- 04. [M][5p] Număr de stații în rețea
- 05. [M][5p] Prezentare subnetare
- 06. [M][10p] Întrebări de subnetare
- 07. [M][10p] Subnetare
- 08. [M][10p] Subnetare avansată
- 09. [M][5p] Prezentare VLSM
- 10. [M][10p] VLSM
- 11. [M][10p] VLSM avansat
- 12. [A][10p] Configurare adrese IP și ruter în Packet Tracer
- 13. [A][BONUS - 10p] Configurare VLSM în Packet Tracer

Curs

Informații generale RL

- **Regulament**
  - Notare Seria CA
  - Notare Seriile CB/CC
- **Resurse**
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

BY-SA CHIMERIC DE W3C CSS DOKUWIKI GET FIREFOX RSS XML FEED W3C XHTML 1.0

Laborator 4. VLAN (Virtual LAN)

Cunoștințe și abilități ce vor fi dobândite

- Ce sunt rețelele locale virtuale (*Virtual Local Area Networks* – VLAN)?
- Tipuri de legături în VLAN-uri (access, trunk)
- Configurarea VLAN-urilor în Cisco IOS

Cheat sheet

- Cheat Sheet

Navigare

Laboratorul 4

- 00. [A][5p] VLAN-uri în Cisco IOS
- 01. [A][10p] VLAN de management
- 02. [A][10p] Porturi în modul access
- 03. [A][15p] Subnetare și VLAN-uri
- 04. [A][10p] Legătură de tip trunchi (trunk)
- 05. [A][20p] Extindere VLAN de management
- 06. [A][15p] Configurare legături de tip trunchi cascadate
- 07. [A][15p] Depanare probleme în lucrul cu VLAN-uri
- 08. [A][BONUS - 10p] VLAN nativ
- 09. [A][BONUS - 10p] Filtrare VLAN-uri pe legătura de tip trunchi

Exerciții

00. [A][5p] VLAN-uri în Cisco IOS

Topologie: download

- Rețelele locale virtuale (*Virtual Local Area Networks* – VLANs) sunt moduri de a realiza o separație logică a unei rețele locale (LAN) în mai multe subrețele pe aceeași infrastructură fizică.
- Separația este realizată la nivelul Legătură de Date prin introducerea unui câmp suplimentar în antetul de nivel 2. VLAN-urile sunt identificate în cadrul frame-ului printr-un VLAN ID.

- Configurarea VLAN-urilor se realizează pe switch-uri, mai exact pe interfețele/porturile switch-urilor.
- Stațiile nu au cunoaștere despre existența unor VLAN-uri; perspectiva lor este aceea a unei rețele locale, adică rețeaua virtuală aferentă unui VLAN ID.
- O stație se va găsi în VLAN-ul specific portului la care este conectată (configurație existentă pe switch).

- În topologie stațiile `PC0` și `PC2` fac parte din VLAN-ul `10`, iar stațiile `PC1` și `PC3` din VLAN-ul `20`. Observați că pot comunica doar două câte două, deși adresele lor IP sunt din același spațiu de adrese.
- Configurarea celor două VLAN-uri a fost realizată pe switch-ul `Switch0`.

- Pentru a investiga configurația de VLAN-uri de pe switch-ul `Switch0`, rulați comenzile de mai jos și urmăriți output-ul acestora. Pentru a accesa intefața de configurare a switch-ului, urmații pașii:
  1. Dați click pe switch.
  2. Accesați tab-ul `CLI`.
  3. Apăsați tasta `Enter` pentru a apărea promptul de transmitere de comenzi pentru switch.
  4. Folosiți comanda `enable` (urmată de apăsarea tastei `Enter`) pentru a accesa modul privilegiat de configurare a switch-ului.

- Afișați configurația vlan-urilor.
- Urmăriți că VLAN-ul cu ID-ul `10` (denumit și `zece`) conține porturile `Fa0/1` și `Fa2/1` adică porturile aferente stațiilor `PC0` și `PC2`. La fel, VLAN-ul cu ID-ul `20` (denumit și `douăzeci`) conține porturile `Fa1/1` și `Fa3/1` adică porturile aferente stațiilor `PC1` și `PC3`.
- Afisati configurația curentă. Din output-ul comenzii observați că porturile sunt configurate în modul acces în VLAN-urile respective.

01. [A][10p] VLAN de management

Topologie: download

- În topologie se află o rețea cu un switch (`Switch0`) și trei stații. Toate stațiile sunt conectate una la cealaltă și inclusiv la switch-ul `Switch0` (și switch-ul are adresă IP).

> În Cisco IOS primul pachet trimis poate să nu fie de fapt trimis din cauza absenței tabelei ARP. De aceea când trimiteți un pachet în Packet Tracer la sau de la switch, prima oară nu va funcționa. Următoarele pachete, însă, vor funcționa. Detalii aici.

- În general un switch poate fi configurat direct în consola acestuia (dați click pe el și selectați tab-ul CLI) sau de la distanță conectându-vă la switch folosind comanda `telnet` urmată de adresa IP a acestuia.
- Conectați-vă la switch-ul `Switch0` de pe fiecare dintre cele 3 stații urmând pașii:
  1. Dați click pe stație.
  2. Alegeți tab-ul `Desktop`.
  3. Dați click pe icon-ul `Command prompt`.
  4. Executați comanda `telnet 192.168.1.254`.
- `192.168.1.254` este adresa IP a switch-ului.
- Observați că vă puteți conecta la switch-ul `Switch0` de pe oricare stație.

- Dorim să securizăm accesul la switch-ul `Switch0`, permițând doar stației `Management` să se conecteze la acesta.
- Pentru acest lucru vom configura un nou VLAN, având ID-ul `100`; din acest VLAN va face parte doar stația `Management`. Deseori acest VLAN poartă numele de *VLAN de management*.

- Pentru a configura un VLAN pe un port al switch-ului, urmăm pașii:
  1. În consola de configurare activam modul privilegiat (folosiți parola `student`).
  2. Intrăm în modul de configurare.
  3. Creăm VLAN-ul cu ID-ul 100: `vlan 100`
  4. Opțional putem configura și un nume pentru VLAN, să zicem `management`.
  5. Intrăm în modul de configurare al interfaței relevante.
  6. Configurăm interfața/portul pentru modul acces.
  7. Configurăm pe interfața de tip acces numărul VLAN-ului.
- În acest moment stația `Management` se află în VLAN-ul `100`.
- Trebuie să configurăm și switch-ul pentru a răspunde cererilor de configurare tot pe VLAN-ul `100`.
- Pentru a putea fi configurat corespunzător, unui switch i se pot crea interfețe virtuale de forma `vlan X`, unde X este numărul VLAN-ului de pe care poate fi accesat în vederea configurării.
- Interfața implicită pentru orice switch este `vlan 1` pe care va trebui să o dezactivăm și să ștergem adresa IP
- Configurați adresa IP pe interfața `vlan 100` aferentă VLAN-ului `100`.
- După această configurare, switch-ul `Switch0` va fi accesibil la adresa IP `192.168.1.254` doar de pe stația `Management`.
- Folosiți comanda `telnet 192.168.1.254` din interfața CLI a fiecărei stații pentru a verifica faptul că vă puteți conecta sau nu la switch-ul `Switch0`. Observați că doar de pe stația `Management` poate fi realizată conectarea.

> Conectivitatea între stații s-a păstrat. Nu a fost afectată de configurarea VLAN-ului de management.

02. [A][10p] Porturi în modul access

Topologie: download

- În topologie se află o rețea cu un switch, o stație de management și 4 stații (`PC1`, `PC2`, `PC3`, `PC4`) folosite de utilizatori. Observați că cele 4 stații pot comunica între ele fiecare cu fiecare.

- Dorim să izolăm `PC1` și `PC3` de celelalte două stații (`PC2` și `PC4`) astfel încât `PC1` să poată comunica doar cu `PC3`.
- Acest lucru se poate realiza configurând porturile aferente lui `PC1` și `PC3` să facă parte din VLAN-ul `10`, iar porturile aferente lui `PC2` și `PC4` să facă parte din VLAN-ul `20`.
- Verificati adăugarea celor două VLAN-uri.

- După ce am creat VLAN-urile în baza de date a switch-ului, trebuie să configurăm porturile să facă parte din aceste VLAN-uri, după cum urmează:
  - `Fa1/1` - `PC1` - VLAN `10`
  - `Fa2/1` - `PC2` - VLAN `20`
  - `Fa6/1` - `PC3` - VLAN `10`
  - `Fa3/1` - `PC4` - VLAN `20`
- Înainte de a fi configurate VLAN-urile, porturile trebuie trecute în mod acces.
- Verificați că VLAN-ul `10` conține porturile `Fa1/1` și `Fa6/1`, iar VLAN-ul `20` conține porturile `Fa2/1` și `Fa3/1`.
- Verificați conectivitatea între fiecare pereche de două stații. Observați faptul că doar stațiile din același VLAN pot să comunice între ele.

03. [A][15p] Subnetare și VLAN-uri

Topologie: download

- Topologia porneste de la topologie de la exercitiul 2 in care am mai adăugat un switch (`Switch1`) la care au fost conectate alte 4 stații (`PC5`, `PC6`, `PC7`, `PC8`).
- Configurați switch-ul `Switch1` astfel încât `PC5` și `PC7` să facă parte din VLAN-ul `10`, iar celelalte 2 (`PC6` și `PC8`) din VLAN-ul `20`.

- Împărțiți spațiul `172.16.234.96/27` în două subrețele de dimensiuni egale și configurați stațiile (`PC1`, `PC3`, `PC5`, `PC7`) din VLAN-ul `10` cu adrese IP din prima subrețea obținută, iar cele din VLAN-ul `20` (`PC2`, `PC4`, `PC6`, `PC8`) cu adrese IP din a doua subrețea.
- Verificați conectivitatea între stațiile din același VLAN, două câte două, precizând de ce nu funcționează atunci când este cazul.

> Nu verificați conectivitatea între două stații din switch-uri diferite întrucât între aceste switch-uri NU există legătură fizică.

04. [A][10p] Legătură de tip trunchi (trunk)

- Pe topologia de la exercițiul 03. [A][15p] Subnetare și VLAN-uri, realizați o legătură de fibră între switch-urile `Switch0` și `Switch1` pe portul `Fa4/1` al fiecărui switch.
- Testați conectivitatea între stații din același VLAN, dar switch-uri diferite. Observați că nu există conectivitate din cauză că nu există nici un mecanism activat prin care VLAN-urile de pe switch-uri diferite să comunice între ele.

- Pentru a permite conectivitatea între stații aflate în același VLAN, dar conectate prin switchuri diferite, trebuie să configurăm legătura dintre switch-uri în mod trunchi (*trunk*).
- Această legătură permite încapsularea pachetelor cu VLAN-uri diferite.
- Identificați numărul portului de interconectare pe fiecare din switch-uri. Pe fiecare switch, intrați pe interfața aferentă și setați legătura în mod trunchi.

- Pe switch-ul `Switch0` configurați interfața `Fa4/1` in mod trunk si permiteți accesul tuturor vlan-urilor.
- Pe switch-ul `Switch1` configurați interfața `Fa4/1` în mod trunk si permiteti accesul tuturor vlan-urilor.
- Investigați configurația de tip trunk pe ambele switch-uri.
- Se observă că interfața `Fa4/1` este o interfață de tip trunk care transportă VLAN-urile 1 (cel implicit), `10` și `20`. În cazul switch-ului `Switch0` transferă și VLAN-ul de management (`100`). Aceasta se întâmplă întrucât ultimul argument al comenzii `switchport trunk …` a fost `all`, reprezentând toate VLAN-urile.
- Verificați că stațiile din același VLAN pot comunica între ele indiferent de switch-ul la care sunt interconectate.

05. [A][20p] Extindere VLAN de management

- În topologia configurată la 04. [A][10p] Legătură de tip trunchi (trunk) dorim să configurăm switch-ul `Switch1` ca să poată fi accesat prin `telnet` de pe stația `Management`.
- Pentru aceasta configurați adresa IP `192.168.1.252` pe switch-ul `Switch1` pe interfața aferentă VLAN-ului de management (`vlan 100`).
- Testați prin conectarea prin `telnet` de pe stația `Management` la `Switch1`.

> Salvați topologia rezolvată dacă vreți să faceți exercițiile bonus, vă va trebui! ;)

06. [A][15p] Configurare legături de tip trunchi cascadate

Topologie: download

- Ne propunem să construim rețele locale virtuale (VLAN-uri) pe o topologie cu mai multe switch-uri. Vom folosi legături de tip trunchi între switch-uri.
- Topologia conține 4 stații (`PC0`, `PC1`, `PC2`, `PC3`) și trei switch-uri (`Switch0`, `Switch1`, `Switch2`).
- Cele patru stații au deja adresele IP configurate și trebuie să se găsească două câte două în VLAN-uri:
  - VLAN `10`: `PC0` și `PC2`
  - VLAN `20`: `PC1` și `PC3`

- Pentru început, după ce switch-urile rulează STP, verificați conectivitatea între **toate** cele patru stații.
- Nefiind realizate configurații de VLAN pe switch-uri, stațiile pot comunica între ele, oricare două.

- Configurați switch-urile astfel încât stațiile să poată comunica doar în cadrul VLAN-ului propriu (să existe separație la nivelul 2).
- Verificați transmițând pachete între oricare două stații.

07. [A][15p] Depanare probleme în lucrul cu VLAN-uri

Topologie: download

- Ne propunem să discutăm despre cum depanăm problemele în lucrul cu VLAN-uri.
- Topologia conține cinci stații care se găsesc în două VLAN-uri, astfel:
  - stațiile `PC0`, `PC1` și `PC3` se găsesc în VLAN-ul `10`
  - stațiile `PC2`, `PC4` se găsesc în VLAN-ul `20`

- Pentru început verificați de ce nu există conectivitate între stațiile `PC0` și `PC1`, deși se găsesc în același switch. Investigați porturile switch-ului `Switch0`.
- Verificați de ce nu există conectivitate între stațiile `PC2` și `PC4`. Investigați configurația pentru VLAN-ul de tip trunchi de pe switch-ul `Switch1`.

> Pentru depanare urmăriți configurația curentă a switch-ului cu ajutorul comenzii
>
> ```
> show running-config
> ```

08. [A][BONUS - 10p] VLAN nativ

- Vom lucra cu noțiunea de VLAN nativ, care permite transferul fără tag VLAN într-o legătură de tip trunchi.
- Folosiți topologia de la 05. [A][20p] Extindere VLAN de management. Urmăriți formatul cadrelor transmise între stații din VLAN-ul `10` conectate la switch-uri diferite.

- Configurați apoi VLAN-ul `10` ca VLAN nativ pentru legătura trunchi dintre `Switch0` și `Switch1` și urmăriți din nou cadrele transmise între stații din VLAN-ul `10` conectate la switch-uri diferite.
- Dacă ați realizat configurația corectă, cadrele nu vor avea tag VLAN (*dot1q*), fiind vorba de VLAN-ul nativ.

> Puteți urmări formatul cadrelor în modul `Simulation` din Packet Tracer.

> Pentru documentație legată de configurarea unui VLAN nativ, urmăriți această secțiune.

09. [A][BONUS - 10p] Filtrare VLAN-uri pe legătura de tip trunchi

- Folosiți topologia de la 05. [A][20p] Extindere VLAN de management și configurați legătura de tip trunchi între switch-ul `Switch0` și `Switch1` să nu permită încapsularea pachetelor din VLAN-ul `20` (adică pachetele din VLAN-ul `20` să nu mai fie trimise pe legătura de trunchi).
- Va trebui permisa doar incapsularea pachetelor din VLAN-urile `10` si `100`.
- Astfel stațiile din VLAN-ul `20` conectate la switch-ul `Switch0` nu mai pot comunica cu cele din VLAN-ul `20` conectate la switch-ul `Switch1`.
- Stația `Management` trebuie să poată în continuare să acceseze `Switch1`, iar stațiile din VLAN-ul `10` să poată comunica indiferent de switch-ul la care sunt conectate.

> Pentru documentație legată de configurarea VLAN-urilor permise, urmăriți această secțiune.

Navigare

Laboratorul 4

- 00. [A][5p] VLAN-uri în Cisco IOS
- 01. [A][10p] VLAN de management
- 02. [A][10p] Porturi în modul access
- 03. [A][15p] Subnetare și VLAN-uri
- 04. [A][10p] Legătură de tip trunchi (trunk)
- 05. [A][20p] Extindere VLAN de management
- 06. [A][15p] Configurare legături de tip trunchi cascadate
- 07. [A][15p] Depanare probleme în lucrul cu VLAN-uri
- 08. [A][BONUS - 10p] VLAN nativ
- 09. [A][BONUS - 10p] Filtrare VLAN-uri pe legătura de tip trunchi

Laborator

Laboratorul 4

- 00. [A][5p] VLAN-uri în Cisco IOS
- 01. [A][10p] VLAN de management
- 02. [A][10p] Porturi în modul access
- 03. [A][15p] Subnetare și VLAN-uri
- 04. [A][10p] Legătură de tip trunchi (trunk)
- 05. [A][20p] Extindere VLAN de management
- 06. [A][15p] Configurare legături de tip trunchi cascadate
- 07. [A][15p] Depanare probleme în lucrul cu VLAN-uri
- 08. [A][BONUS - 10p] VLAN nativ
- 09. [A][BONUS - 10p] Filtrare VLAN-uri pe legătura de tip trunchi

Curs

Informații generale RL

- **Regulament**
  - Notare Seria CA
  - Notare Seriile CB/CC
- **Resurse**
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

rl/labs/04.txt · Last modified: 2020/10/10 23:24 by octavian.grigorescu

(cc) BY-SA CHIMERIC DE W3C CSS DOKUWIKI GET FIREFOX RSS XML FEED W3C XHTML 1.0

Laborator 5. Rutare

Cunoștințe și abilități ce vor fi dobândite

- Procesul de rutare, utilitatea procesului de rutare
- Rute, părțile componente ale unei rute: *rețea destinație*, *next hop*, *default gateway*
- Adăugarea de rute statice pe Cisco IOS

Cheat sheet

- Cheat Sheet

Navigare

Laboratorul 5

- 00. [M][5p] Procesul de rutare
- 01. [A][10p] Configurare default gateway
- 02. [M][10p] Antetul pachetelor în procesul de rutare
- 03. [A][5p] Rute și pachete de reply
- 04. [A][10p] Adresare și default gateway
- 05. [A][10p] Configurare rute statice
- 06. [M][5p] Vizualizare tabelă de rutare
- 07. [A][15p] Adresare și configurare rute statice
- 08. [A][10p] Depanare probleme de configurare a rutării
- 09. [A][10p] Adresare și configurare rute statice (reiterare)
- 10. [A][10p] Rute implicite pe rutere
- 11. [A][BONUS - 10p] Adresare și configurare rute statice (avansat)
- 12. [A][BONUS - 10p] Rutare inter-VLAN

Exerciții

00. [M][5p] Procesul de rutare

- Pentru ca un dispozitiv să poată comunica în afara rețelei locale, trebuie să cunoască modul în care ajunge la acea rețea.
- Acest mod este specificat prin **rute**. O rută descrie unui dispozitiv ce cale să urmeze pentru a ajunge la rețeaua destinație.

- Procesul de rutare constă în primirea unui pachet, investigarea tabelei de rutare, descoprirea rutei corespunzătoare și rutarea pachetului, adică transmiterea pachetului mai departe conform rutei.

- Rutele se găsesc în tabela de rutare și constau din două elemente:
  1. partea de match: adresa de rețea destinație (adresă și mască de rețea)
  2. partea de acțiune: următorul dispozitiv de rutare (*next hop*) (sau interfața de ieșire)

- În momentul în care un dispozitiv care rutează (un **ruter**) primește un pachet, extrage adresa IP destinație, localizează adresa de rețea destinație în tabela de rutare și apoi **rutează** (dirijează, retransmite) pachetul către următorul ruter (*next hop*).
- Procesul se reia pe următorul ruter până când pachetul ajunge la destinație.
- Rutarea este un proces care are loc la nivelul 3 (Rețea) din stiva OSI, lucrând cu adresa IP.

01. [A][10p] Configurare default gateway

Topologie: download

- În topologie se află două rețele interconectate printr-un ruter.
- Observați că există conectivitate între ruter și oricare stație (folosiți comanda `ping` de pe ruter sau mesajele din Packet Tracer).

> În Cisco IOS primul pachet trimis poate să nu fie de fapt trimis din cauza absenței tabelei ARP. De aceea când trimiteți un pachet în Packet Tracer la sau de la ruter, prima oară nu va funcționa. Următoarele pachete, însă, vor funcționa. Detalii aici.

- Observați că **nu** există conectivitate între stații din rețele diferite, deși adresele IP și măștile de rețea sunt configurate pe ruter și pe toate stațiile.
- Nu există conectivitate pentru că o stație nu are configurată rută către rețeaua cealaltă. Pentru a asigura conectivitate între stații din rețele diferite, va trebui să adăugăm, pe fiecare stație, rută către cealaltă rețea având ca next hop ruterul. Vom realiza în continuare acest lucru – conectivitatea între toate stațiile.

- Pentru a asigura conectivitate între toate stațiile, indiferent din ce rețea fac parte trebuie să configurăm o rută implicită (*default gateway*) pe fiecare stație.
- Pașii urmați vor fi:
  1. Determinăm adresele IP ale interfețelor ruterului.
  2. Pentru fiecare stație, adăugăm o rută implicită (de tip *default gateway*) având ca next hop adresa IP a interfeței ruterului din rețeaua din care face parte stația.
  3. Folosim comanda `ping` de pe ruter sau mesajele din Packet Tracer pentru a vedea conectivitatea între oricare două stații, chiar aflate în rețele diferite.

- Pentru început, vrem să aflăm adresele IP configurate pe fiecare interfață a ruterului.
  1. Dați click pe `Router0`.
  2. Accesați tab-ul `CLI`.
  3. Apăsați tasta `Enter` pentru a accesa promptul de comenzi.
  4. Folosiți comanda `enable` (urmată de apăsarea tastei `Enter`) pentru a accesa modul privilegiat de configurare a ruterului.
  5. Afișați adresele IP configurate pe interfețele ruterului.
- Interfața `FastEthernet0/0` este conectată la rețeaua din care fac parte stațiile `PC0` și `PC1`. Adresa IP a acestei interfețe este `172.16.100.1`.

- Pentru a asigura conectivitate cu stațiile din cealaltă rețea, pe stațiile `PC0` și `PC1` vom configura o rută implicită (de tip *default gateway*) având ca next hop adresa `172.16.100.1`.
- Pentru a configura *default gateway*, faceți click în Packet Tracer pe stația dorită (`PC0` sau `PC1`), mergeți pe tab-ul `Desktop` și apăsați pe iconița `IP Configuration`. În zona de text cu eticheta *Default gateway* completați adresa `172.16.100.1`.
- În mod similar, configurați pe stațiile `PC2` și `PC3` adresa `192.168.100.1` (adresa interfeței `FastEthernet1/0`) pe post de *default gateway*.
- Observați conectivitatea între oricare două stații indiferent de rețeaua de care aparțin.

02. [M][10p] Antetul pachetelor în procesul de rutare

- Vrem să urmărim antetul pachetelor în timpul procesului de rutare. Vom trimite un pachet între stații din rețele diferite și vom observa modificarea antetului.
- Folosiți în continuare topologia de la 01. [A][10p] Configurare default gateway. Intrați în modul `Simulation` și trimiteți un mesaj Packet Tracer de la stația `PC0` la stația `PC2`. Urmăriți antetul celor două pachete transmise:
  - pachetul trimis de stația `PC0` și primit de ruter (îl vom denumi `pkt1`)
  - pachetul trimis de ruter și primit de stația `PC2` (îl vom denumi `pkt2`)

- Pentru a urmări antetul unui pachet folosiți lista de evenimente din dreapta spațiului de lucru (`Event list`) și dați click pe pătrățelul colorat al fiecărui eveniment din listă.
- Observați următoarele efecte ale procesului de rutare.
  - Pachetul `pkt1` are ca adresă MAC sursă adresa MAC a stației `PC0`, iar ca adresă MAC destinație adresa MAC a interfeței `FastEthernet0/0` a ruterului.
  - Pachetul `pkt2` are ca adresă MAC sursă adresa MAC a interfeței `FastEthernet1/0` a ruterului, iar ca adresă MAC destinație adresa MAC a stației `PC2`.
  - Se decrementează câmpul `TTL` (*Time To Live*) din antetul IP în pachetul `pkt2` față de pachetul `pkt1`.
  - Adresa IP sursă, respectiv adresa IP destinație rămân nemodificate între cele două pachete.
- În concluzie, procesul de rutare folosește adresele MAC specifice fiecărei rețele locale în care este trimis pachetul, lasă nemodificate adresele IP (sursă și destinație) și decrementează câmpul `TTL` (un contor al numărului de hop-uri).

> Decrementarea câmpului `TTL` are rolul de a atenua efectele unor posibile bucle de rutare. În momentul în care câmpul `TTL` ajunge la valoarea `0` pachetul este aruncat (*dropped*).

03. [A][5p] Rute și pachete de reply

- Vrem să urmărim de ce sunt necesare rute pe ambele stații care comunică din rețele diferite. Vom folosi în continuare topologia de la 01. [A][10p] Configurare default gateway.

- Intrați în modul `Simulation` și trimiteți un mesaj Packet Tracer de la stația `PC0` la stația `PC2`. Observați drumul parcurs de mesaj.
- Pachetul urmează drumul `PC0` → ruter → `PC2` → ruter → `PC0`. Este vorba de un pachet transmis și un pachet de răspuns (de tip *reply*).

- Ștergeți de pe stația `PC2` configurația de *Default Gateway*.
- In modul `Simulation`, trimiteți încă o dată un mesaj Packet Tracer de la stația `PC0` la stația `PC2`. Observați drumul parcurs de mesaj.
- Observați că pachetul este aruncat (*dropped*) de stația PC2. De ce se întâmplă acest lucru și nu se trimite pachet de răspuns (de tip *reply*)?

> Înainte de a trece la exercițiul următor, refaceți configurația de *Default Gateway* pe stația `PC2`.

04. [A][10p] Adresare și default gateway

- Dorim să extindem topologia anterioară cu încă o rețea și să asigurăm conectivitatea stațiilor din această rețea la stațiile existente.
- Folosiți în continuare topologia de la 01. [A][10p] Configurare default gateway. Adăugați un nou switch și conectați-l la una dintre interfețele libere ale ruterului.

> Folosiți un switch de tip `Generic`, `Switch-PT` (**nu** `Switch-PT-Empty`).
>
> Ruterul nu are un port Ethernet suplimentar pentru a conecta noul switch. Pentru conectarea noului switch la ruter, realizați o legătură de fibră optică între switch și ruter (între portul `FastEthernet4/1` al switch-ului și portul `FastEthernet4/0` al ruterului).

- Adăugați două stații noi și conectați-le la noul switch.
- Folosind primele 3 adrese IP asignabile din spațiul `10.0.100.0/24`, configurați noua interfață a ruterului cu prima adresa asignabila și cele două stații proaspăt adăugate (`PC4` si `PC5` cu urmatoarele doua adrese asignabile).

> Un model de configurare al adresei IP pe interfața unui ruter îl găsiți în Laboratorul 3
>
> Pentru a activa o interfață a ruterului, accesați modul de configurare a interfeței și folosiți comanda `no shutdown`.

- Pe cele două stații proaspăt adăugate configurați ca adresă pentru *default gateway* adresa IP a noii interfețe de pe ruter.
- Dacă aveți o configurație corespunzătoare, veți avea conectivitate între toate stațiile din topologie.

05. [A][10p] Configurare rute statice

Topologie: download

- În topologie se află două rețele interconectate prin două rutere cascadate. Fiecare rețea conține un switch și două stații.
- Observați că există conectivitate între stațiile din aceeași rețea și ruterul conectat la acea rețea.
- Observați că nu există conectivitate între stații din rețele diferite. Mai mult nu există conectivate între cele două rutere: trimiteți mesaje Packet Tracer între rutere.

- Pentru a asigura conectivitate completă în cadrul topologie vom realiza următorii pași:
  1. Vom configura adrese IP în rețeaua formată de cele două rutere, adică pe legătura `Router0(Fa1/0)` ↔ `Router1(Fa1/0)`. După acest pas ruterele vor comunica între ele.
  2. Vom configura pe fiecare ruter o rută către rețeaua la care nu are acces. Ruta va trece prin celălalt ruter. În acest moment vom avea conectivitate la nivelul întregii topologii.

- Pentru început observăm că ruterele nu au configurate adrese IP pe interfețele de legătură între ele.
- Vom folosi spațiul `30.30.30.32/30` pentru a aloca două adrese IP în vederea conectării celor două rutere:
  1. Vom configura adresa IP `30.30.30.33` cu masca `255.255.255.252` pe interfața `FastEthernet1/0` a ruterului `Router0`.
  2. Vom configura adresa IP `30.30.30.34` cu masca `255.255.255.252` pe interfața `FastEthernet1/0` a ruterului `Router1`.
- Observați că acum există conectivitate între cele două rutere: puteți folosi comanda `ping` din consola ruterului sau mesaje de tip Packet Tracer. Există de asemenea, conectivitate între stații și ruterul direct conectat la rețeaua lor.

- Nu există însă conectivitate între stațiile din rețele diferite.
- Intrați în modul `Simulation` și observați că pachetele trimise de la `PC0` la `PC2` se opresc la `Router0` întrucât acesta nu știe unde se află rețeaua din care face parte `PC2` (`15.15.15.0/24`). Altfel spus nu are rută către rețeaua `15.15.15.0/24`. Pentru aceasta trebuie să adăugăm pe fiecare ruter o rută către rețeaua la care nu este conectat în mod direct:
  - pe ruterul `Router0` o rută către rețeaua `15.15.15.0/24` având ca next hop ruterul `Router1` (mai precis prin interfața `FastEthernet1/0` a ruterului `Router1`)
  - pe ruterul `Router1` o rută către rețeaua `14.14.14.0/24` având ca next hop ruterul `Router0` (mai precis prin interfața `FastEthernet1/0` a ruterului `Router0`)

- Formatul comenzii de adăugare de rute este următorul:
  - șirul `ip route`
  - adresa de rețea: `15.15.15.0` respectiv `14.14.14.0`
  - masca de rețea: `255.255.255.0`
  - next hop-ul, adică adresa către care trebuie să fie trimise pachetele: `30.30.30.34` respectiv `30.30.30.33`
- Dacă aveți o configurație corespunzătoare, veți avea conectivitate între toate stațiile din topologie.

> În modul `Simulation` puteți observa cum se modifică la fiecare pas cum se modifică adresele MAC ale pachetului (adresa MAC sursă și adresa MAC destinație) și câmpul `TTL`.

06. [M][5p] Vizualizare tabelă de rutare

- Rutele adăugate anterior sunt rute statice: rute adăugate manual de utilizator.
- Vizualizați tabela de rutare (conținând rute statice, dinamice sau direct conectate).
- Fiecare dintre cele două rutere din topologie va avea în tabela de rutare trei rute:
  - două rute **direct conectate** (marcate cu `C`)
  - o rută **statică** (marcată cu `S`)

```
Router1#show ip route
Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     14.0.0.0/24 is subnetted, 1 subnets
S       14.14.14.0 [1/0] via 30.30.30.33
     15.0.0.0/24 is subnetted, 1 subnets
C       15.15.15.0 is directly connected, FastEthernet0/0
     30.0.0.0/30 is subnetted, 1 subnets
C       30.30.30.32 is directly connected, FastEthernet1/0
```

- Rutele direct conectate (marcate cu `C`) sunt adăugate de sistemul de operare la configurarea unei adrese IP pe ruter și asigură conectivitatea la alte stații din rețeaua respectivă.
- Pentru a permite conectivitate la alte rețele (care nu sunt direct conectate), trebuie adăugate alte rute. Aceste rute sunt adăugate de administrator (manual, rute statice) sau de un protocol de rutare (automat, rute dinamice).
- Pe Cisco IOS, rutele statice sunt marcat cu `S`, iar rutele dinamice sunt marcate cu o literă specifică protocolului (`O` pentru OSPF, `R` pentru RIP). În tabela de rutare de mai sus avem o singură rută statică, adăugată manual pentru a permite conectivitatea la rețeaua care nu era direct conectată la ruter.

07. [A][15p] Adresare și configurare rute statice

Topologie: download

- Dorim să configurăm rutarea într-o topologie cu trei rutere.
- Pe moment sunt configurate:
  - adresă IP pe fiecare stație
  - default gateway pe fiecare stație: ruterul din rețea (de fapt adresa IP a ruterului din rețea)
  - adresă IP pe fiecare interfață de ruter dintr-o rețea cu stații: `Router0(Fa0/0)`, `Router1(Fa1/0)`

> Glisați (*hover*) mouse-ul deasupra stațiilor și ruterelor pentru a afla adresa IP a fiecărei stații și a interfeței configurate a fiecărui ruter.

- Folosiți `ping` de pe fiecare din ruterele cu interfețele în rețele cu stații (`Router0` și `Router1`) pentru verificarea conectivității la stațiile din rețea. La fel, pentru stații, folosiți `ping` pentru a verifica conectivtatea la ruterul din rețea și la cealaltă stație.
- Pentru stații, pentru a folosi `ping`, accesați tab-ul `Desktop` și opțiunea `Command Prompt`.

- Veți realiza următori pași:
  1. Veți configura adrese IP pe cele două rețele dintre rutere (`Router0(Fa1/0)` ↔ `Router2(Fa0/0)`, `Router2(Fa1/0)` ↔ `Router1(Fa0/0)`).
  2. Veți adăuga rutele necesare pe fiecare ruter (vor trebui adăugate câte două rute).
  3. Veți verifica faptul că toate stațiile sunt conectate cu ajutorul comenzii `ping` și veți depana problemele întâlnite.

- Pentru cele două rețele dintre rutere configurați adresele astfel:
  1. Pentru rețeaua `Router0(Fa1/0)` ↔ `Router2(Fa0/0)` configurați adrese din spațiul `4.4.4.0/30` (prima adresa asignabila pe `Router0(Fa1/0)`, a doua pe `Router2(Fa0/0)`).
  2. Pentru rețeaua `Router2(Fa1/0)` ↔ `Router1(Fa0/0)` configurați adrese din spațiul `4.4.4.4/30` (prima adresa asignabila pe `Router2(Fa1/0)`, a doua pe `Router1(Fa0/0)`).

- Pentru a asigura conectivitate între toate stațiile va trebui să adăugați rute statice pe rutere.
- Întrucât există patru rețele în topologie, fiecare ruter va avea în final patru rute. Momentan, fiecare ruter are câte două rute, ambele direct conectate. Verificați acest lucru prin rularea comenzii `show ip route`.

- Vor trebui adăugate pe fiecare ruter câte două rute statice către rețelele care nu sunt direct conectate. Veți adăuga astfel:
  - pe ruterul `Router0`
    1. o rută către rețeaua `Router2(Fa1/0)` ↔ `Router1(Fa0/0)` (adică `4.4.4.4/30`) având ca next hop adresa IP a interfeței `Fa0/0` a ruterului `Router2`
    2. o rută către rețeaua stațiilor `PC2` și `PC3` (adică `172.16.100.0/24`) având ca next hop **tot** adresa IP a interfeței `Fa0/0` a ruterului `Router2`
  - pe ruterul `Router1`
    1. o rută către rețeaua `Router0(Fa1/0)` ↔ `Router2(Fa0/0)` (adică `4.4.4.0/30`) având ca next hop adresa IP a interfeței `Fa1/0` a ruterului `Router2`
    2. o rută către rețeaua stațiilor `PC0` și `PC1` (adică `192.168.100.0/24`) având ca next hop **tot** adresa IP a interfeței `Fa1/0` a ruterului `Router2`
  - pe ruterul `Router2`
    1. o rută către rețeaua stațiilor `PC0` și `PC1` (adică `192.168.100.0/24`) având ca next hop adresa IP a interfeței `Fa1/0` a ruterului `Router0`
    2. o rută către rețeaua stațiilor `PC2` și `PC3` (adică `172.16.100.0/24`) având ca next hop adresa IP a interfeței `Fa0/0` a ruterului `Router1`

- Verificați ca, în final, fiecare ruter să aibă patru rute: două rute direct conectate și două rute statice conform configurației.

> Dacă ați greșit o rută, folosiți `no` în fața comenzii de adăugare de rute. De exemplu, dacă dorim să ștergem ruta către rețeaua `10.0.100.0/24` având ca next hop adresa IP `3.3.3.3`, vom folosi comanda:
>
> ```
> Router(config)#no ip route 10.0.100.0 255.255.255.0 3.3.3.3
> ```
>
> Puteți căuta în istoria de comenzi, găsi comanda cu care ați adăugat ruta greșită și pune `no` în fața comenzii pentru a anula efectul acesteia.

- După o configurare corectă veți avea conectivitate între toate stațiile din topologie.
- Pentru verificare folosiți comanda `ping` din meniul unei stații și transmiteți pachete către celelalte stații.

> În Cisco IOS primul pachet trimis poate să nu fie de fapt trimis din cauza absenței tabelei ARP. De aceea este posibil ca un set întreg de pachete trimise la rularea unei comenzi `ping` pe stație (sunt trimise patru în mod implicit) să eșueze, întrucât traversează multe rutere. A doua comandă `ping` pe stație va reuși, în cazul unei configurări corecte.

- Dacă aveți probleme de conectivitate, verificați din aproape în aproape între ce dispozitive aveți sau nu conectivitate.
- Verificare din aproape în aproape înseamnă folosirea comenzii `ping` către adrese IP aflate la început aproape și apoi din ce în ce mai departe de stația care le-a generat.
- O verificare din aproape în aproape a conectivității între stațiile `PC0` și `PC2` înseamnă folosirea stația `PC0` a comenzii `ping` cu următoarele argumente:
  1. adresa IP a interfeței `Fa0/0` a ruterului `Router0`, adică `192.168.100.1`
  2. adresa IP a interfeței `Fa1/0` a ruterului `Router0`, adică o adresă din rețeaua `4.4.4.0/30`
  3. adresa IP a interfeței `Fa0/0` a ruterului `Router2`, adică cealaltă adresă din rețeaua `4.4.4.0/30`
  4. adresa IP a interfeței `Fa1/0` a ruterului `Router2`, adică o adresă din rețeaua `4.4.4.4/30`
  5. adresa IP a interfeței `Fa0/0` a ruterului `Router1`, adică cealaltă adresă din rețeaua `4.4.4.4/30`
  6. adresa IP a interfeței `Fa1/0` a ruterului `Router1`, adică `172.16.100.1`
  7. adresa IP a stației `PC2`, adică `172.16.100.2`
- În caz de probleme, urmăriți pe rutere adresele IP configurate și tabela de rutare pentru fiecare ruter și identificați cauza problemei.

> Nu ștergeți fișierul pkt după finalizarea exercițiului. Veți avea nevoie de el mai departe.

08. [A][10p] Depanare probleme de configurare a rutării

Topologie: download

- În topologie se află două rețele interconectate prin trei rutere cascadate. Există câteva probleme în cadrul toplogiei pe care ne propunem să le rezolvăm.
- Pentru început, aflați adresele stațiilor și a interfețelor ruterelor glisând mouse-ul (*hover*) peste fiecare dispozitiv.

- De pe stația `PC0` verificați conectivitatea la stația `PC2`, adică adresa IP `15.15.15.2`. Comanda `ping` oferă răspunsul `Destination host unreachable.`, deci există o problemă pe parcurs.
- De pe stația `PC0` verificați conectivitatea la ruterul `Router2`, adică adresa IP `15.15.15.1`. Comanda `ping` nu reușește. Înseamnă că există o problemă la un ruter pe parcurs.
- Afișați tabela de rutare a ruterului `Router0`.
- Observați că ruterul `Router0` nu are rută către rețeaua `15.15.15.0/24`, rețea care conține stațiile `PC2` și `PC3`. Adăugați ruta corespunzătoare.

- Pe stația `PC0` verificați conectivitatea la stația `PC2` (adresa IP `15.15.15.2`) și la stația `PC3` (`15.15.15.254`). Există conectivitate, deci această problemă este rezolvată.
- Să verificăm și că ruterele au, de asemenea, conectivitate la toate celelalte rutere și la stații. De pe ruterul `Router0` verificăm conectivitatea la stația `PC2` (adresa IP `15.15.15.2`).
- Observați că nu există conectivitate. Pare ciudat, pentru că stația `PC0` are conectivitate la stația `PC2` dar ruterul `Router0` nu are.
- În continuare, verificăm conectivitatea la ruterul `Router2` (adresele `15.15.15.1` și `30.30.30.38`) și la ruterul `Router1` (adresele `30.30.30.37` și `30.30.30.34`).
- Observați că avem conectivitate între ruterul `Router0` și ruterul `Router1`, dar nu între ruterul `Router0` și ruterul `Router2`.

- Investigați tabela de rutare a ruterului `Router1` și a ruterului `Router2`.
- Observați că ruterul `Router2` nu are rută către rețeaua `30.30.30.32/30`, rețeaua dintre ruterele `Router0` și `Router1`. Adăugați ruta corespunzătoare.
- Verificați că în acest moment există conectivitate între toate dispozitivele (stații și rutere) din topologie.

09. [A][10p] Adresare și configurare rute statice (reiterare)

- Dorim să suplimentăm tabelele de rutare de la 07. [A][15p] Adresare și configurare rute statice. Pentru aceasta, în topologia (funcțională) de la 07. [A][15p] Adresare și configurare rute statice adăugați un switch și două stații și conectați switch-ul la interfața `Fa2/0` a ruterului `Router2`.
- Atribuiți celor două stații și interfeței `Fa2/0` a ruterului `Router2` adrese IP din rețeaua `10.0.100.0/24` (prima adresa asignabila pentru interfata de pe ruter). Apoi asigurați conectivitate cu restul stațiilor din topologie.

- Întrucât topologia conține acum cinci rețele, fiecare ruter va trebui să aibă cinci rute.
- Va trebui să adăugați încă o rută pe ruterele `Router0`, respectiv `Router1` către rețeaua `10.0.100.0/24`, rută ce va avea ca next hop adresa IP a interfeței corespunzătoare a ruterului `Router2`.
- După o configurare corectă veți avea conectivitate între toate stațiile din topologie. Pentru verificare folosiți comanda `ping` din meniul unei stații și transmiteți pachete către celelalte stații.

> În Cisco IOS primul pachet trimis poate să nu fie de fapt trimis din cauza absenței tabelei ARP. De aceea este posibil ca un set întreg de pachete trimise la rularea unei comenzi `ping` pe stație (sunt trimise patru în mod implicit) să eșueze, întrucât traversează multe rutere. A doua comandă `ping` pe stație va reuși, în cazul unei configurări corecte.

- Dacă aveți probleme de conectivitate, verificați din aproape în aproape între ce dispozitive aveți sau nu conectivitate. În caz de probleme, urmăriți pe rutere adresele IP configurate și tabela de rutare pentru fiecare ruter și identificați cauza problemei.

10. [A][10p] Rute implicite pe rutere

- Dorim să simplificăm configurarea rutelor în topologia de la 07. [A][15p] Adresare și configurare rute statice, actualizată la 09. [A][10p] Adresare și configurare rute statice (reiterare), prin folosirea rutelor implicite (*default route* și *default gateway*).
- Pentru aceasta, continuați pe topologia (funcțională) de la 07. [A][15p] Adresare și configurare rute statice, actualizată la 09. [A][10p] Adresare și configurare rute statice (reiterare).
- O rută implicită este ruta care va fi aleasă pentru toate pachetele dacă nu există o altă rută care se potrivește.

- Pentru aceasta veți urma doi pași:
  1. Pe ruterele `Router0` și `Router1` veți șterge cele trei rute statice adăugate anterior (două la 07. [A][15p] Adresare și configurare rute statice și una la 09. [A][10p] Adresare și configurare rute statice (reiterare)).
  2. Pe ruterele `Router0`, respectiv `Router1` veți configura rute implicite pe ruterele având ca next hop ruterul `Router2`
- Puteți căuta în istoria de comenzi, găsi comanda cu care ați adăugat ruta greșită și pune `no` în fața comenzii pentru a anula efectul acesteia.

- Adăugarea rutelor implicite înseamnă in mod detaliat:
  1. Pe ruterul `Router0` veți adăuga rută implicită având ca next hop adresa IP a interfeței `Fa0/0` a ruterului `Router2`.
  2. Pe ruterul `Router1` veți adăuga rută implicită având ca next hop adresa IP a interfeței `Fa1/0` a ruterului `Router2`.
- O rută implicită se configurează prin specificarea rutei către rețeaua `0.0.0.0/0` (sau rețeaua `0.0.0.0` cu masca `0.0.0.0`).

> Detalii despre configurarea rutei implicite (*default gateway*) în Cisco IOS se găsesc aici.

- O configurație corectă va înseamna conectivitate între toate stațiile din rețea. Dacă întâmpinați probleme:
  1. Verificați din aproape în aproape conectivitatea între dispozitive (stații sau rutere) și identificați dispozitivul cu probleme.
  2. Verificați adresele IP de pe ruterul problematic afisând detalii despre interfețele configurate pe acesta.
  3. Verificați tabele de rutare de pe ruterul problematic.

11. [A][BONUS - 10p] Adresare și configurare rute statice (avansat)

Topologie: download

- În topologie se află trei rețele interconectate prin patru rutere în stea. Dorim să asigurăm conectivate completă între toate stațiile.
- Pe moment sunt configurate:
  - adresă IP pe fiecare stație
  - default gateway pe fiecare stație: ruterul din rețea (de fapt adresa IP a ruterului din rețea)
  - adresă IP pe fiecare interfață de ruter dintr-o rețea cu stații: `Router0(Fa1/0)`, `Router1(Fa1/0)`, `Router2(Fa1/0)`

> Glisați (*hover*) mouse-ul deasupra stațiilor și ruterelor pentru a afla adresa IP a fiecărei stații și a interfeței configurate a fiecărui ruter.

- Folosiți `ping` de pe fiecare din ruterele cu interfețele în rețele cu stații (`Router0`, `Router1` și `Router2`) pentru a valida conectivitatea la stațiile din rețeaua proprie.

- Pentru cele trei rețele dintre rutere (`Router0` ↔ `Router3`, `Router1` ↔ `Router3`, `Router2` ↔ `Router3`) împărțiți spațiul `32.32.32.32/28` în rețele cu masca `/30` (adică `255.255.255.252`) astfel:
  - prima subretea obtinuta pentru reteaua dintre `Router0` ↔ `Router3` (prima adresa asignabila pe interfata de pe `Router0`, a doua pe interfata de pe `Router3`)
  - a doua subretea obtinuta pentru reteaua dintre `Router1` ↔ `Router3` (prima adresa asignabila pe interfata de pe `Router1`, a doua pe interfata de pe `Router3`)
  - a treia subretea obtinuta pentru reteaua dintre `Router2` ↔ `Router3` (prima adresa asignabila pe interfata de pe `Router2`, a doua pe interfata de pe `Router3`)

Pentru a asigura conectivitate între toate stațiile va trebui să adăugați rute statice pe rutere. Recomandăm să adăugați astfel:

- pe ruterul `Router0` o rută implicită (*default gateway*) trecând prin ruterul `Router3` (adică adresa interfeței `Fa0/0` a ruterului `Router3`)
- pe ruterul `Router1` o rută implicită (*default gateway*) trecând prin ruterul `Router3` (adică adresa interfeței `Fa1/0` a ruterului `Router3`)
- pe ruterul `Router2` o rută implicită (*default gateway*) trecând prin ruterul `Router3` (adică adresa interfeței `Fa2/0` a ruterului `Router3`)
- pe ruterul `Router3` trei rute statice către fiecare dintre cele trei rețele cu stații, trecând respectiv prin ruterele `Router0`, `Router1` și `Router2` (adică adresele IP ale interfețelor `Fa0/0` ale celor trei rutere)

> Configurarea *default route/gateway* în Cisco IOS se realizează folosind adresa de rețea `0.0.0.0/0`. Detalii aici.

- După o configurare corectă veți avea conectivitate între toate stațiile din topologie. Pentru verificare folosiți comanda `ping` din meniul unei stații și transmiteți pachete către celelalte stații: accesați tab-ul `Desktop`, icon-ul `Command Prompt`.

> În Cisco IOS primul pachet trimis poate să nu fie de fapt trimis din cauza absenței tabelei ARP. De aceea este posibil ca un set întreg de pachete trimise la rularea unei comenzi `ping` pe stație (sunt trimise patru în mod implicit) să eșueze, întrucât traversează multe rutere. A doua comandă `ping` pe stație va reuși, în cazul unei configurări corecte.

- Dacă aveți probleme de conectivitate, verificați din aproape în aproape între ce dispozitive aveți sau nu conectivitate. Urmăriți adresele IP configurate și tabela de rutare pentru fiecare ruter și identificați cauza problemei.

12. [A][BONUS - 10p] Rutare inter-VLAN

Topologie: download

- În cazul în care avem o configurație cu mai multe VLAN-uri care sunt colectate în același switch se pune problema rutării pachetelor din acele VLAN-uri, eventual chiar posibilitatea comunicării între VLAN-uri (*inter-VLAN routing*).
- Pentru aceasta putem configura o legătură de tip trunchi între un switch care colectează VLAN-uri și un ruter care poate ruta pachetele încapsulate. Un astfel de ruter are nevoie de o singură interfață care să permită rutarea între VLAN-uri, motiv pentru care se mai cheamă *router on a stick* sau *one-armed router*[1].

- În topologie se găsește o rețea cu două VLAN-uri (VLAN `10` și VLAN `20`).
- Stațiile `PC0` și `PC1` fac parte din VLAN-ul `10`, iar stațiile `PC2` și `PC3` fac parte din VLAN-ul `20`.
- Stațiile au configurate adrese IP și pot comunica în cadrul aceluiași VLAN.

- Switch-ul `Switch1` are configurate două porturi în modul acces:
  - portul `Fa0/1` în VLAN-ul `10`
  - portul `Fa1/1` în VLAN-ul `20`

- Switch-urile `Switch0` și `Switch2` nu au configurate VLAN-uri; fac simple operații de comutare.
- Legătura dintre switch-ul `Switch1` și ruterul `Router0` este o legătură de tip trunchi, cu ruterul `Router0` pe post de *router on a stick*.

- Realizați configurările necesare pentru a permite rutarea inter-VLAN și, deci, stațiilor din VLAN-uri diferite să comunice unele cu celelalte.
- Va trebui să configurați *Default Gateway* pe stații (folositi prima adresa asignabila din ambele spatii de adrese ca gateway-uri; aceste adrese IP vor fi adresele subinterfetelor de pe ruter).

> Va trebui să configurați interfața `FastEthernet2/1` a switch-ului `Switch1` în modul `trunk`.

> Informații despre configurarea încapsulării `dot1Q` pe un ruter se găsesc aici: frame-ul `2600 Router`.

Navigare

Laboratorul 5

- 00. [M][5p] Procesul de rutare
- 01. [A][10p] Configurare default gateway
- 02. [M][10p] Antetul pachetelor în procesul de rutare
- 03. [A][5p] Rute și pachete de reply
- 04. [A][10p] Adresare și default gateway
- 05. [A][10p] Configurare rute statice
- 06. [M][5p] Vizualizare tabelă de rutare
- 07. [A][15p] Adresare și configurare rute statice
- 08. [A][10p] Depanare probleme de configurare a rutării
- 09. [A][10p] Adresare și configurare rute statice (reiterare)
- 10. [A][10p] Rute implicite pe rutere
- 11. [A][BONUS - 10p] Adresare și configurare rute statice (avansat)
- 12. [A][BONUS - 10p] Rutare inter-VLAN

---

[1] http://en.wikipedia.org/wiki/One-armed_router

Search

Laborator

Laboratorul 5

- 00. [M][5p] Procesul de rutare
- 01. [A][10p] Configurare default gateway
- 02. [M][10p] Antetul pachetelor în procesul de rutare
- 03. [A][5p] Rute și pachete de reply
- 04. [A][10p] Adresare și default gateway
- 05. [A][10p] Configurare rute statice
- 06. [M][5p] Vizualizare tabelă de rutare
- 07. [A][15p] Adresare și configurare rute statice
- 08. [A][10p] Depanare probleme de configurare a rutării
- 09. [A][10p] Adresare și configurare rute statice (reiterare)
- 10. [A][10p] Rute implicite pe rutere
- 11. [A][BONUS - 10p] Adresare și configurare rute statice (avansat)
- 12. [A][BONUS - 10p] Rutare inter-VLAN

Curs

Informații generale RL

- **Regulament**
  - Notare Seria CA
  - Notare Seriile CB/CC
- **Resurse**
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

BY-SA CHIMERIC DE CSS DOKUWIKI GET FIREFOX RSS XML FEED XHTML 1.0

Laborator 6. Virtualizare în Docker

Cunoștințe și abilități ce vor fi dobândite

- Intoducere în Docker CLI
- Docker Container vs Docker Image
- Dockerfile și construcția de Imagini
- Union File System: Overlay2 - image layers
- Docker Volumes
- Docker Networking: bridge & overlay (VXLAN)
- Docker SWARM
- Introducere în arhitecturi pe microservicii
- Arhitecturi pe microservicii vs Arhitecturi monolite

Cheat sheet

- Cheat Sheet

Pregătire infrastructură de laborator

În cadrul laboratorului vom utiliza Play with Docker o platformă online pusă la dispoziție de Marcos Liljedhal și Jonathan Leibiusky și sponsorizată de Docker Inc. Pentru uz personal recomandam insa instalarea Docker pe calculator.

Pentru instalarea Docker pe Ubuntu urmăriți procedura de aici [1]. Suplimentar față de pașii descriși anterior, rulați comanda: `sudo usermod -aG docker $USER` desprinsă din documentația [2]

- [1] https://docs.docker.com/install/linux/docker-ce/ubuntu/
- [2] https://docs.docker.com/install/linux/linux-postinstall/

Introducere

Întrucât, preponderența claselor din anii anteriori a fost cu precădere orientată pe multe din aspectele complexe ale dezvoltării de software, în laboratorul de azi ne propunem să oferim o altă perspectivă în conceperea unei aplicații sau a unei soluții. Aspecte precum **volumul de date de procesat**, **toleranța la defecte**, **portabilitatea** sau **scalabilitatea** sunt constrângeri esențiale care trebuie tratate în procesul de implementare a unei soluții sau aplicații.

Standardele și cerințele dictate de industrie se focalizează din ce în ce mai mult pe adoptarea modelului de cloud, menit, de cele mai multe ori, să ofere un șablon/pattern care să adreseze parțial sau integral constrângerile (sau oportunitățile) menționate anterior.

Laboratorul curent își propune o introducere pragmatică în modele clasice de system design & architecture încercând să atingă gradual mici elemente de complexitate, care pot aduce un plus de valoare unei aplicații în contextul de cloud.

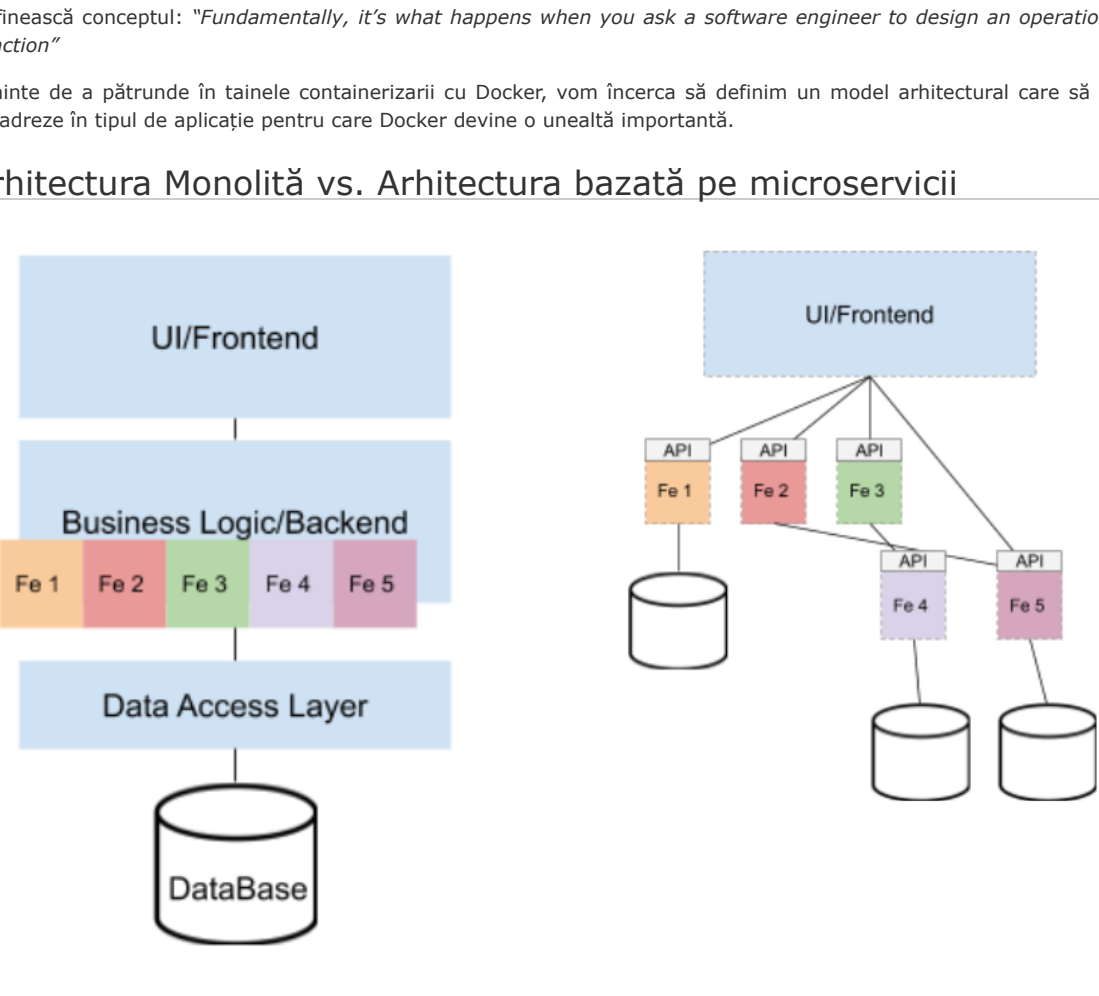
Modelele prezentate pot fi puncte de plecare importante pentru parcurgerea unor interviuri de Site Reliability Engineer, Cloud Engineer sau DevOps Engineer.

Mai mult, laboratorul deservește, în egală măsură, o introducere în Docker, Docker Networking & Docker Orchestration, facilitând implementarea graduală a unui serviciu web, de la concept până într-o formă care ar putea răspunde unor cerințe reale de producție. Laboratorul se adresează în egală măsură atât oamenilor cu înclinații către dezvoltare cât și oamenilor care se orientează spre poziții operaționale.

Ben Treynor *VP of Engineers @ Google* și inventatorul termenului **Site Reliability Engineer** încercând să definească conceptul: *"Fundamentally, it's what happens when you ask a software engineer to design an operations function"*

Înainte de a pătrunde în tainele containerizarii cu Docker, vom încerca să definim un model arhitectural care să se încadreze în tipul de aplicație pentru care Docker devine o unealtă importantă.

Arhitectura Monolită vs. Arhitectura bazată pe microservicii



Este greșit să plecăm de la premisa că o abordare este mai bună decât cealaltă, fiecare fiind o opțiune optimă în contextul potrivit. O abordare monolită implică dezvoltarea întregului stack de aplicație pe un singur nod, un singur sistem, în timp ce, în cadrul modelului bazat pe microservicii, fiecare unitate funcțională va avea propriul mediu izolat în care rulează. Gradul de rezoluție pentru definirea unității funcționale poate varia, de la viziunea macro, cu 2 componente: Frontend - Backend, până la nivelul de granularitate prin care putem avea fiecare feature al aplicației dezvoltat ca un microserviciu (ca un obiect rulând individual).

Mai multe referințe despre microservicii:

- https://martinfowler.com/articles/microservices.html
- https://microservices.io/
- https://nigelpoulton.com/blog/f/demystifying-docker-overlay-networking

O comparație minimală între cele două modele arhitecturale, reliefând aspecte pozitive și problematice în ambele abordări, poate fi văzută în tabelul următor:

Arhitecturi Monolite	Arhitecturi bazate pe Microservicii
Modulele unei aplicații sunt Tightly Coupled (ex: Comunicare Inter Proces - IPC)	Serviciile unei aplicații sunt Loosely Coupled (ex: communicare peste rețea utilizând mesaje HTTP via REST API microservices)
Fiecare funcționalitate nouă trebuie scrisă în același limbaj de programare.	Fiecare serviciu component al aplicației poate fi scris într-un limbaj diferit.
Modificarea codului presupune recompilarea și reinstalarea (redeployment-ul) întregii aplicații.	Modificarea unui serviciu/feature necesită reinstalarea acelui serviciu, celelalte rămânând intacte și operaționale.
Simplu de dezvoltat.	Poate aduce un nivel de complexitate mult prea mare (ex: pentru fiecare serviciu trebuie construit un API care să faciliteze comunicația între microservicii)
Scalarea aplicației se aplică întregului stack.	Procedurile de scalare pot fi aplicate pe servicii individuale (eg: multiplicarea serviciului de Frontend al unui magazin online în timpul reducerilor de Black Friday)

Abordarea pe care vom merge în continuare este una bazată pe servicii, cu amendamentul că unitățile funcționale vor fi construite la un nivel de rezoluție macro: **Frontend** și **Backend**, la care vom atașa și o **bază de date**. Să presupunem că avem o cerere din partea unui angajator de a dezvolta un site de stocare și partajare de imagini. Trebuie să ne asigurăm că acesta îndeplinește un set minimal de caracteristici, înainte de a fi lansat în producție:

- nivelul de utilizare va fi unul mediu
- va porni cu un set minimal de funcționalități, dar va fi extins în timp
- echipa de dezvoltatori este foarte dinamică din perspectiva migrării către alte proiecte, iar cunoștințele legate de anumite limbaje de programare sunt neuniforme
- se dorește ca serviciul să aibă o rată de availability care să tindă spre procentul de 99% (Maxim 7 ore 18 minute și 17.5 secunde downtime pe lună - https://uptime.is/)
- se estimează că în perioada sărbătorilor vor exista spike-uri în utilizare, întrucât audiența va căuta activ poze cu mesaje motivaționale.

Aplicația construită pentru laborator va aborda o arhitectură simplificată, bazată pe servicii, Vom încerca să utilizăm Docker pentru a ne asigura că îndeplinim cât mai multe din cerințele date. Înainte de a parcurge cerințele proiectului, ne vom familiariza cu tehnologia de containerizare.

Navigare

Laboratorul 6

- 00. [0p] Play With Docker
- 01. [20p] Hello Docker
- 02. [20p] Docker Images
- 03. [20p] Docker Networking
- 04. [20p] Docker Volumes
- 05. [20p] Orchestrare cu Docker SWARM
- 06

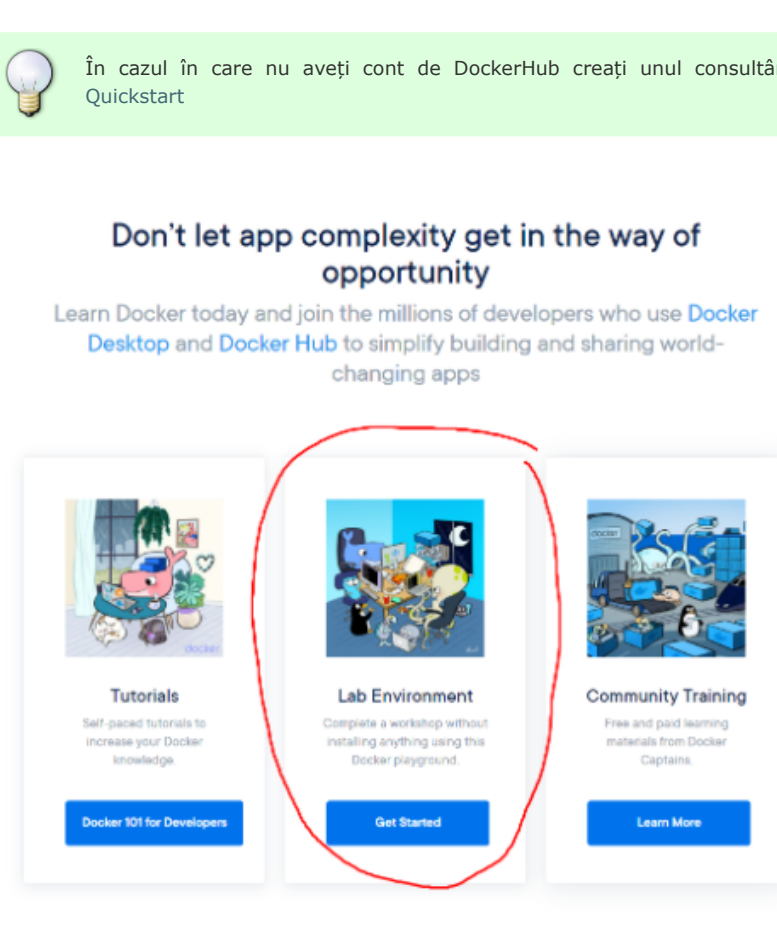
Exerciții

00. [0p] Play With Docker

Play With Docker (PWD) este un mediu online dedicat învățării și testării Docker. Oferă mașini virtuale gratuit în limita a 4h/sesiune. Reprezintă un bun loc de început interacțiunea în lumea Docker și permite utilizatorilor să ruleze comenzile Docker fără a fi necesare instalări sau configurări suplimentare. Oferă experiența de a avea o mașină virtuală Alpine Linux gratuită în browser, unde puteți construi și rula containere Docker și chiar crea clustere.

Pentru a începe interacțiunea cu PWD utilizați linkul: https://labs.play-with-docker.com/. Vă puteți autentifica folosind contul vostru de **Dockerhub**: https://hub.docker.com/

În cazul în care nu aveți cont de DockerHub creați unul consultând manualul: Docker Hub Quickstart

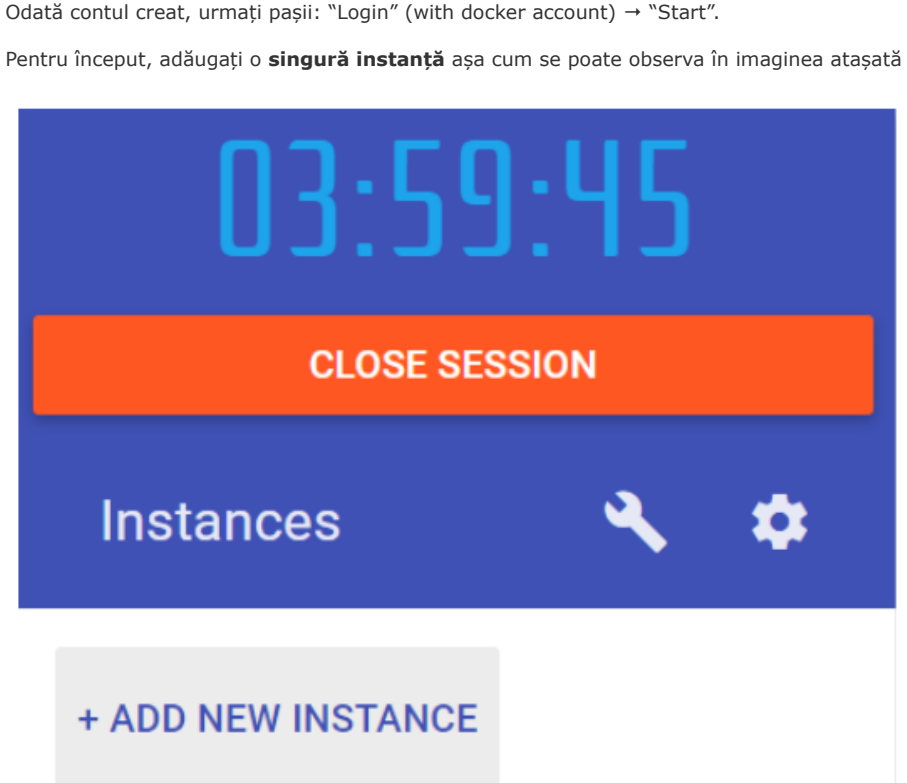


Don't let app complexity get in the way of opportunity

Learn Docker today and join the millions of developers who use Docker Desktop and Docker Hub to simplify building and sharing world-changing apps

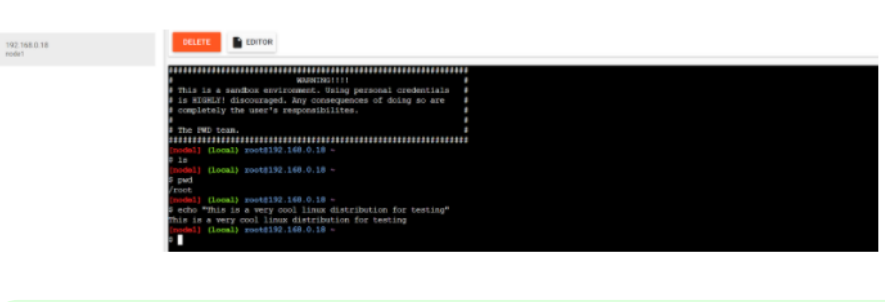
Odată contul creat, urmați pașii: "Login" (with docker account) → "Start".

Pentru început, adăugați o **singură instanță** așa cum se poate observa în imaginea atașată mai jos:



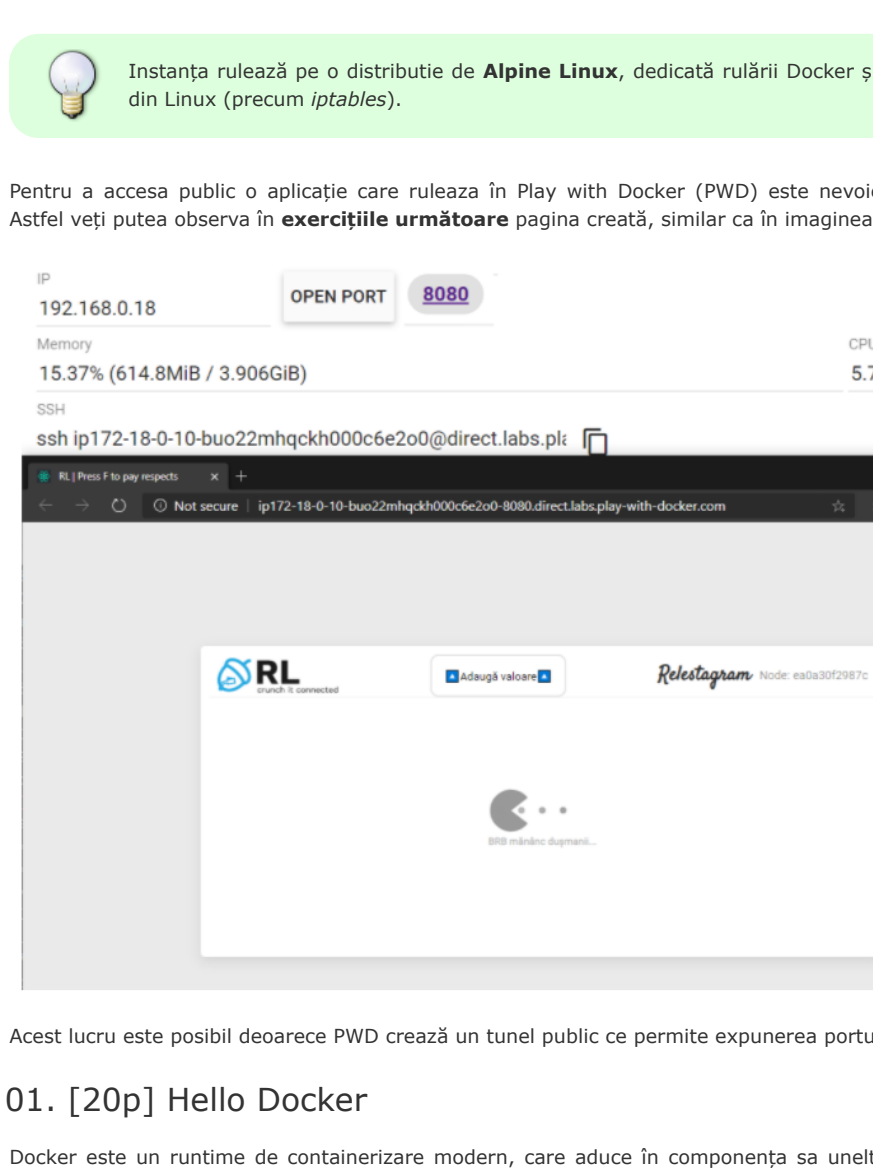
Este posibil să aveți probleme de afișaj dacă nu optați pentru opțiunea de "Maximized" a ferestrei browser-ului.

Odată lansată o instanță, va trebui să fiți direct conectati la ea. Interacțiunea făcându-se direct în consola atașată instanței create anterior.



Instanța rulează pe o distributie de **Alpine Linux**, dedicată rulării Docker și a comenzilor de rețea din Linux (precum *iptables*).

Pentru a accesa public o aplicație care ruleaza în Play with Docker (PWD) este nevoie să folosiți portul expus. Astfel veți putea observa în **exercițiile următoare** pagina creată, similar ca în imaginea de mai jos:



IP
192.168.0.18
OPEN PORT 8080
Memory
15.37% (614.8MiB / 3.906GiB)
CPU
5.79
SSH
ssh ip172-18-0-10-buo22mhqckh000c6e2o0@direct.labs.pl

Not secure | ip172-18-0-10-buo22mhqckh000c6e2o0-8080.direct.labs.play-with-docker.com
RL
Adaugă valoare
Relestagram

Acest lucru este posibil deoarece PWD crează un tunel public ce permite expunerea portului afișat.

01. [20p] Hello Docker

Docker este un runtime de containerizare modern, care aduce în componența sa uneltele necesare împachetării unei aplicații, și a dependințelor acesteia, într-un obiect numit container. Asemănător LXC, Docker utilizează mecanismele native din Kernel-ul sistemului de operare care permit rularea izolată a unei aplicații, fără ca aceasta să interacționeze sau să influențeze procesele sistemului gazdă sau procesele aflate în execuție în alte containere. Dintr-o perspectivă mai pragmatică, putem considera un container ca fiind o multiplicare a user-space-ului (terminologia utilizată în sistemelor de operare monolitice)

Dacă nu înțelegeți încă această ultimă sintagmă… SO is coming!

Ca în orice început din IT, vom porni cu Hello, world!

```
student@aldebaran:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
1b930d010525: Pull complete
Digest: sha256:4df8ca8a7e309c256d60d7971ea14c27672fc0d10c5f303856d7bc48f8cc17ff
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Să verificăm ce se întâmplă. Pentru a vedea containerele care sunt în running state:

```
student@aldebaran:~$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
```

Lista este goală. Pentru a vedea toate containerele, atât cele în running state cât și cele oprite:

```
student@aldebaran:~$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
4bedb1ae0304        hello-world         "/hello"            4 minutes ago       Exited (0) 4 mi
```

Voila!

Să încercăm să rulăm un container de Alpine Linux, una din distribuțiile lightweight de Linux existente.

```
student@aldebaran:~$ docker run alpine
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
89d9c30c1d48: Pull complete
Digest: sha256:c19173c5ada610a5989151111163d28a67368362762534d8a8121ce95cf2bd5a
Status: Downloaded newer image for alpine:latest
student@aldebaran:~$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
student@aldebaran:~$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
7e1f217f73d4        alpine              "/bin/sh"           38 seconds ago      Exited (0) 35 se
4bedb1ae0304        hello-world         "/hello"            6 minutes ago       Exited (0) 6 mi
```

Container-ul este oprit. Motivul pentru care se întâmplă acest lucru se datorează faptului că ciclul de viață al unui container este 1 la 1 cu ciclul de viață al aplicației care rulează în acel container. Astfel, în cazul în care aplicația își finalizează execuția, container-ul aferent își va încheia și el rularea. Pentru a vedea ce proces rulează în mod implicit în container putem folosi docker inspect: `docker inspect <nume sau id container>`

Pentru simplitate, putem obține informația căutată utilizând filtre (--format): `docker inspect <nume sau id container> --format='{{.Config.Cmd}}'`

Pentru a forța execuția continuă a containerului, putem utiliza argumentele -dit (referințe suplimentare în pagina de manual docker-run)

- -d detach
- -i interactive
- -t pseudo-tty

```
student@aldebaran:~$ docker run -dit --name infinite-alpine alpine
b3bbcf9fcd8c265cf96e50c885c10ff68b597e4ba987f7e78630c094432f14bc
```

Putem schimba, totodată, comanda care rulează în container:

```
docker run -dit --name sleep-alpine alpine sleep 10000
```

Și putem atașa sesiunea curentă de shell unui container aflat în execuție:

```
student@aldebaran:~$ docker exec -it infinite-alpine /bin/sh
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys
```

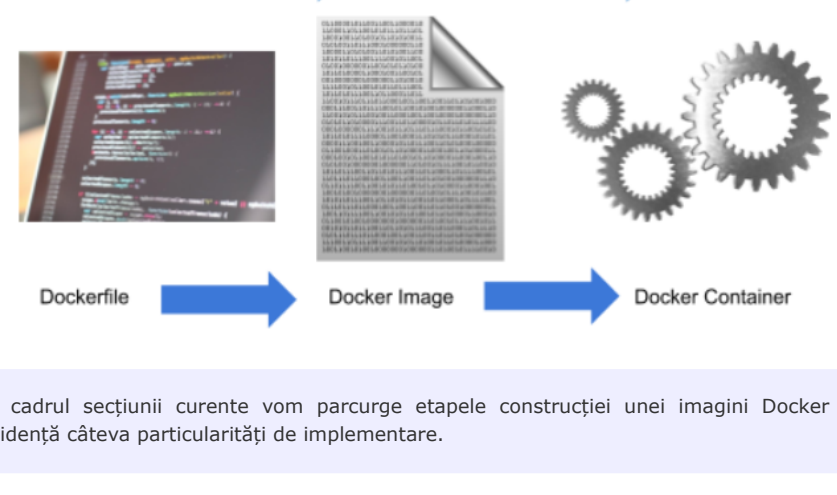
Afișați pe rând:

- pid-ul utilizatorului curent: `id`
- adresa ip a containerului: `ip a`
- toate procesele care rulează pe sistem: `ps -ef` sau `ps aux`
- hostname-ul: `hostname`
- variabilele de mediu: `env`
- și ieșiți: `exit`

În continuare vom vorbi despre conceptul de Docker Images.

02. [20p] Docker Images

Făcând o analogie cu lumea sistemelor de operare, putem considera un Docker image ca fiind similar unui binar - o componenta statică ce poate fi lansată în execuție. Un Docker container este asemănător procesului, o componenta dinamică care este o instanță aflată în execuție a unei imagini Docker.



În cadrul secțiunii curente vom parcurge etapele construcției unei imagini Docker scoțând în evidență câteva particularități de implementare.

Pentru început, să listăm imaginile stocate local. În lista desfășurată veți vedea imaginile pe care le-am utilizat în secțiunea 1:

```
student@aldebaran:~$ docker image ls
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
alpine              latest              965ea09ff2eb        6 weeks ago         5.55MB
hello-world         latest              fce289e99eb9        11 months ago       1.84kB
```

Ca strategie, Docker își propune să ofere o bibliotecă extrem de largă de servicii gata configurate, împachetate sub formă de containere. În cazul în care ne dorim să căutăm o aplicație, un middleware, un runtime etc. care ne-ar fi utile pentru a rezolva o cerință sau a dezvolta o soluție complexă, putem căuta în upstream dacă există deja o astfel de imagine dezvoltată. Pentru a face acest lucru avem diferite metode, atât din CLI cât și din browser. Implicit, engine-ul docker va căuta în **DockerHub registry**, echivalentul GitHub pentru imagini Docker. În terminologia Docker, un registry este un repository de imagini Docker: `docker search <nume imagine cautata>`

- `docker search alpine`
- `docker search httpd`
- `docker search tomcat`
- `docker search --filter is-official=true golang`

Un aspect foarte important din output-ul anterior îl reprezintă coloana Official, atribut ce garantează faptul că imaginea vizată este întreținută și verificată de către o comunitate de încredere, validată de către Docker.

Online putem accesa: https://hub.docker.com Pentru informații despre imagine de Alpine: https://hub.docker.com/_/alpine Meniul tags este utilizat pentru diferențierea imaginilor pe versiuni/pe releases: https://hub.docker.com/_/alpine?tab=tags

Să creăm o imagine pornind de la imaginea oficială **Alpine** cu versiunea de release **latest**. În folder-ul home al utilizatorului curent creați un fișier numit `Dockerfile` și scrieți următoarele 2 linii:

```
student@aldebaran:~$ cat Dockerfile
FROM alpine:latest
CMD ["echo", "Am facut si bune am facut si RL"]
```

Se poate observa ușor forma declarativă, auto-descriptivă, a instrucțiunilor Docker.

Urmând același flux ca în cazul construcției de software, vom compila codul Docker anterior scris, rezultatul fiind un obiect de tip Docker Image. Timpul de execuție poate varia și poate dura chiar și câteva minute.

```
student@aldebaran:~$ docker build -t rlrules/rlhello:v1.0 .
```

Înainte de a merge mai departe, să disecăm succint convenția de nume anterior folosită. Numele imaginii Docker are în general următorul format:

`<DockerHub User (Repository) >/<Nume Aplicație (Nume Imagine)>:<tag (versiune, release)>` `rlrules/rlhello:v1.0`

Convenție de nume extinsă

Respectarea unui astfel de format ne permite ulterior să putem face operațiuni de push sau pull asemănător acțiunilor pe care le putem face în GitHub/git server pentru un repository în care putem versiona și stoca codul.

În continuare vom rula imaginea anterior creată:

```
student@aldebaran:~$ docker run rlrules/rlhello:v1.0
Am facut si bune am facut si RL
```

Utilizarea instucțiunii **CMD**, din Dockerfile, ne oferă, suplimentar, capabilitatea suprascrierii la runtime a comenzii specificate în declarație. Să testăm:

```
student@aldebaran:~$ docker run rlrules/rlhello:v1.0 "hello, world"
docker: Error response from daemon: OCI runtime create failed: container_linux.go:346: starting
```

Eroarea apărută se datorează faptului că "hello, world" nu este o comandă viabilă în Linux și deci nu este interpretată de către shell-ul utilizat (ex: bash, sh etc.)

```
student@aldebaran:~$ docker run rlrules/rlhello:v1.0 cat /etc/issue
Welcome to Alpine Linux 3.10
Kernel \r on an \m (\l)

student@aldebaran:~$ docker run rlrules/rlhello:v1.0 echo "hello, world"
hello, world
```

Cele două comenzi vor rula cu succes, prima afișând o informație de pe container în timp ce a doua afișează mesajul transmis comenzii echo: `hello, world`

Să modificăm imaginea anterioară înlocuind instrucțiunea **CMD** cu instrucțiunea **ENTRYPOINT**.

```
student@aldebaran:~$ cat Dockerfile
FROM alpine:latest
ENTRYPOINT ["echo", "Am facut si bune am facut si RL"]
```

În etapa de build vom construi un nou release v2.0:

```
student@aldebaran:~$ docker build -t rlrules/rlhello:v2.0 .
```

Rularea release-ului v2.0 va avea același outcome ca în cazul versiunii anterioare :

```
student@aldebaran:~$ docker run rlrules/rlhello:v2.0
```

Rezultatul utilizării celor 2 instrucțiuni (**CMD** și **ENTRYPOINT**) este același! Care este totuși diferența între **CMD** și **ENTRYPOINT**?

Pentru a vedea acest lucru, să încercăm să rulăm imaginea nouă utilizând, de data aceasta, argumente:

```
student@aldebaran:~$ docker run rlrules/rlhello:v2.0 "hello, world"
Am facut si bune am facut si RL hello, world
student@aldebaran:~$ docker run rlrules/rlhello:v2.0 cat /etc/issue
Am facut si bune am facut si RL cat /etc/issue
student@aldebaran:~$ docker run rlrules/rlhello:v2.0 echo "hello, world"
Am facut si bune am facut si RL echo hello, world
```

Vedem că în cazul **ENTRYPOINT**, indiferent de argumentul adăugat la finalul comenzii docker run, acesta va fi anexat la comanda setată în instrucțiunea din Dockerfile. Întrucât comanda configurată este `echo <string>`, șirurile de caractere adăugate ulterior vor fi atașate stringului "Am făcut și bune, am făcut și RL". Într-o descriere mai pragmatică, **ENTRYPOINT** imprimă rolul unui executabil clasic asupra imaginii docker.

Un mecanism frecvent utilizat este conjuncția dintre cele două instrucțiuni. Putem folosi **ENTRYPOINT** pentru a desemna comanda fixă, explicit atașată containerului, în timp ce instrucțiunea **CMD** oferă argumentul default, interschimbabil, al comenzii definite de **ENTRYPOINT**.

Vom modifica imaginea astfel încât să reflecte scenariul definit anterior:

```
student@aldebaran:~$ cat Dockerfile
FROM alpine:latest

ENTRYPOINT ["echo"]
CMD ["Am facut si bune, am facut si RL"]
```

Comanda echo este fixă, imutabilă, în timp ce string-ul `Am făcut și bune, am făcut și RL` devine valoare implicită în cazul în care niciun argument suplimentar nu este specificat în comanda docker run.

Compilăm și rulăm noua imagine:

```
student@aldebaran:~$ docker build -t rlrules/rlhello:v3.0 .
student@aldebaran:~$ docker run rlrules/rlhello:v3.0
Am facut si bune, am facut si RL
student@aldebaran:~$ docker run rlrules/rlhello:v3.0 "hello, world"
hello, world
student@aldebaran:~$ docker run rlrules/rlhello:v3.0 cat /etc/issue
cat /etc/issue
student@aldebaran:~$ docker run rlrules/rlhello:v3.0 echo "hello, world"
echo hello, world
```

Lista instrucțiunilor Docker nu se rezumă doar la **FROM** sau **CMD**/**ENTRYPOINT**. Pentru configurații mai complexe există o colecție largă de comenzi ce pot fi declarate la nivelul unui Dockerfile. Referințe despre aceste opțiuni se pot găsi pe site-ul Docker (https://docs.docker.com/engine/reference/builder/)

În continuare vom crește complexitatea și vom construi un Dockerfile extins:

```
student@aldebaran:~$ cat Dockerfile
FROM ubuntu:latest
MAINTAINER rl-awesome-students-team

RUN apt-get update && apt-get -y install cowsay
ENV RL_EXEC_PATH "/usr/games/"
ENV PATH=${PATH}:${RL_EXEC_PATH}

ENTRYPOINT ["cowsay"]
CMD ["Am facut si bune, am facut si RL"]
```

Vom compila și rula imaginea nou definită:

```
student@aldebaran:~$ docker build -t rlrules/rlhello:v4.0 .
```

```
student@aldebaran:~$ docker run rlrules/rlhello:v4.0
```

```
student@aldebaran:~$ docker run rlrules/rlhello:v4.0 "Hello, world"
```

În cadrul configurației curente, plecând de la imaginea de Ubuntu:latest, la build time am instalat pachetul cowsay, am setat variabile de mediu, am specificat maintainer-ul imaginii și am configurat un parametrul implicit, în cazul în care imaginea este rulată fără parametrii adiționali. Complexitatea poate crește, în funcție de acțiunile pe care trebuie să le facem pentru a prepara containerul cu toate dependințele de care are nevoie.

Un aspect important și interesant legat de imagini este reprezentat de construcția pe layere/straturi. Fiecare modificare adusă imaginii anterioare poate rezulta în construcția unui strat suplimentar. Dintr-o perspectivă simplificată, putem spune că fiecare instrucțiune din Dockerfile construiește o imagine nouă, intermediară, care se diferențiază de cea anterioară prin modificările aduse de instrucțiunea curentă. Plecând de la această premisă, un layer este exact acest delta dintre 2 imagini succesive. Dintr-o perspectivă mai complexă, nu orice instrucțiune rezultă într-un strat nou. Comenzi precum **CMD**, **MAINTAINER**, **ENTRYPOINT** nu aduc o modificare substanțială imaginii, ci alterează aspecte legate de metadatele atașate unei imagini.

Să validăm această informație pentru imaginea rlrules/rlhello:v4.0:

```
student@aldebaran:~$ docker image inspect rlrules/rlhello:v4.0
```

În output-ul comenzii anterioare, căutați secțiunea RootFS. Pentru simplitate puteți rula:

```
student@aldebaran:~$ docker image inspect --format "{{json .RootFS}}" rlrules/rlhello:v4.0
```

Câte layere pot fi numărate? De ce?

Să vedem care au fost acele instrucțiuni care au dus la construcția celor 5 straturi:

```
student@aldebaran:~$ docker image history rlrules/rlhello:v4.0
```

Toate instrucțiunile care au un `SIZE > 0` sunt responsabile de crearea unui layer suplimentar în imagine:

- primele nivele sunt moștenite din construcția imaginii de bază Ubuntu cu tag-ul latest (`FROM ubuntu:latest`)
- ultimul nivel este adăugat de noi prin instrucțiunea **RUN** `apt-get update && apt-get install` ce creează noi fișiere în sistemul de fișiere

În cazul în care ne dorim să ștergem o imagine de pe sistem, mai întâi trebuie să obținem identificatorul unic al imaginii (`IMAGE ID`):

```
student@aldebaran:~$ docker image ls
```

Să încercăm să ștergem imaginea rlrules/rlhello:v1.0:

```
student@aldebaran:~$ docker image rm <id imagine rlrules/rlhello:v1.0>
```

De ce returnează eroare? Ce putem face pentru a șterge imaginea?

O soluție a problemei anterioare o reprezintă ștergerea containerului care utilizează imaginea vizată:

```
student@aldebaran:~$ docker container ps -a
```

Care este containerul care utilizează imaginea vizată?

```
student@aldebaran:~$ docker container rm <id container oprit care utilizeaza imagir
```

Acum putem efectua ștergerea imaginii:

```
student@aldebaran:~$ docker image rm <id imagine rlrules/rlhello:v1.0>
```

Voila!

Până în acest moment, ne-am familiarizat cu Docker Engine și cu obiectele cele mai importante din lumea Docker. În continuare, în cadrul laboratorului, vom începe să punem server-ul nostru de imagini în contextul de rulare cerut. Întrucât aplicația este slab cuplată, un prim pas care trebuie realizat este să stabilim comunicarea între serviciile componente. În acest context ne vom muta atenția asupra conceptelor de Networking din Docker.

03. [20p] Docker Networking

Așa cum ați văzut în laboratoarele anterioare de LXC, și în cazul rulării unui container Docker există necesitatea stabilirii comunicației către exterior prin rețele virtuale sau prin configurații particulare de rețea. Avantajul major îl constituie faptul că multe din aspectele anevoioase de configurare sunt abstractizate, fiind reduse la un număr mic de obiecte predefinite ce pot fi utilizate pentru scenarii complexe. O parte din terminologia aferentă Docker networking se poate găsi în tabelul următor, unde vom defini Driverele Docker - acele obiecte de rețea predefinite cu care vine Docker Engine-ul și care facilitează comunicarea la nivelul containerelor:

Driver network	Descriere
bridge	tip de driver utilizat în mod implicit, util pentru aplicații care stau în containere individuale și care au nevoie de conectivitate între ele sau către exterior
host	disponibil doar în SWARM mode, tip de network care partajează rețeaua cu mașina gazdă
overlay	tip de driver care permite comunicarea între containere aflate pe noduri Docker diferite din rețele diferite - comunicare la nivel 2 utilizând VXLAN
macvlan	tip de network care alocă containerelor adrese MAC și permit atașarea container-ului în aceiași rețea cu sistemul gazdă
none	tip de driver care dezactivează conectivitatea la rețea

Referințe:

- https://www.youtube.com/watch?v=Yr6-2ddhLVo
- http://blog.nigelpoulton.com/demystifying-docker-overlay-networking/
- https://docs.docker.com/network/

În continuare vom începe să construim aplicația de sharing de imagini, realizând în paralel și comunicația între serviciile componente, în concordanță cu modelul slab cuplat ales.

Expunerea către exterior a serviciilor dintr-un container

Să încercăm să lansăm serviciul de frontend. Întrucât acesta va fi expus către exterior, pentru a fi accesibil din browser, vom publica portul 80 al containerului:

```
student@aldebaran:~$ docker run -d --name=rl-frontend  \
   -e BACKEND_SERVER=rl-backend \
   -p 8080:80 \
   rlrules/docker-lab-frontend
```

Parametrul -p (publish) cu valoarea 8080:80 (<port sistem gazdă>:<port container>) expune/alocă pe mașina gazdă portul TCP 8080, urmând ca traficul orientat către acest port să fie trimis, mai departe, către socket-ul containerului expus pe portul TCP 80.

Unde ați mai văzut un astfel de setup? Cum se numește acest procedeu?

Prin abstractizarea implementată prin flag-ul publish, Docker crează, în backstage, o regulă iptables de DNAT (port-forwarding) care redirecționează traficul venit pe portul TCP/8080 al mașinii gazdă către portul TCP/80 de pe container. În plus, o altă regulă definită prin utilizarea publish, permite traficul destinat interfeței virtuale docker0 (network de tip bridge, creat implicit la instalarea Docker, util pentru a conecta toate containerele care nu au o configurație de rețea particulară).

```
student@aldebaran:~$ sudo iptables -L -nv -t filter
student@aldebaran:~$ sudo iptables -L -nv -t nat
```

Care sunt regulile de iptables care au fost generate de parametrul publish?

```
student@aldebaran:~$ sudo iptables -L -nv -t filter | grep -i 80 -A 3 -B 3
student@aldebaran:~$ sudo iptables -L -nv -t nat | grep -i 8080 -A 3 -B 3
```

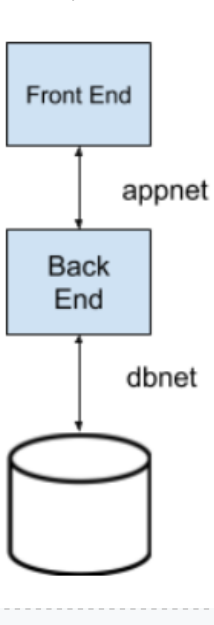
Un bridge network, așa cum se auto-definește, este un echipament (appliance) virtual care permite comunicarea la nivel 2 a entităților de rețea conectate la acesta. În sensul terminologiei Docker, un bridge network adaugă suplimentar capabilitățiii anterior definite, o configurație de Default Gateway la nivelul interfeței virtuale a containerului și implementarea de **Masquerade - Network Address Translation/Port Address Translation (NAT/PAT)**. În acest fel, containerul va putea accesa, utilizând conectivitatea sistemului gazdă, resurse externe din afara rețelei sau Internet.

Accesați din Play with Docker portul 8080 pentru serviciul de frontend: Dati click pe 8080

Service Discovery

Am observat că serviciul de frontend funcționează (`http://<IP mașină gazdă>:8080`), însă pare că nu reușește să încarce conținut. Acest comportament se petrece deoarece aplicația încearcă să consume date de la un serviciu de backend (componentă ce înmagazinează toate funcțiile de procesare a informației) și așteaptă să îi devină disponibil. Este foarte important ca în design-ul unei aplicații, componenta de frontend să ofere doar interfața grafică (de interacțiune) și nimic mai mult, urmând ca procesele de manipulare a datelor să se petreacă în backend.

Ca în orice configurație loosely coupled (slab-cuplată), trebuie să ne asigurăm de faptul că elementele componente ale unei soluții pot comunica între ele. Pentru a obține acest rezultat, va trebui să creăm un Docker bridge în care să conectăm containerele aplicației. Pentru a crește un pic complexitatea din perspectiva izolării traficului, vom crea două rețele: o rețea între frontend și backend și o rețea între backend și baza de date:



```
student@aldebaran:~$ docker network create --driver bridge appnet
student@aldebaran:~$ docker network create --driver bridge dbnet
```

În continuare vom afișa cele două entități nou create:

```
student@aldebaran:~$ docker network ls --filter name=net
```

Întrucât rețelele sunt deja construite, vom lansa cele 2 servicii suplimentare din componența aplicației - backend și baza de date:

```
student@aldebaran:~$ docker run -d --name=rl-database \
   -e MYSQL_DATABASE=rl-database \
   -e MYSQL_USER=rl-user \
   -e MYSQL_PASSWORD=rl-specialpassword \
   -e MYSQL_ROOT_PASSWORD=root \
   -e TZ=Europe/Bucharest \
   --network=dbnet \
   mysql
[...]
```

```
student@aldebaran:~$ docker run -d --name=rl-backend  \
   -e DB_SERVER=rl-database \
   --network=dbnet \
   rlrules/docker-lab-backend
[...]
```

```
student@aldebaran:~$ docker network connect appnet rl-backend
[...]
student@aldebaran:~$ docker network connect appnet rl-frontend
[...]
```

Va trebui să așteptăm câteva secunde pentru inițializarea bazei de date. Să testăm funcționalitatea platformei accesând aplicația din Browser (`http://<IP mașină gazdă>:8080`).

Dacă Pacman a dispărut de pe ecran, iar în locul lui a apărut o colecție impresionantă de meme-uri, înseamnă că platforma funcționează! În cazul în care întâmpinați probleme:

```
student@aldebaran:~$ docker restart rl-frontend
```

Dar cum au reușit serviciile să se conecteze între ele?

În lumea Docker conectivitatea cu alte servicii se face pe bază de nume. Să presupunem că vrem să ne conectăm la baza de date. În conectivitatea clasică, de cele mai multe ori, trebuie specificată adresa IP a bazei de date, portul dacă nu este cel standard soluției de baze de date, user-ul bazei de date și parola. În Docker, simplul re-deployment al unui container nu asigură persistența adresei IP alocat interfeței virtuale și deci, nu putem utiliza adresarea IP pentru a asigura conectivitatea dintre două servicii. Pentru a abstractiza elementul de configurare, Docker propune un model de Service Discovery bazat pe DNS. În acest sens, un identificator care rămâne nemodificat între două rulări succesive ale aceluiași container, reprezintă numele pe care îl atribuim containerului (în exemplul nostru: --name rl-frontend, --name rl-backend, --name rl-database). În cele mai multe cazuri, aplicațiile împachetate în container au hardcodate astfel de nume pentru a stabili conectivitatea, și totodată un set implicit de credențiale. În setup-uri mai inteligente, coordonatele de conectare între servicii pot fi configurate prin variabile de mediu (parametrul -e) admise la runtime, prin fișiere de configurare partajate cu sistemul gazdă etc.

În continuare vom testa funcționalitatea mecanismului de service discovery. Ne vom conecta la instanța de backend:

```
student@aldebaran:~$ docker exec -it rl-backend bash
```

Și vom rula utilitarele clasice de testare a conectivității - ping, telnet și host (pentru validarea rezoluției de nume)

```
root@rl-backend:~$ ping rl-frontend
[...]
root@rl-backend:~$ apt update && apt install -y host && host rl-frontend
[...]
root@rl-backend:~$ telnet rl-frontend 80
[...]
```

Pentru a ieși `CTRL + ]` sau comanda `quit`

```
root@rl-backend:~$ ping rl-database
[...]
root@rl-backend:~$ host rl-database
[...]
exit
```

Dacă ștergem containerul de frontend și îl reconstruim:

```
student@aldebaran:~$ docker rm -f rl-frontend
```

```
student@aldebaran:~$ docker run -d --name=rl-frontend \
   -e BACKEND_SERVER=rl-backend \
   -p 8080:80 --network appnet \
   rlrules/docker-lab-frontend
```

```
student@aldebaran:~$ docker exec -it rl-backend host rl-frontend
```

```
student@aldebaran:~$ docker exec -it rl-frontend ip a
```

Vom vedea că serviciul de DNS se va actualiza cu noua adresă IP a containerului, în cazul în care aceasta a fost schimbată între cele 2 rulări consecutive. În caz particular, este puțin probabil ca Docker engine să aloce o altă adresă IP.

În continuarea laboratorului, perspectiva va fi mutată pe date și persistența acestora.

04. [20p] Docker Volumes

Defineam un container Docker ca fiind o instanță în execuție a unei imagini Docker. În acest sens, o mare parte din datele existente în interiorul unui container provin din nivelele (layers) de construcție a imaginii instanțiate. O

Laborator
Laboratorul 6
- 00. [0p] Play With Docker
- 01. [20p] Hello Docker
- 02. [20p] Docker Images
- 03. [20p] Docker Networking
- 04. [20p] Docker Volumes
- 05. [20p] Orchestrare cu Docker SWARM
- 06

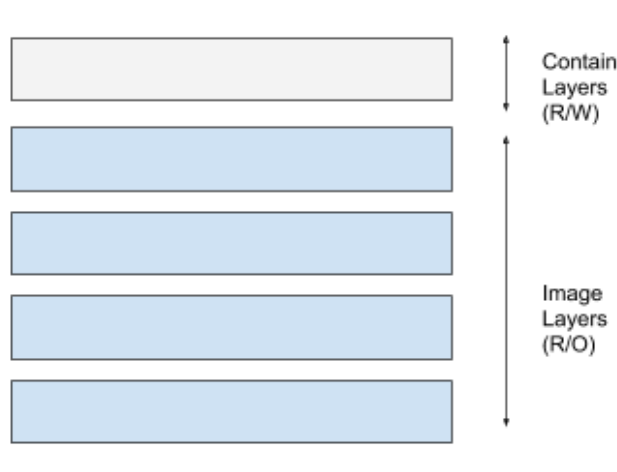
Curs
Informații generale RL
- Regulament
  - Notare Seria CA
  - Notare Seriile CB/CC
- Resurse
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri
- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare
- Laborator 1. Introducere în Rețele de Calculatoare
- Laborator 2. Comutarea pachetelor
- Laborator 3. Adresarea IP
- Laborator 4. VLAN (Virtual LAN)
- Laborator 5. Rutare
- Laborator 6. Virtualizare în Docker
- Laborator 7. Adresare IP și rutare în Linux
- Laborator 8. Securizarea unui server
- Laborator 9. Translatarea de adrese
- Laborator 10. Clienți de rețea
- Laborator 11. Securitatea Rețelelor Locale
- Laborator 12. Amazon Web Services
- Laborator extra - Virtualizare în Networking
- Laborator Recapitulare

Teme
- Tema1
- Tema2

aplicație containerizată, odată lansată, poate genera date pe care le stochează pe sistemul de fișiere. În caz particular, aplicația noastră va stoca imaginile urcate în platformă pe sistemul de fișiere din backend, într-o cale specifică: `/home/rl-user/rl-app/rl-images/upload`.



Straturile/layerele din construcția imaginii sunt read-only, ele deservind totalitatea containerelor aflate în execuție, care sunt instanțe ale imaginii. În cazul în care este necesară manipularea datelor existente sau crearea de date noi, funcția de storage a Docker Engine-ului va separa modificările aduse imaginii de bază print-un strat (layer) adițional, poziționat peste straturile read-only existente. Nivelul suplimentar, cu drepturi Read-Write (R/W), este activ atâta timp cât containerul își desfășoară ciclul de viață. La finalizarea execuției unui container, informația generată la runtime și stocată pe sistemul de fișiere se pierde. Mecanismul de stocare a datelor, așa cum a fost descris anterior poartă numele de Copy-on-write (COW), și este realizat prin utilizarea unui sistem de fișiere special din categoria Union File Systems.

În continuare ne propunem să exploatăm mecanismele native Docker pentru asigurarea persistenței datelor.

În demo-ul anterior, ați avut ocazia să urcați poze în platforma de sharing de imagini. Dacă nu ați făcut-o, acum este un moment propice (formatul suportat este PNG). În continuare vom șterge containerul de backend și îl vom reconstrui:

```
student@aldebaran:~$ docker rm -f rl-backend
```

```
student@aldebaran:~$ docker run -d --name=rl-backend  \
   -e DB_SERVER=rl-database \
   --network=dbnet \
   rlrules/docker-lab-backend
```

```
student@aldebaran:~$ docker network connect appnet rl-backend
```

Dați refresh la pagina web, încărcată în browser. Pozele nu mai există întrucât acestea au fost scrise de backend pe sistemul de fișiere. Pentru a păstra consistența informației stocate în baza de date vom reseta (șterge) atât baza de date cât și containerul de backend.

```
student@aldebaran:~$ docker rm -f rl-backend
```

```
student@aldebaran:~$ docker rm -f rl-database
```

Știm că aplicația stochează pozele uploadate în locația `/home/rl-user/rl-app/rl-images/upload` de pe sistemul de fișiere. Pentru a asigura persistența acestui folder, în cazul în care accidental ștergem containerul de backend, sau în cazul în care suntem nevoiți să îl relansăm, din cauza unei probleme de funcționare, vom crea ceea ce se numește un **Docker volume**.

```
student@aldebaran:~$ docker volume create poze
```

```
student@aldebaran:~$ docker volume list
```

In continuare vom relansa serviciul baze de date:

```
student@aldebaran:~$ docker run -d --name=rl-database \
   -e MYSQL_DATABASE=rl-database \
   -e MYSQL_USER=rl-user \
   -e MYSQL_PASSWORD=rl-specialpassword \
   -e MYSQL_ROOT_PASSWORD=root \
   -e TZ=Europe/Bucharest \
   --network=dbnet \
   mysql
```

Urmat de serviciul de backend, atașând în plus volumul nou creat.

```
student@aldebaran:~$ docker run -d --name=rl-backend  \
   -e DB_SERVER=rl-database \
   --network=dbnet \
   --mount type=volume,src=poze,dst=/home/rl-user/rl-app/rl-images/upload \
   rlrules/docker-lab-backend
[...]
```

```
student@aldebaran:~$ docker network connect appnet rl-backend
```

Alternativ puteti utiliza si constructia `-v poze:/home/rl-user/rl-app/rl-images/upload`

Să vedem dacă putem obține persistența datelor în urma unor rulări succesive. Urcați câteva poze în platformă și refaceți containerul de backend:

```
student@aldebaran:~$ docker rm -f rl-backend
```

```
student@aldebaran:~$ docker run -d --name=rl-backend  \
   -e DB_SERVER=rl-database \
   --network=dbnet \
   -v poze:/home/rl-user/rl-app/rl-images/upload \
   rlrules/docker-lab-backend
```

```
student@aldebaran:~$ docker network connect appnet rl-backend
```

Disecând mecanismul de abstractizare prin volume, un obiect de tip volum este în, în spate, un folder stocat pe sistemul de fișiere al mașinii gazdă, într-o zonă specifică a Docker Engine-ului. Să încercăm să găsim această locație utilizând comanda `docker volume inspect`:

```
student@aldebaran:~$ docker volume inspect poze
```

Pentru simplitate:

```
student@aldebaran:~$ docker volume inspect poze --format {{.Mountpoint}}
```

O altă metodă de a asigura persistența datelor o poate furniza mecanismul de **Mount Binding**. În loc de a crea și atașa un volum unui container Docker, o altă posibilitate o presupune partajarea directă a unui custom folder de pe sistemul de fișiere al mașinii gazdă, între cele 2 entități. Întrucât anterior am descoperit unde stochează Docker datele dintr-un obiect de timp volum, în continuare vom copia fișierele din volum într-un custom folder creat în home-ul utilizatorului student și vom monta acel folder utilizând mecanismul de mount binding.

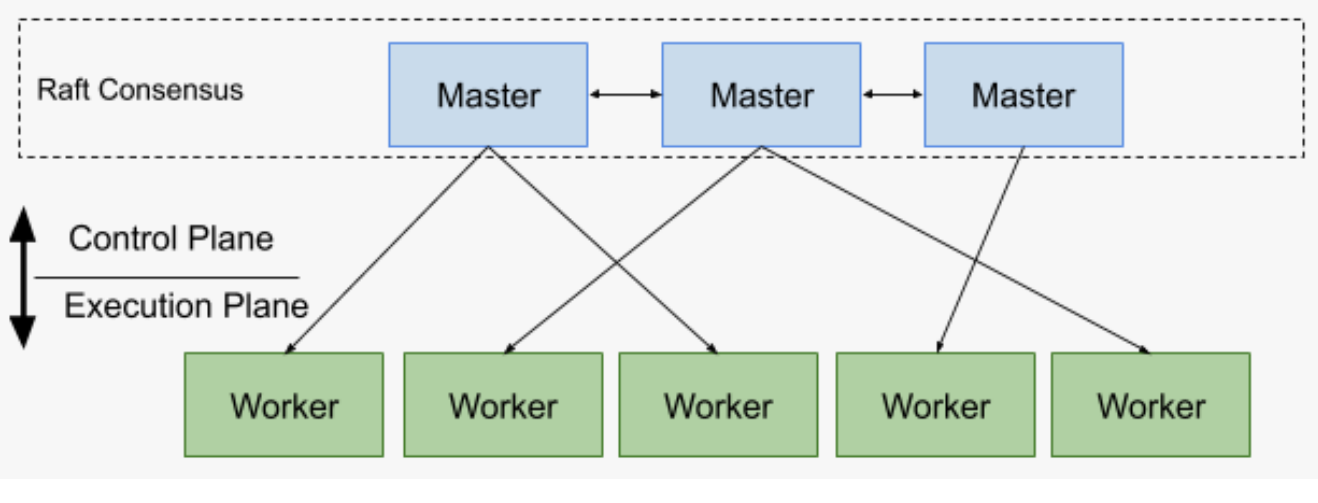
```
student@aldebaran:~$ mkdir $HOME/poze
student@aldebaran:~$ sudo cp -p `docker volume inspect poze --format {{.Mountpoint}}/*` $HOME/po
student@aldebaran:~$ docker rm -f rl-backend
[...]
student@aldebaran:~$ docker run -d --name=rl-backend  \
   -e DB_SERVER=rl-database \
   --network=dbnet \
   --mount type=bind,src=$HOME/poze,dst=/home/rl-user/rl-app/rl-images/upload \
   rlrules/docker-lab-backend
[...]
student@aldebaran:~$ docker network connect appnet rl-backend
[...]
```

Urcați o nouă poză în platformă. Apare aceasta în directorul `poze`?

05. [20p] Orchestrare cu Docker SWARM

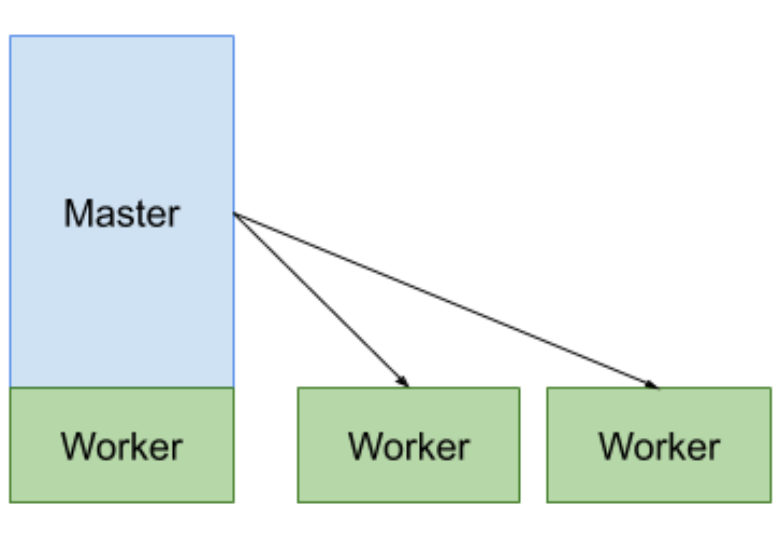
Am ajuns în punctul în care trebuie să lansăm site-ul în producție și stim că trebuie să ne preocupăm de aspecte precum scalabilitate (capacitatea de a suporta o încărcătură mai mare de lucru - un caz particular pentru perioada sărbătorilor), high-availability (Service-Level Agreement stabilit la 99%), redundanță sau toleranță la defecte (auto-healing). În forma standard, Docker Engine nu poate asigura niciunul din aspectele prezentate anterior, el furnizând doar un runtime de containerizare. Pentru a suporta constrângerile de producție, într-o formă autonomă și automată, este necesar un sistem de clusterizare și orchestrare. Soluția pe care o vom utiliza poartă numele de Docker SWARM.

Docker SWARM este o soluție existentă în piață pentru orchestrarea de containere de tip Docker, în competiție cu platforme precum Kubernetes, Apache Mesos, OpenShift ș.a. Avantajul principal cu care vine platforma este ușurința în instalare și configurare, fiind un produs integrat nativ în Engine-ul de Docker. Din perspectivă arhitecturală, un cluster de Docker SWARM este alcătuit din două tipuri de noduri: Master Node - implementează nivelul de control, preluând cererile de deployment și monitorizând continuu starea cluster-ului Docker; Worker Node - asigură execuția containerelor solicitate de nivelul de control.



În cadrul laboratorului, vom utiliza o implementare simplificată, cu 2 noduri (1 master care are rol și de worker și 1 worker dedicat).

O mașină virtuală va avea rol de Master, iar cealaltă mașină va deservi ca Worker.



Înainte de a începe secțunea următoare, vom curăța nodurile de obiectele Docker create în exercițiile anterioare:

```
student@aldebaran:~$ docker rm -f rl-frontend rl-backend rl-database
```

```
student@aldebaran:~$ docker volume prune
```

```
student@aldebaran:~$ docker system prune
```

Totodată, pentru a simplifica terminologia utilizată în continuare, numim:

- MASTER - nodul/VM-ul care a fost ales ca Docker SWARM master
- WORKER - nodurile/VM-urile care au fost alese ca Docker SWARM workers

Pe nodul (VM) MASTER:

```
student@aldebaran:~$ docker swarm init --advertise-addr <IP interfață VM>
```

Din outputul anterior puteți obține comanda de join pe care o puteți utiliza pe nodurile care vor deservi ca workers. In cazul în care ulterior doriți să atașați noi noduri clusterului, puteți obține comanda anterioară astfel:

```
student@aldebaran:~$ docker swarm join-token worker
```

Pe celelalte noduri (pe nodurile WORKER):

```
student@aldebaran:~$ docker swarm join --token <SWARM cluster token> <IP interfață VM MASTER>:23
```

Pe MASTER:

```
student@aldebaran:~$ docker node ls
```

În continuare vom vedea cum utilizarea unei soluții de orchestrare poate simplifica mentenanța, în producție, a unei aplicații critice, oferind capabilități și mecanisme de auto-healing, scheduling sau replicare.

Pe MASTER vom face o cerere de construire a unui tip special de setup de rețea, facil rulării aplicației noastre în mod distribuit, pe diverse noduri ale clusterului:

```
student@aldebaran:~$ docker network create -d overlay rlapp-overlay
```

Ulterior acestui pas, vom crea fiecare componentă a aplicație. În configurația curentă, toate cele 3 servicii vor fi conectate la același obiect de rețea:

```
student@aldebaran:~$ docker service create --name=rl-database \
   -e MYSQL_DATABASE=rl-database \
   -e MYSQL_USER=rl-user \
   -e MYSQL_PASSWORD=rl-specialpassword \
   -e MYSQL_ROOT_PASSWORD=root \
   -e TZ=Europe/Bucharest \
   --network=rlapp-overlay \
   --replicas 1 \
   mysql
[...]
student@aldebaran:~$ docker service create --name=rl-backend  \
   -e DB_SERVER=rl-database \
   --network=rlapp-overlay \
   --replicas 1 \
   rlrules/docker-lab-backend
[...]
student@aldebaran:~$ docker service create --name=rl-frontend  \
   -e BACKEND_SERVER=rl-backend \
   -p 8080:80 --network rlapp-overlay \
   --replicas 1 \
   rlrules/docker-lab-frontend
[...]
```

Pentru a verifica distribuția serviciilor containerizate pe nodurile disponibilite:

```
student@aldebaran:~$ docker service ls
```

```
student@aldebaran:~$ docker service ps <numele serviciului>
```

Pentru a vedea doar acele containere care sunt în running state:

```
student@aldebaran:~$ docker service ps <numele serviciului> -f desired-state=running
```

Pentru simplitate, vom putea utiliza filtre:

```
student@aldebaran:~$ docker service ps $(docker service ls -q) -f desired-state=running
```

Din perspectiva unui orchestrator, Docker SWARM trebuie să se asigure că starea serviciilor pe care le-am lansat este consistentă. În cazul în care un serviciu(container) pică, SWARM trebuie să readucă starea clusterului la cea configurată. Pentru fiecare serviciu lansat anterior, am setat o constrângere particulară, care desemnează o stare a clusterului pe care o vrem constantă: `--replicas 1`. Să validăm faptul că Docker SWARM are capabilitatea de self-healing, aspect ce ne permite să stabilim prerogativele asigurării unui disponibilități crescute a aplicației web de partajare de imagini.

```
student@aldebaran:~$ docker service ps rl-frontend -f desired-state=running
```

În coloana NODE vedem unde a fost planificată execuția containerului asociat serviciului de frontend.

Pe nodul anterior obținut vom șterge container-ul de frontend:

```
student@aldebaran:~$ docker ps -f name=rl-frontend*
```

```
student@aldebaran:~$ docker rm -f <nume container sau id container>
```

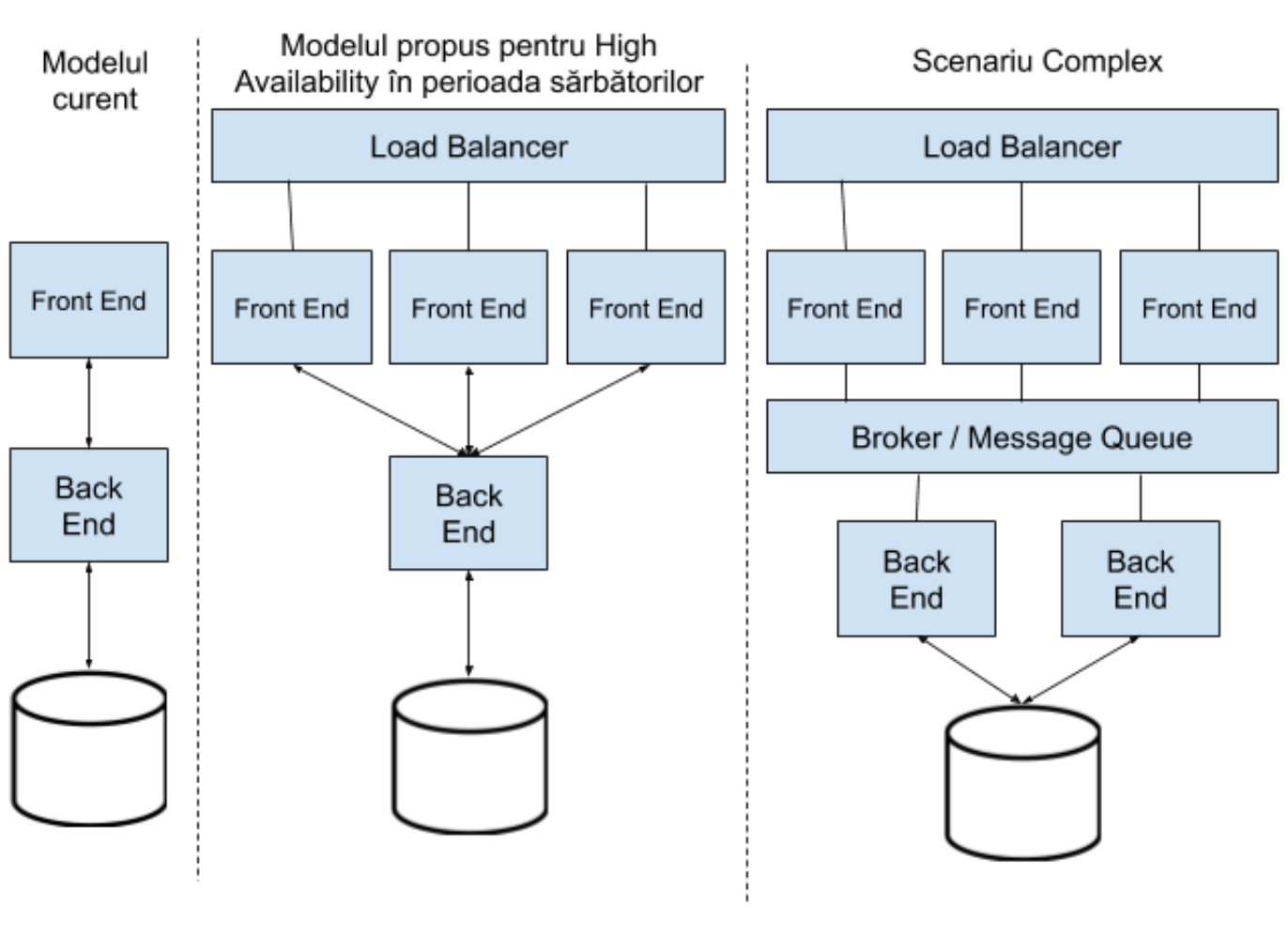
Pe MASTER:

```
student@aldebaran:~$ watch docker service ps rl-frontend -f desired-state=running
```

Se poate vedea cum Docker SWARM a sesizat modificările aduse stării clusterului și a relansat unui nou container de frontend.

Într-un scenariu realist, simpla oprire a execuției unui container nu este singurul scenariu valid, ce trebuie luat în considerare, pentru a stabili dacă un serviciu este disponibil sau nu. Ce se întâmplă dacă aplicația rulează în continuare în container, dar intră într-un context de eroare ca de exemplu: o buclă infinită? Docker poate asigura mecanisme de probing, însă, într-un design matur, efortul de auto-healing trebuie susținut atât de runtime-ul de containerizare cât și de aplicația care rulează în container.

Știm, din incipit, faptul că în perioada sărbătorilor va exista un spike în utilizarea platformei de sharing de poze, astfel încât se conturează scenariul probabil ca serviciul de frontend, expus către exterior, să fie bombardat de cereri HTTP și să cedeze în fața numărului de conexiuni lansate către acesta. O soluție simplă pentru o astfel de problemă o reprezintă multiplicarea serviciului preconizat a fi afectat. Traficul HTTP va trebui, totodată, balansat echitabil între instanțele (replicile) serviciului multiplicat.



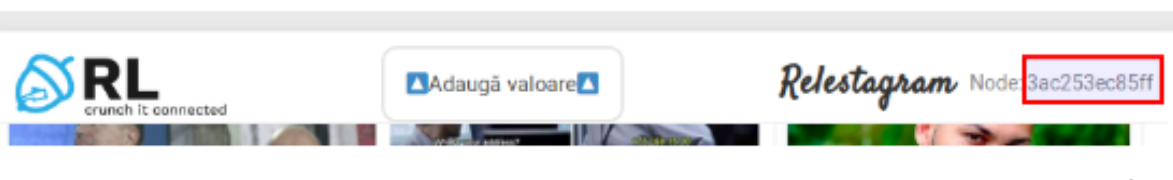
Cu Docker SWARM implementarea unui astfel de model de rețea redundant se poate realiza simplu, întrucât orchestratorul oferă posibilitatea creșterii numărului de replici ale unui container și totodată oferă nativ funcția de balansare între instanțele aplicației replicate. Să validăm aceste capabilități:

```
student@aldebaran:~$ docker service scale rl-frontend=3
```

```
student@aldebaran:~$ docker service ps rl-frontend -f desired-state=running
```

Din browser, incercați să vă conectați la aplicație utilizând IP-ul MASTER-ului atât dintr-o fereastră normală cât și dintr-o fereastră incognito/private.

Dați refresh de mai multe ori paginii. Ce observați?



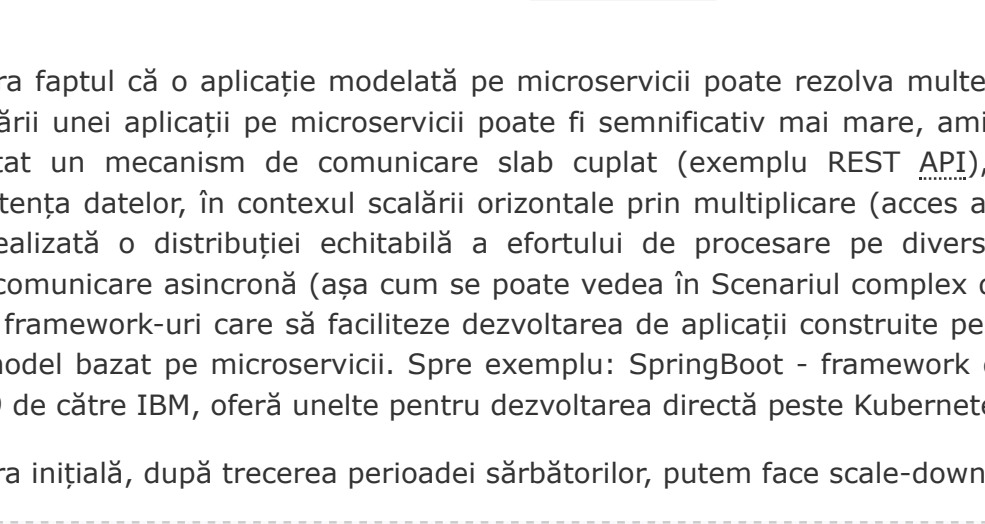
3ac253ec85ff

Id-ul din colțul dreapta sus este id-ul unic al containerului în care rulează instanța de frontend. Întrucât am creat 3 replici ale aplicației, Docker SWARM va balansa traficul(load) către toate instanțele generate.

Mai mult, încercați să vă conectați la aplicație utilizând IP-ul Worker1 sau/și IP-ul Worker2. Funcționează? Un port expus în cadrul unui cluster Docker SWARM este disponibil pe întreaga componentă a clusterului. Astfel, pentru accesa o aplicație expusă este suficient să cunosc una din adresele nodurilor clusterului și portul aplicației, fără a conta explicit unde rulează containerul.

Chiar și cu capabilitatea nativă de balansare pe care o aduce Docker SWARM, în producția reală, în cele mai multe cazuri, se recomandă utilizarea unor appliances dedicate, optimizate pentru funcția de **Load Balancing**.

Revenind la comparația dintre arhitecturi pe microservicii și arhitecturi monolite, observăm oportunitatea de scalare granulară, per serviciu/feature, cu care vine un model loosely coupled. În imaginea de mai jos se poate vedea metoda de scalare orizontală ce poate fi aplicată în ambele abordări: aplicații monolit și aplicații distribuite pe microservicii.



Deși aparențele pot sugera faptul că o aplicație modelată pe microservicii poate rezolva multe din problemele de producție, efortul dezvoltării unei aplicații pe microservicii poate fi semnificativ mai mare, amintind aici faptul că trebuie definit și dezvoltat un mecanism de comunicare slab cuplat (exemplu REST API), trebuie asigurate mecanisme pentru consistența datelor, în contextul scalării orizontale prin multiplicare (acces atomic asupra bazei de date etc.), trebuie realizată o distribuției echitabilă a efortului de procesare pe diversele replici, trebuie asigurate mecanisme de comunicare asincronă (așa cum se poate vedea în Scenariul complex de mai sus) etc. Cu toate acestea piața oferă framework-uri care să faciliteze dezvoltarea de aplicații construite peste o infrastructura de tipcloud și peste un model bazat pe microservicii. Spre exemplu: SpringBoot - framework de Java, Kabanero, lansat în vara anului 2019 de către IBM, oferă unelte pentru dezvoltarea directă peste Kubernetes etc.

Pentru a reveni la structura inițială, după trecerea perioadei sărbătorilor, putem face scale-down:

```
student@aldebaran:~$ docker service scale rl-frontend=1
```

```
student@aldebaran:~$ docker service ps rl-frontend
```

În acest laborator am iterat prin câteva concepte din lumea Docker și totodată am adresat probleme și concepte de system design. Echipa de RL speră să vi le însușiți și să vă fie benefice în carieră.

Navigare

Laboratorul 6

- 00. [0p] Play With Docker
- 01. [20p] Hello Docker
- 02. [20p] Docker Images
- 03. [20p] Docker Networking
- 04. [20p] Docker Volumes
- 05. [20p] Orchestrare cu Docker SWARM
- 06

Cunoștințe acumulate

- Intoducere în Docker CLI
- Docker Container vs Docker Image
- Dockerfile și construcția de Imagini
- CMD vs ENDPOINT
- Union File System: Overlay2 - image layers
- Docker Volumes
- Mount binding vs volumes
- Docker Networking: bridge & overlay (VXLAN)
- Docker SWARM
- Introducere în arhitecturi pe microservicii
- Arhitecturi pe microservicii vs Arhitecturi monolite

Laborator 7. Adresare IP și rutare în Linux

Cunoștințe și abilități ce vor fi dobândite

- Configurare de adrese IP în Linux
- Configurarea rutării în Linux
- Depanarea problemelor de configurare a rețelei în Linux

Cheat sheet

- Cheat Sheet

Pregătire infrastructură de laborator

- Avem nevoie de o masina virtuală a laboratorului. Vă rugăm urmăriți pagina aceasta pentru instrucțiuni, apoi reveniți.
- Pentru a pregăti configurația de laborator, pe mașina virtuală (stația `host`) folosiți comenzile următoare din contul utilizatorului `root` de pe stația `host` (puteți da copy/paste la comenzi în terminal):

```
root@host:~# update_lab --force
root@host:~# start_lab lab7
```

- Deschideți trei noi tab-uri în terminal (folosiți combinația de taste `Ctrl+Shift+t`), și conectați-vă, din nou, la mașina virtuală folosind comanda `ssh` de mai sus.
  - De pe cele trei noi tab-uri, conectați-vă la cele trei containere (`red`, `green` și `blue`).
  - Pentru o conectare mai usoara puteti folosi aliasul `go` (ex. `go red`)

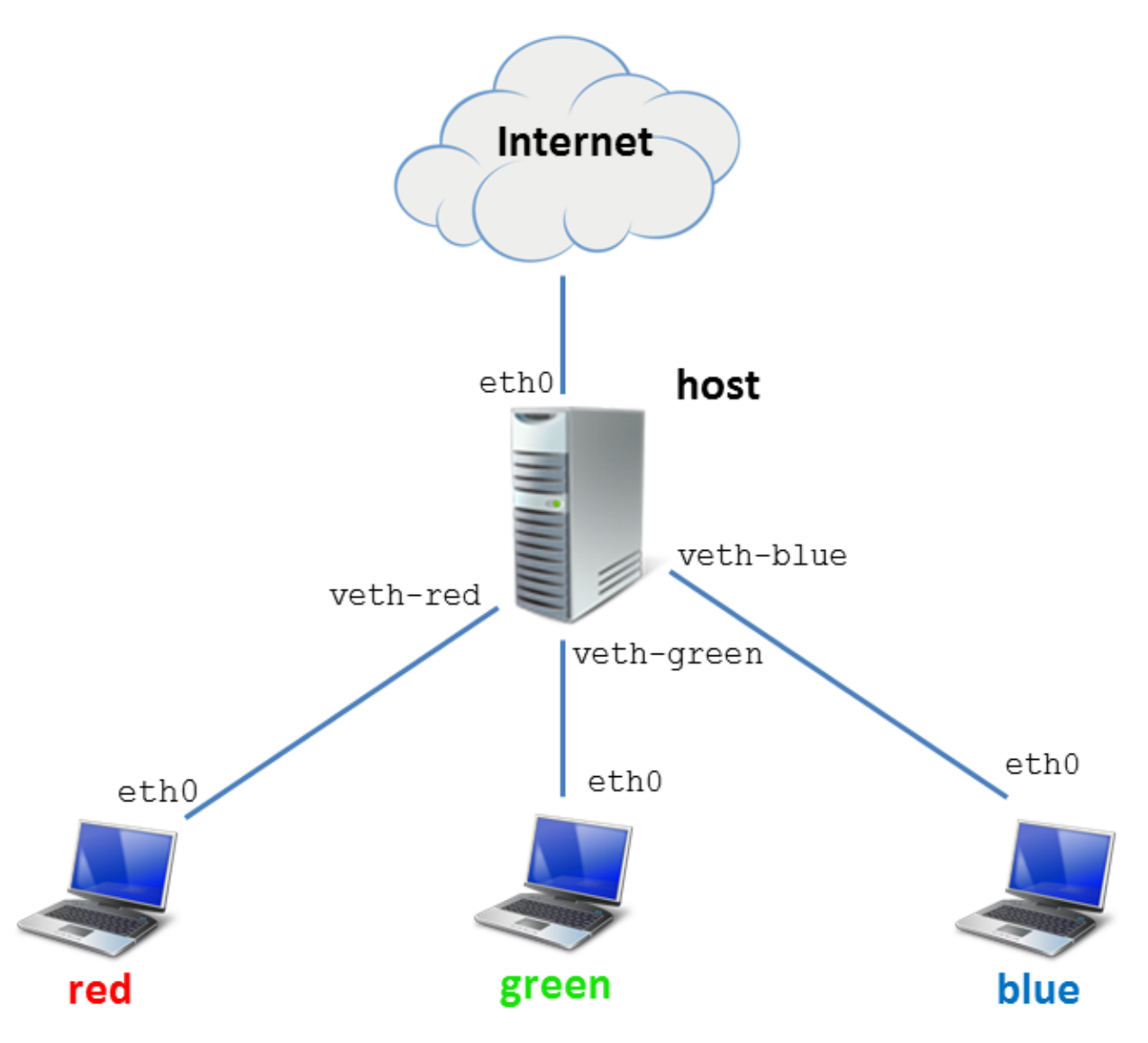
Pentru a vedea cum accesați stațiile `red`, `green` și `blue` (containere Docker configurate peste mașina virtuală VMware - stația `host`) urmăriți indicațiile din pagina cu instrucțiuni de utilizare a mașinii virtuale.

Conturile de acces la mașina virtuală (stația `host`) sunt (`username:parola`):

- `root:student`
- `student:student`

În mod implicit folosiți contul `root` pentru conectare pe toate stațiile. Aveți nevoie de drepturi privilegiate pentru configurare. Folosiți contul `student` doar unde vi se cere explicit.

Topologie



Navigare

Laboratorul 7

- 01. [10p] Configurare și ștergere adrese IP
- 02. [20p] Configurare adrese IP
- 03. [10p] Adresare IP și rutare
- 04. [20p] Configurare conectivitate completă
- 05. [10p] Tabela ARP
- 06. [10p] Depanare problemă de configurare adresă IP
- 07. [10p] Depanare problemă de conectivitate
- 08. [10p] IPv6
- 09. [BONUS - 10p] Configurare persistentă

Exerciții

În cadrul exercițiilor din laboratoarele de Linux vom folosi topologia de mai sus.

01. [10p] Configurare și ștergere adrese IP

Dorim, pentru început, să asigurăm conectivitate între stația `host` și `red`. În acest tutorial vom folosi suita `iproute` de pe Linux pentru a realiza configurările frecvente de nivel 3 (adresare IP).

Vom configura câte o adresă IP din clasa `192.168.0.0/24` pe interfețele de legătură dintre stația `host` și stația `red`. Adică între `host(veth-red)` (interfața `veth-red` de pe stația `host`) și `red(red-eth0)` (interfața `red-eth0` de pe stația `red`).

Pe interfața `veth-red` de pe stația `host` vom configura adresa IP `192.168.0.1` cu masca `255.255.255.0` (`/24` în forma prefixată):

```
root@host:~# ip address add 192.168.0.1/24 dev veth-red
```

Observați că suita `iproute2` (adică utilitarul `ip`) folosește masca în format prefixat: `/24`.

Imediat după o configurare de rețea rulați o comandă pentru validarea configurării. În cazul nostru este comanda de afișare a configurării de nivel 3 (Rețea), adică a adresei IP:

```
root@host:~# ip address show dev veth-red
47: veth-red: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state DOWN qlen 1000
    link/ether 4e:1b:b8:d9:14:bb brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.1/24 scope global veth-red
```

Pe interfața `red-eth0` de pe stația `red` vom configura adresa IP `192.168.0.2` cu masca `255.255.255.0` (`/24` în forma prefixată):

```
root@host:~# go red
[...]
root@red:~# ip address add 192.168.0.2/24 dev red-eth0
root@red:~# ip address show dev red-eth0
46: red-eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state DOWN qlen 1000
    link/ether 00:16:3e:8e:84:21 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.2/24 scope global eth0
    inet6 fe80::216:3eff:fe8e:8421/64 scope link
       valid_lft forever preferred_lft forever
```

La fel, după o configurare de rețea, am rulat comanda de validare, în cazul acesta `ip address`.

Pentru a vă putea întoarce la consola stației `host`, puteți da `exit` (sau deschideți / folosiți alt terminal).

Pentru a testa conectivitatea între stațiile `host` și `red` folosim comanda `ping`:

```
root@host:~# ping 192.168.0.2
PING 192.168.0.2 (192.168.0.2) 56(84) bytes of data.
^C
--- 192.168.0.2 ping statistics ---
2 packets transmitted, 0 received, 100% packet loss, time 1007ms
```

După câteva secunde opriți comanda `ping` folosind combinația de taste `Ctrl+c`.

Observați că nu există conectivitatea între cele două stații: pachetele sunt pierdute în întregime (`100% packet loss`). Motivul este că nu am activat interfețele, ci doar am realizat configurații de nivel 3.

Urmăriți configurația de nivel 2 al interfețelor folosind comanda `ip link`:

```
root@host:~# ip link show dev veth-red
10: veth-red: <BROADCAST,MULTICAST> mtu 1500 qdisc pfifo_fast state DOWN qlen 1000
    link/ether 3e:03:f0:76:76:ab brd ff:ff:ff:ff:ff:ff
```

Observați că interfața nu este activă la nivelul 2 (Legătură de date). Pentru a activa interfața folosiți comanda:

```
root@host:~# ip link set dev veth-red up
```

Urmăriți din nou configurația de nivel 2 (Legătură de date) a interfeței `veth-red` și observați că acum este parțial `UP` (apare și `UP` și `DOWN` în output-ul comenzii):

```
root@host:~# ip link show dev veth-red
10: veth-red: <NO_CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc pfifo_fast state DOWN qlen 1000
    link/ether 3e:03:f0:76:76:ab brd ff:ff:ff:ff:ff:ff
```

Testați din nou conectivitatea folosiți comanda `ping`. În continuare nu există conectivitate. Acest lucru și faptul că apărea și `DOWN` în output-ul comenzii anterioare se datorează faptului că nu am activat interfața `red-eth0` de pe stația `red`. Interfața `red-eth0` de pe stația `red` este cea conectată la interfața `veth-red` de pe stația `host`; ambele trebuie să fie activate pentru a avea o conexiune activă.

Pe stația `red` verificați configuratia de nivel 2 a interfeței `red-eth0` de pe `red`. Observați că este `DOWN` și activați nivelul 2 dacă este cazul, folosind comanda

```
root@red:~# ip link set dev red-eth0 up
```

Verificați că acum interfața este activă folosind comanda

```
root@red:~# ip link show dev red-eth0
```

Folosiți comanda `ping` ca să retestați conectivitatea între stațiile `host` și `red`.

Dorim să resetăm configurația la configurația inițială. Pentru acesta rulați o comandă de forma

```
# ip address flush dev INTERFACE
```

unde `INTERFACE` este interfața interfața `veth-red` pe stația `host`, respectiv `red-eth0` pe stația `red`. Asigurați-vă că nu mai este configurată nici o adresă IP pe interfețe folosind o comandă de forma

```
# ip address show dev INTERFACE
```

unde `INTERFACE` este interfața interfața `veth-red` pe stația `host`, respectiv `red-eth0` pe stația `red`.

02. [20p] Configurare adrese IP

Dorim să avem conectivitate între stația `red` și stația `host`, respectiv între stația `green` și stația `host`. Pentru aceasta, vom configura adrese IP pe fiecare.

Configurați câte o adresă IP din clasa 10.10.10.0/24 pe legătura dintre stația `red` și stația `host` (adică legătura `red(red-eth0)` ↔ `host(veth-red)`) și testați conectivitatea.

Configurați câte o adresă IP din clasa 10.10.20.0/24 pe legătura dintre stația `green` și stația `host` (adică legătura `green(green-eth0)` ↔ `host(veth-green)`) și testați conectivitatea.

Aveți în vedere să verificați nivelul *Legătură de date* folosind comanda `ip link` și să activați, la nevoie, interfețele.

03. [10p] Adresare IP și rutare

Dorim să realizăm conectivitate și între stațiile `red` și `green`. Întrucât cele două stații sunt în rețele locale diferite, va trebui să configurăm stația `host` ca *default gateway* pe fiecare stație.

Pentru a adăuga *default gateway* pe stația `red` folosiți comenzile:

```
root@host:~# go red
[...]
root@red:~# ip route add default via 10.10.10.1
```

După configurare (adăugarea rutei), validăm configurația cu o comandă specifică. În acest caz urmărim tabela de rutare folosind comanda:

```
root@red:~# ip route show
default via 10.10.10.1 dev red-eth0
10.10.10.0/24 dev red-eth0  proto kernel  scope link  src 10.10.10.2
```

Adresa IP `10.10.10.1` reprezintă adresa IP a interfeței `veth-red` de pe stația `host`.

Intrați pe stația `green` și executați comenzile similar.

Testați conectivitatea, folosind `ping` între stațiile `green` și `red`. Observați că nu funcționează. Motivul pentru care nu există conectivitate este reprezentat de faptul că stația `host` nu are activată rutarea (nu trimite pachetele ce vin de pe o interfață pe altă interfață). Pentru a activa rutarea pe stația `host`, rulați comanda:

```
root@host:~# sysctl -w net.ipv4.ip_forward=1
```

Pentru a valida configurarea de activare a rutării, rulăm comanda:

```
root@host:~# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

Testați din nou conectivitatea între `red` și `green` și observați că funcționează.

Porniți comanda `ping` de pe stația `red` către stația `green`. Deschideți un nou terminal și executați pe stația `host` comanda:

```
root@host:~# tcpdump -n -i veth-red
listening on veth-red, link-type EN10MB (Ethernet), capture size 65535 bytes
18:46:48.783576 IP red.local > 10.10.20.2: ICMP echo request, id 434, seq 163, length 64
18:46:48.783622 IP 10.10.20.2 > red.local: ICMP echo reply, id 434, seq 163, length 64
```

Observați pachetele de tip `ICMP echo request/reply` care trec prin stația `host` (sau altfel zis stația `host` le rutează).

04. [20p] Configurare conectivitate completă

Dorim să asigurăm conectivitate completă între toate stațiile din topologie. Trebuie configurată corespunzător stația `blue`.

Configurați adrese IP din clasa 10.10.30.0/24 pe legătura dintre stația `host` și stația `blue` (adică între `host(veth-blue)` și `blue(blue-eth0)`).

Țineți cont să verificați legătura de nivel 2 folosind comanda `ip link`.

Testați conectivitatea între stația `host` și stația `blue`.

Pe stația `blue` configurați ca *default gateway* stația `host`, pentru a permite conectivitata la celelalte stații.

Pe stația `blue` folosiți ca *default gateway* adresa IP de pe interfața `veth-blue` a stației `host`.

Testați conectivitatea între oricare două stații.

05. [10p] Tabela ARP

ARP (*Address Resolution Protocol*) este protocol care face intern fiecărui sistem de operare asocierea între adresele IP și adresele MAC ale stațiilor cu care comunică. De multe ori stațiile cunosc adresele IP ale vecinilor dar nu știu adresele MAC; protocolul ARP completează o tabelă ARP locală sistemului cu intrările necesare. Protocolul ARP este rulat implicit de sistemul de operare atunci când se comunică cu o stație a cărei adresă MAC nu este cunoscută.

Ne propunem să urmărim tabela ARP a unui sistem Linux.

Pe stația `host` urmăriți tabela ARP folosind comanda:

```
root@host:~# ip neighbor show
[...]
```

Este posibil ca tabela să fie vidă (neexistând comunicație recentă) sau să aibă unele intrări (cele mai recente comunicații) sau intrările să fie marcate `STALE` (intrări nesigure).

Pentru a popula tabela ARP inițiați comunicație cu celelalte stații folosind comanda `ping`:

```
root@host:~# ping -c 1 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_req=1 ttl=64 time=0.033 ms
[...]
root@host:~# ping -c 1 10.10.20.2
PING 10.10.20.2 (10.10.20.2) 56(84) bytes of data.
64 bytes from 10.10.20.2: icmp_req=1 ttl=64 time=0.036 ms
[...]
root@host:~# ping -c 1 10.10.30.2
PING 10.10.30.2 (10.10.30.2) 56(84) bytes of data.
64 bytes from 10.10.30.2: icmp_req=1 ttl=64 time=0.080 ms
[...]
```

Urmăriți din nou tabela ARP:

```
root@host:~# ip neighbor show
10.10.10.2 dev veth-red lladdr 00:16:3e:8e:84:21 REACHABLE
10.10.20.2 dev veth-green lladdr 00:16:3e:d1:b2:95 REACHABLE
10.10.30.2 dev veth-blue lladdr 00:16:3e:32:0f:ae REACHABLE
10.8.0.1 dev eth0 lladdr 0a:00:27:00:00:00 REACHABLE
```

Observați că fiecare stație (`red`, `green` și `blue`) are câte o intrare aferentă în tabela ARP marcată cu `REACHABLE` (intrare validă). Intrarea suplimentară este comunicarea mașinii virtuale (stației `host`) cu sistemul `fep.grid.pub.ro`.

Realizați pașii de mai sus pentru fiecare dintre stațiile `red`, `green` și `blue`:

1. Urmăriți tabela ARP.
2. Inițiați comunicație cu celelalte stații pentru a popula tabela ARP.
3. Urmăriți din nou tabela ARP.

Observați că în tabela ARP a fiecăreia dintre stațiile `red`, `green` și `blue` se găsește câte o intrare ARP, corespunzătoare stației `host`. Aceasta se întâmplă întrucât comunicațiile trec prin *default gateway* (adică prin stația `host`) iar fiecare stație trebuie să cunoască doar adresa MAC a gateway-ului.

06. [10p] Depanare problemă de configurare adresă IP

Rulați scriptul de pregătire cu noul argument `ex6`:

```
root@host:~# start_lab lab7 ex6
```

În urma rulării scriptului au fost repornită stațiile `red` și au fost refăcute configurațiile. Va trebui să vă reconectați pe stația `red` folosind comanda:

```
root@host:~# go red
```

și folosind username-ul `root` și parola `student`.

Scriptul configurează adresa IP `7.7.7.1` pe interfața `veth-red` a stației `host` și adresa IP `7.7.7.2` pe interfața `red-eth0` a stației `red`. Folosiți comenzile:

```
root@red:~# ip address show red-eth0
46: red-eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 00:16:3e:8e:84:21 brd ff:ff:ff:ff:ff:ff
    inet 7.7.7.2/32 scope global red-eth0
    inet6 fe80::216:3eff:fe8e:8421/64 scope link
       valid_lft forever preferred_lft forever

root@host:~# ip address show veth-red
47: veth-red: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 4e:1b:b8:d9:14:bb brd ff:ff:ff:ff:ff:ff
    inet 7.7.7.1/32 scope global veth-red
```

pentru a urmări configurația IP pe interfața `INTERFACE`.

Folosiți comanda `ping` pentru a testa conectivitatea între cele două adrese IP (`7.7.7.1` și `7.7.7.2`) pe cele două stații. Observați ca nu există conectivitate.

Pentru a depana această problemă, urmărim tabela de rutare a fiecărei stații:

```
root@red:~# ip r s
root@red:~#

root@host:~# ip r s
10.10.20.0/24 dev veth-green  proto kernel  scope link  src 10.10.20.1
10.10.30.0/24 dev veth-blue  proto kernel  scope link  src 10.10.30.1
192.168.56.0/24 dev eth0  proto kernel  scope link  src 192.168.56.101
```

Observați că nu apar rutele relevante în tabela de rutare. Fie interfețele sunt dezactivate, fie configurația este greșită. Dacă rulăm comenzile de afișare a informațiilor de nivel 3, observăm:

```
root@red:~# ip address show red-eth0
46: red-eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 00:16:3e:8e:84:21 brd ff:ff:ff:ff:ff:ff
    inet 7.7.7.2/32 scope global red-eth0
    inet6 fe80::216:3eff:fe8e:8421/64 scope link
       valid_lft forever preferred_lft forever

root@host:~# ip address show veth-red
47: veth-red: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP qlen 1000
    link/ether 4e:1b:b8:d9:14:bb brd ff:ff:ff:ff:ff:ff
    inet 7.7.7.1/32 scope global veth-red
```

Interfețele sunt active (`UP`). Observăm însă că aceste adrese au masca `/32`. Acest lucru înseamnă că nu pot comunica în rețea unele cu altele și explică și absența rutei relevante din tabela de rutare. Repararea acestei greșeli se face prin folosirea unor adrese cu mască potrivită (de exemplu, pentru o mască `/24` este vorba de `7.7.7.1/24` și `7.7.7.2/24`).

O greșeală relativ frecventă în configurarea adresei IP în Linux este omiterea măștii de rețea. Aveți în vedere să nu omiteți masca în momentul în care configurați adrese IP pe interfețe în Linux.

Listarea tabelei de rutare a unei stații este printre primii pași care trebuie urmați pentru depanarea unei probleme de conectivitate.

07. [10p] Depanare problemă de conectivitate

Ne propunem să depanăm o problemă de conectivitate. Pentru a "genera" problema rulați scriptul de pregătire cu noul argument `ex7`:

```
root@host:~# start_lab lab7 ex7
```

Pentru depanare, primul pas recomandat este afișarea tabelei de rutare. Tabela de rutare vă va ajuta pentru depanare în cazul în care anumite intrări sunt absente sau configurate greșit.

Verificați conectivitatea între toate stațiile din topologie. Observați că nu există conectivitate între nici o stație și stația `blue`. Rezolvați problemele de conectivitate la stația `blue` de pe stația `host`.

Nu schimbați adresele IP deja configurate. Doar identificați și soluționați problemele.

08. [10p] IPv6

Dorim să asigurăm conectivitate IPv6 între stația host și red. În acest tutorial vom folosi suita `iproute` din Linux pentru a realiza configurările necesare. Folositi parametrul `-6` pentru a face setarile aferente IPv6.

Vom configura câte o adresă IP din clasa `2201::/64` pe interfețele de legătură dintre stația `host` și stația `red`. Adică între `host(veth-red)` (interfața `veth-red` de pe stația `host`) și `red(red-eth0)` (interfața `red-eth0` de pe stația `red`).

Pe interfața `veth-red` de pe stația `host` vom configura adresa IP `2201::1/64`

```
root@host:~# ip -6 address add 2201::1/64 dev veth-red
```

Imediat după o configurare de rețea rulați o comandă pentru validarea configurării. În cazul nostru este comanda de afișare a adresei IPv6:

```
root@host:~# ip -6 address show dev veth-red
47: veth-red: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP qlen 1000
    inet6 2201::1/64 scope global
       valid_lft forever preferred_lft forever
    inet6 fe80::215:5dff:fe5b:a38e/64 scope link
       valid_lft forever preferred_lft forever
```

Pe interfața `red-eth0` de pe stația `red` vom configura adresa IP `2201::2/64`:

Configurați pe legătura host-blue adrese IPv6 din rețeaua 2202::/64 și pe legătura host-green adrese IPv6 din rețeaua 2203::/64. Activați rutarea și verificați conectivitatea între containere folosind comanda **ping**.

09. [BONUS - 10p] Configurare persistentă

Dorim ca la o reponire a unei stații configurațiile de nivel 3 (adresele IP) să se păstreze. Configurările pe care le-am făcut până acum sunt temporare și sunt pierdute la repornirea stației. În Linux, persistența configurațiilor se realizează prin plasarea acestora în fișiere text specifice, fiecare distribuție (ex.: Debian, RedHat) având propriul mod de configurare.

Pentru a pregăti exercițiul, rulați scriptul de pregătire:

```
root@host:~# start_lab lab7
root@host:~# ip address flush dev veth-red
```

Realizați **persistent** configurația de la exercițiul 08. [10p] IPv6 (adrese IP și *default gateway*) pentru `host`. Distribuția de Linux folosită în laborator este Debian-based.

Pentru detalii despre cum puteți face configurații persistent pe sisteme Debian, consultați această pagină. Veți realiza o configurație statică.

După ce ați realizat configurațiile necesare pentru red, executați pe host:

```
ifdown veth-red
ifup veth-red
```

Observați că la executarea comenzii `ifup` interfața a preluat configurările din fișier.

Configurați sistemul astfel încât rutarea să fie activată la pornirea sistemului.

Pentru informații legate de activarea rutări, consultați această pagină.

Reporniți mașina virtuală (stația `host`), folosind comanda:

```
root@host:~# reboot
```

După repornire ar trebui să aveți configurațiile activate și conectivitate completă la nivelul topologiei.

Navigare

Laboratorul 7

- 01. [10p] Configurare și ștergere adrese IP
- 02. [20p] Configurare adrese IP
- 03. [10p] Adresare IP și rutare
- 04. [20p] Configurare conectivitate completă
- 05. [10p] Tabela ARP
- 06. [10p] Depanare problemă de configurare adresă IP
- 07. [10p] Depanare problemă de conectivitate
- 08. [10p] IPv6
- 09. [BONUS - 10p] Configurare persistentă

Laborator
Laboratorul 7
- 01. [10p] Configurare și ștergere adrese IP
- 02. [20p] Configurare adrese IP
- 03. [10p] Adresare IP și rutare
- 04. [20p] Configurare conectivitate completă
- 05. [10p] Tabela ARP
- 06. [10p] Depanare problemă de configurare adresă IP
- 07. [10p] Depanare problemă de conectivitate
- 08. [10p] IPv6
- 09. [BONUS - 10p] Configurare persistentă

Curs
Informații generale RL
- Regulament
  - Notare Seria CA
  - Notare Seriile CB/CC
- Resurse
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri
- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare
- Laborator 1. Introducere în Rețele de Calculatoare
- Laborator 2. Comutarea pachetelor
- Laborator 3. Adresarea IP
- Laborator 4. VLAN (Virtual LAN)
- Laborator 5. Rutare
- Laborator 6. Virtualizare în Docker
- Laborator 7. Adresare IP și rutare în Linux
- Laborator 8. Securizarea unui server
- Laborator 9. Translatarea de adrese
- Laborator 10. Clienți de rețea
- Laborator 11. Securitatea Rețelelor Locale
- Laborator 12. Amazon Web Services
- Laborator extra - Virtualizare în Networking
- Laborator Recapitulare

Teme
- Tema1
- Tema2

Laborator 8. Securizarea unui server

Cunoștințe și abilități ce vor fi dobândite

- Cunoștințe generale legate de securizarea rețelei
- Avantajele și dezavantajele formelor de transfer securizat (criptat) și nesecurizat (în clar)
- Folosirea utilitarului `iptables` pentru configurarea unui firewall pe Linux
- Folosirea protocolului SSH și a utilitarelor aferente pentru conectarea la distanță și transferul securizat de fișiere

Cheat sheet

- Cheat Sheet

Pregătire infrastructură de laborator

- **Reminder**: avem nevoie de o masina virtuală a laboratorului. Vă rugăm urmăriți pagina aceasta pentru instrucțiuni, apoi reveniți.
- Schimbati utilizatorul curent ca `root` folosind comanda

```
student@host:~$ sudo su
```

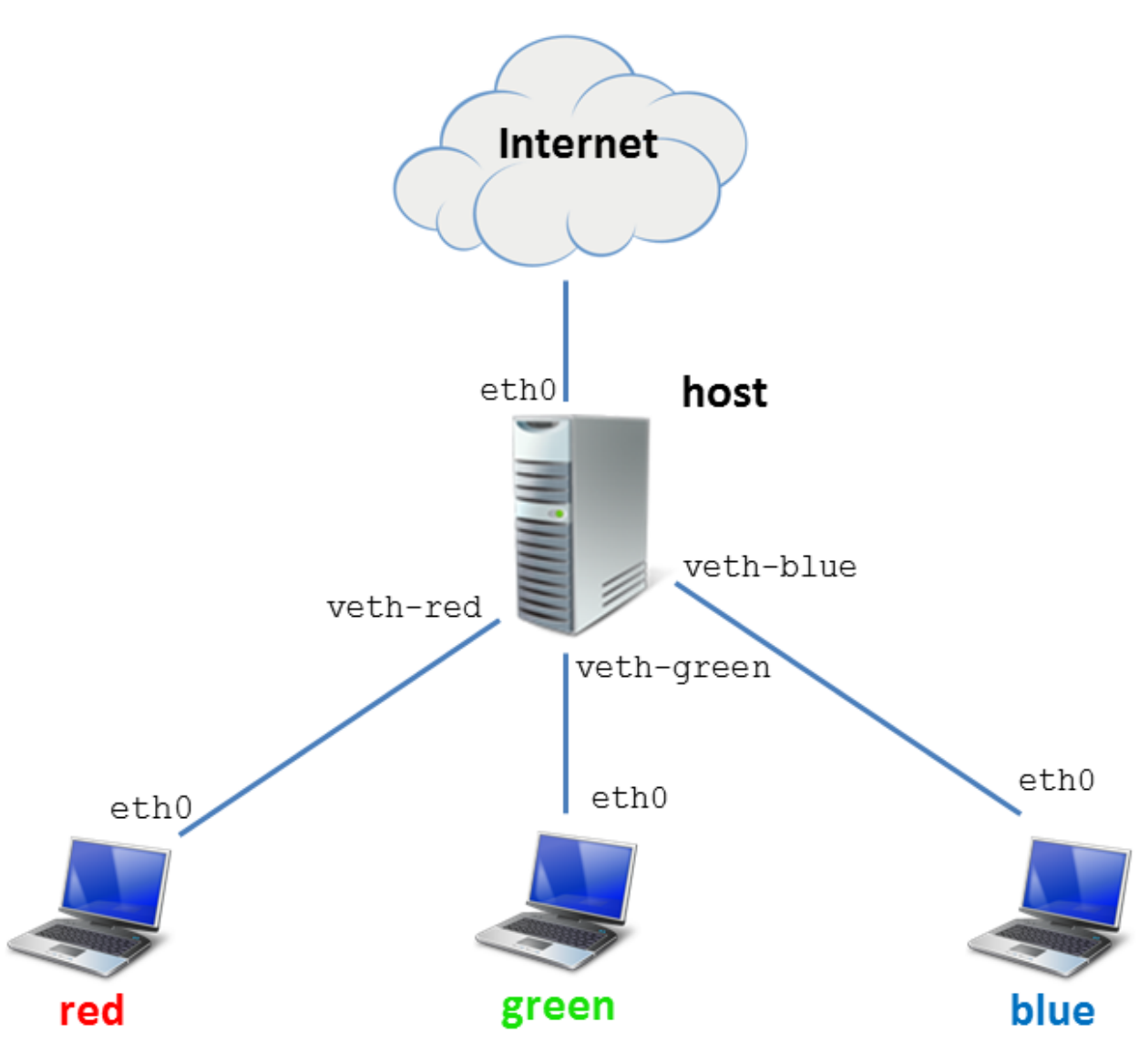
- Pentru a pregăti configurația de laborator, pe mașina virtuală (stația `host`) folosiți comenzile următoare din contul utilizatorului `root` de pe stația `host` (puteți da copy/paste la comenzi în terminal):

```
root@host:~# update_lab --force
root@host:~# start_lab lab8
```

- Deschideți trei noi tab-uri în terminal (folosiți combinația de taste `Ctrl+Shift+T`), și conectați-vă, din nou, la mașina virtuală folosind comanda `ssh` de mai sus.
- De pe cele trei noi tab-uri, conectați-vă la cele trei containere (`red`, `green` și `blue`).
- Pentru o conectare mai usoara puteti folosi aliasul `go` (ex. `go red`)

În mod implicit folosiți contul `root` pentru conectare pe toate stațiile. Aveți nevoie de drepturi privilegiate pentru configurare. Folosiți contul `student` doar unde vi se cere explicit.

Topologie



Navigare

Laboratorul 8

- 1. [10p] Conectare SSH folosind cheie publică
- 2. [10p] Generare cheie publică și autentificare
- 3. [10p] Download și upload de director folosind "scp"
- 4. [10p] Copiere fișiere cu diverse protocoale: durată și consum de resurse
- 5. [10p] Trafic criptat și necriptat
- 6. [10p] Blocare servicii necriptate
- 7. [10p] Blocare SSH
- 8. [10p] Permitere trafic SSH
- 9. [10p] Ștergere reguli adăugate
- 10. [10p] Capturi de trafic
- 11. [BONUS - 10p] Blocare acces din Internet la rețeaua locală
- 12. [BONUS - 10p] Transfer sincronizat de fișiere folosind "rsync" peste SSH

Exerciții

Pentru a primi întregul punctaj va trebui ca la finalul laboratorului să ștergeți mașina virtuală pornită și să îi arătați asistentului listarea instanțelor din OpenStack.

1. [10p] Conectare SSH folosind cheie publică

Dorim să realizăm o sesiune securizată shell la distanță (*remote shell*).

Sumar exerciții conectare SSH (dați click)

Din contul `student` de pe stația `red` conectați-vă la stația `host` prin SSH prin rularea comenzii

```
student@red:~$ ssh student@host
[...]
student@host:~$
```

Observați faptul că v-ați conectat direct, fără parolă, întrucât autentificarea s-a realizat folosind cheie publică. Folosiți combinația de taste `Ctrl+d` sau comanda `exit` pentru a închide sesiunea de shell la distanță.

Cheia publică folosită la conectare este disponibilă în fișierul `~/.ssh/id_rsa.pub`. Pentru vizualizarea cheii rulați, pe stația `red` din contul utilizatorului `student`, comanda

```
student@red:~$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1[...] student@red
```

Un alt format de comandă SSH de conectare este cel de mai jos. Pe stația `red` rulați comanda

```
student@red:~$ ssh -l student host
[...]
student@host:~$
```

La fel, închideți sesiunea de shell la distanță prin folosirea combinației de taste `Ctrl+d`.

La distanță, cheia publică este stocată, împreună cu alte chei publice folosite pentru conectare, în fișierul `~/.ssh/authorized_keys`. Rulați comanda de mai jos pentru a confirma prezența cheii publice la distanță

```
student@red:~$ ssh -l student host "cat ~/.ssh/authorized_keys"
ssh-rsa AAAAB3N[...] student@blue
ssh-rsa AAAAB3N[...] student@green
ssh-rsa AAAAB3N[...] student@red
ssh-rsa AAAAB3N[...] student@host
```

Se observă că există cheia `student@red` în fișierul de la distanță de pe stația `host`, deci se poate realiza autentificarea SSH pe bază de chei.

2. [10p] Generare cheie publică și autentificare

În contul utilizatorului `corina` de pe stația `blue` generați o pereche cheie publică/cheie privată SSH prin rularea comenzii

```
corina@blue:~$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/corina/.ssh/id_rsa):
Created directory '/home/corina/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/corina/.ssh/id_rsa.
Your public key has been saved in /home/corina/.ssh/id_rsa.pub.
[...]
```

Folosiți `ENTER` pentru a folosi căile implicite. Nu folosiți passphrase (adică apăsați `ENTER` când vi se solicită passphrase-ul). Cheile sunt generate respectiv, în fișierele, `.ssh/id_rsa` (cheia privată) și `.ssh/id_rsa.pub` (cheia publică).

Realizați operațiile necesare pentru a permite autentificarea pe bază de chei din contul utilizatorului `corina` de pe stația `blue` în contul utilizatorului `student` de pe stația `host`. După aceea, verificați faptul că autentificarea se face pe baza de chei.

Aceștia sunt pașii pe care îi veți urma pentru a configura conectare SSH pe bază de chei:

1. generare pereche de chei SSH (folosind comanda `ssh-keygen`)
2. copiere cheie publică în contul de la distanță (folosind comanda `ssh-copy-id`)
3. conectare la distanță (folosind comanda `ssh`)

3. [10p] Download și upload de director folosind "scp"

Indicație: Pentru download-ul și upload-ul unui director folosiți opțiunea `-r` a comenzii `scp`.

Vom folosi două directoare:

- directorul `assignment/` din directorul home al utilizatorului `student` de pe stația `host`;
- directorul `solution/` din directorul home al utilizatorului `corina` de pe stația `blue`.

Din contul `corina` de pe stația `blue`, **descărcați** directorul `assignment/` **din** directorul home al utilizatorului `student` de pe stația `host`.

Din contul `corina` de pe stația `blue`, **uploadați** directorul `solution/` **în** directorul home al utilizatorului `student` de pe stația `host`.

4. [10p] Copiere fișiere cu diverse protocoale: durată și consum de resurse

Ne propunem să măsurăm timpul de copiere și consumul de resurse pentru un transfer realizat între două stații folosind, pe rând, transfer direct, FTP și SSH. În directorul home al utilizatorului `student` de pe stația `green` există un fișier `file-100M.dat`. Vom transfera acest fișier în directorul home al utilizatorului `student` de pe stația `host`. Vom folosi transfer direct, transfer FTP și transfer SSH. Pentru fiecare caz, stația `host` va rula server-ul, iar stația `green` clientul.

Dați copy-paste la comenzile de mai jos ca să vă asigurați că le scrieți corect. Pentru paste folosiți, în terminal, combinația de taste `Shift+Insert`.

Transfer direct

Pentru transfer direct vom porni un server simplu TCP pe stația `host` și un client pe stația `green`; vom folosi utilitarul `netcat`, prescurtat și `nc`. Rulați pe stația `host`, în contul utilizatorului `student`, comanda

```
student@host:~$ nc -l 12345 > file-100M-nc.dat
```

Comanda deschide un server TCP care ascultă conexiuni pe portul `12345`. Comanda rămâne activă în așteptarea unei conexiuni de la un client.

De pe stația `green`, din contul utilizatorului `student`, transferați fișierul și măsurați timpul de transfer și consumul de resurse prin rularea comenzii[1]

```
student@green:~$ time cat file-100M.dat | nc -q0 host 12345
```

Comanda de mai sus trimite un fișier comenzii `nc`, iar comanda `nc` are rolul unui client TCP care se conectează la stația `host` pe portul `12345`.

Transfer FTP

Pentru transfer FTP, serverul de FTP este deja pornit pe stația `host`. De pe stația `green`, din contul utilizatorului `student`, transferați fișierul și măsurați timpul de transfer și consumul de resurse prin rularea comenzii

```
student@green:~$ time curl -T file-100M.dat -u student:student ftp://host/file-100M-ftp.dat
```

Transfer SSH

Pentru transfer SSH, serverul de SSH este deja pornit și configurat pe stația `host`. De pe stația `green`, în contul utilizatorului `student`, transferați fișierul și măsurați timpul de transfer și consumul de resurse prin rularea comenzii

```
student@green:~$ time scp file-100M.dat student@host:file-100M-scp.dat
```

Urmăriți timpii de rulare și consumul de memorie și de procesor pentru cele trei cazuri de mai sus. Observați valorile ridicate pentru SSH pentru timpul utilizator (*User time*) și procentul de procesor (*Percent of CPU*). Acesta valori ridicate se datorează componentei de criptare a SSH: această componentă asigură securizarea datelor cu un cost de reducere de performanță.

5. [10p] Trafic criptat și necriptat

Traficul generat de servicii se clasifică în **trafic criptat** și **trafic în clar**. Traficul **în clar (necriptat)** poate fi interpretat și înțeles dacă este capturat. Traficul criptat nu poate fi interpretat în absența cheii de criptare; doar transmițătorul și receptorul cunosc cheia pentru a putea comunica.

Ne propunem să analizăm, din punctul de vedere al criptării traficului, următoarele protocoale/servicii:

- telnet (port TCP 23)
- SSH (port TCP 22)
- FTP (port TCP 21)

Vom folosi `tcpdump`, un utilitar cu ajutorul căruia putem captura pachetele care trec printr-un anumit server, pentru a afișa datele transmise (parolă). Vom folosi topologia de laborator și ne vom conecta de la stația `red` la stația `green` prin intermediul stației `host`.

Pentru a porni procesul de captură, autentificați-vă ca `root` pe stația `host` și rulați comanda

```
root@host:~# tcpdump -vvv -i veth-green
```

Pentru comunicație prin telnet, rulați pe stația `red` comanda

```
root@red:~# telnet green
```

În urma rulării comenzii ați realizat o conexiune telnet de la stația `red` la stația `green` prin intermediul stației `host`. La prompt-ul generat de comandă folosiți username-ul `student` și parola `student`; după aceea rulați comanda `ls` și apoi comanda `exit` pentru a închide conexiunea. Observați pe stația `host` captura credențialelor (username și parolă) transmise prin telnet între stațiile `red` și `green` ⇒ traficul telnet între cele două stații a fost **trafic în clar** și a fost capturat pe stația `host`.

Pentru comunicație prin FTP, rulați pe stația `red` comanda

```
root@red:~# ftp green
```

În urma rulării comenzii ați realizat o conexiune FTP de la stația `red` la stația `green` prin intermediul stației `host`. La prompt-ul generat de comandă folosiți username-ul `student` și parola `student`; după aceea rulați comanda `ls` și apoi comanda `quit` pentru a închide conexiunea. Observați pe stația `host` captura credențialelor (username și parolă) transmise prin FTP între stațiile `red` și `green` ⇒ traficul FTP între cele două stații a fost **trafic în clar** și a fost capturat pe stația `host`.

Pentru comunicație prin SSH, rulați pe stația `red` comanda

```
root@red:~# ssh -l student green
```

În urma rulării comenzii ați realizat o conexiune SSH de la stația `red` la stația `green` prin intermediul stației `host`. În sesiunea de shell deschisă la distanță, rulați comanda `ls` și apoi comanda `exit` pentru a închide conexiunea. Observați că pe stația `host`, `tcpdump` nu afișează informații în clar, traficul SSH între cele două stații fiind **criptat** și transmis într-un format binar.

Pe stația `host`, pentru a opri comanda `tcpdump` folosiți combinația de taste `Ctrl+c`.

Traficul telnet și FTP este trafic necriptat (în clar), în timp ce traficul SSH este trafic criptat.

6. [10p] Blocare servicii necriptate

Prezentare scurtă iptables (dați click)

După cum ați observat la punctul anterior, traficul pentru protocoalele telnet și FTP este trafic în clar, necriptat, putându-se afla cu ușurință credențialele unui anumit cont și comenzile rulate.

Ne propunem să blocăm accesul de la stația `red` către stația `green` pentru aceste servicii, configurând ruterul dintre cele două stații, adică stația `host`. Practic vom configura pe stația `host` opțiuni de firewall cu ajutorul utilitarului `iptables`.

Autentificați-vă prin SSH ca `root` pe stația `host`. Pentru a bloca accesul la serviciul telnet (port 23) destinat stației `green`, rulați pe stația `host` comanda de mai jos. Comanda adaugă regula `iptables` corespunzătoare.

```
root@host:~# iptables -A FORWARD -d green -p tcp --dport telnet -j REJECT
```

Pentru a verifica adăugarea regulii de mai sus, rulați pe stația `host` comanda

```
root@host:~# iptables -L FORWARD
Chain FORWARD (policy ACCEPT)
target     prot opt source               destination
REJECT     tcp  --  anywhere             green                tcp dpt:telnet reject-with icmp-pc
```

Pentru a afișa informații și despre pachetele prelucrate și interfețele folosite, rulați pe stația `host` comanda

```
root@host:~# iptables -L FORWARD -v
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 REJECT     tcp  --  any    any     anywhere             green                tcp dpt
```

Pentru a afișa informații în format numeric (pentru nume de stații și nume de porturi), rulați pe stația `host` comanda

```
root@host:~# iptables -L FORWARD -v -n
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 REJECT     tcp  --  *      *       0.0.0.0/0            192.168.2.2          tcp dpt
```

De acum înainte recomandăm folosirea acestor opțiuni (`-v -n`) pentru listarea regulilor `iptables`.

Pentru a verifica blocarea traficului telnet către `green`, rulați pe stația `red` comanda

```
telnet green
```

Vă apare un mesaj de forma

```
Trying 192.168.2.2...
telnet: Unable to connect to remote host: Connection refused
```

semnificând faptul că se încearcă realizarea conexiunii dar conexiunea este respinsă

Pentru a vedea că regula de blocare a funcționat, rulați din nou pe stația `host` comanda

```
root@host:~# iptables -L FORWARD -v -n
Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    2   120 REJECT     tcp  --  *      *       0.0.0.0/0            192.168.2.2          tcp dpt
```

Observați, în output-ul comenzii, că există acum valori diferite de 0 în coloana `pkts` și `bytes`, semn că au fost pachete prelucrate de această regulă, deci blocate.

Pentru a verifica funcționarea în continuare a altor conexiuni (diferite de telnet) de la `red` la `green`, rulați pe stația `red` comenzile

```
ftp green
ssh -l student green
```

Dorim să blocăm și celălalt serviciu necriptat, FTP. Adăugați o regulă `iptables` similară pentru a bloca, pe stația `host`, traficul FTP destinat stației `green`. După adăugarea regulii folosiți `iptables -L FORWARD -n -v` pentru a valida adăugarea regulii.

Pentru această regulă puteți transmite argumentul `21` opțiunii `--dport` sau chiar numele `ftp`. Asocierea între port (număr) și protocol (nume) se găsește în fișierul `/etc/services`.

De pe stația `red` verificați blocarea traficului FTP către stația green folosind comanda

```
ftp green
```

7. [10p] Blocare SSH

Ne propunem ca stația `green` să nu fie acesibilă nici prin SSH. Pentru aceasta adăugați pe stația `host` o regulă `iptables` care va bloca traficul aferent serviciului SSH (portul 22).

Verificați adăugarea regulii `iptables` și apoi verificați de pe stația `red` blocarea traficului SSH către stația `green`.

8. [10p] Permitere trafic SSH

În acest moment, traficul SSH către stația `green` este blocat.

Dorim să permitem traficul SSH **de la** stația `red` **către** stația `green`. Adăugați o regulă corespunzătoare pe stația `host`.

După ce ați adăugat regula, încercați realizarea unei conexiuni SSH de la stația `red` la stația `green`. Observați că nu se realizează conexiunea.

Afișați lista de reguli `iptables` de pe stația `host`. De ce nu a reușit conexiunea? Țineți cont de ordinea regulilor afișate; sunt parcurse secvențial.

Pentru rezolvarea problemei ștergeți regula `iptables` introdusă anterior și **inserați** regula pe stația `host`. Pentru inserare folosiți opțiunea `-I` a comenzii `iptables`. Verificați că acum conexiunea SSH între `red` și `green` va fi realizată.

Pentru a șterge o regulă puteți folosi opțiunea `-D`.

Pentru a insera o regulă folosiți opțiunea `-I` urmată de numele lanțului (`INPUT`, `OUTPUT` sau `FORWARD`), urmată de indexul poziției unde doriți plasată regulă (1, 2, 3, …) și apoi urmată de specificarea regulii.

9. [10p] Ștergere reguli adăugate

Pentru a permite tot traficul către stația `green`, ștergeți pe stația `host` toate regulile `iptables` din lanțul `FORWARD`. Folosiți opțiunea `-F` (*flush*) a comenzii `iptables`. Practic revenim la situația inițială, fără reguli `iptables` pe stația `host`. Folosiți comanda `iptables -L FORWARD -n -v` pentru a valida ștergerea regulilor din lanțul `FORWARD`.

După ștergerea regulilor verificați funcționarea serviciilor telnet, FTP, SSH prin conectare de la stația `red` la stația `green`.

10. [10p] Capturi de trafic

Tcpdump este un utilitar din linia de comanda a Linux-ului, care se ocupa de captura si analiza pachetelor de retea la nivel de interfata. Este deseori folosit pentru trobleshooting sau ca tool de securitate. Este versatil, ofera filtre si poate fi folosit intr-o varietate de cazuri. Fiind un utilitar de linie de comanda, cel mai des se foloseste in sistemele care nu au GUI, pentru a colecta date, care apoi pot fi mutate si vizualizate cu Wireshark[1]

Printre optiunile de tcpdump, avem:

- -i: interfata pe care sa asculte
- -p: portul destinatie: filtrare dupa portul destinatie al pachetelor
- -v: verbosity level
- -w: fisierul in care se salveaza datele

Pentru a vedea grafic ce pachete ajung la masina red, trebuie sa capturati traficul generat catre red intr-un fisier si apoi trebuie copiat fisierul pe masina fizica, pentru a-l analiza cu wireshark.

Pasii care trebuie urmati sunt:

1. Porniti tcpdump pentru interfata `veth-red` de pe `host` cu optiunea de salvare a output-ului intr-un fisier.
2. Generati trafic catre red de pe oricare dintre celelalte masini. Puteti folosi orice fel de trafic (ex. ping / ssh / telnet)
3. Folositi scp pentru a muta fisierul de output pe masina de host, apoi pe `fep.grid.pub.ro` si pe masina locala
4. Deschideti fisierul cu Wireshark.

Ce fel de pachete ati analizat?

[1] https://www.wireshark.org/

11. [BONUS - 10p] Blocare acces din Internet la rețeaua locală

Pentru simularea unei situații reale, vom considera stația `red` ca fiind o stație din rețeaua locală (LAN), iar stația `green` o stație din Internet, conectate între ele prin stația `host`, pe post de gateway.

Dorim să blocăm traficul TCP **inițiat** din Internet către rețeaua locală, adică traficul TCP inițiat **de la** stația `green` **către** stația `red`. Traficul inițiat de stația `red` către stația `green` precum și traficul de răspuns de la stația `green` către stația `red` va fi permis.

Ca exemplu practic, de pe stația `red` vor putea fi inițiate conexiuni SSH către stația `green` folosind comanda

```
ssh green
```

, dar de pe stația `green` **nu** vor putea fi inițiate conexiuni SSH către stația `red` folosind comanda

```
ssh red
```

Configurați `iptables` pe stația `host` pentru a realiza acest lucru.

Folosiți modulul `state` al `iptables`. Accesați pagina de manual `iptables` și căutați după `--state`. Pe distribuțiile mai noi accesați pagina de manual `iptables-extensions`. Pagina de manual este instalată și accesibilă pe stația `host`, **nu** pe stațiile `red`, `green` sau `blue`.

12. [BONUS - 10p] Transfer sincronizat de fișiere folosind "rsync" peste SSH

rsync este un utilitar și protocol care permite transfer incremental. Este foarte potrivit pentru situații de backup, când se transferă doar actualizările sau pentru mentenanța unui mirror, la fel transferându-se de la serverul central doar noile date. Altfel de operații poartă numele de "sincronizare" (*syncing*).

Dorim să facem backup prin rsync al directorului `proiecte/` din directorul home al utilizatorului `ana` de pe stația `host` în contul `bogdan` de pe stația `blue`. Atunci când vom actualiza conținutul directorului `proiecte/`, procesul de backup va transfera doar modificările.

Folosiți `rsync` și protocolul SSH pentru a realiza acest lucru. Se dorește ca autentificarea să se realizeze pe bază de chei, fără parolă.

Navigare

Laboratorul 8

- 1. [10p] Conectare SSH folosind cheie publică
- 2. [10p] Generare cheie publică și autentificare
- 3. [10p] Download și upload de director folosind "scp"
- 4. [10p] Copiere fișiere cu diverse protocoale: durată și consum de resurse
- 5. [10p] Trafic criptat și necriptat
- 6. [10p] Blocare servicii necriptate
- 7. [10p] Blocare SSH
- 8. [10p] Permitere trafic SSH
- 9. [10p] Ștergere reguli adăugate
- 10. [10p] Capturi de trafic
- 11. [BONUS - 10p] Blocare acces din Internet la rețeaua locală
- 12. [BONUS - 10p] Transfer sincronizat de fișiere folosind "rsync" peste SSH

---

1) În cazul unui transfer prin rețea folosind `netcat` nu se face verificare de integritate (*integrity check*). Există riscul (mic, dar există) ca fișierul să nu fie transferat corespunzător. De aceea e bine să verificați integritatea acestuia prin calcularea sumei de control (*checksum*) folosind, de exemplu, utilitarul `sha512sum`.

Laborator

Laboratorul 8

- 1. [10p] Conectare SSH folosind cheie publică
- 2. [10p] Generare cheie publică și autentificare
- 3. [10p] Download și upload de director folosind "scp"
- 4. [10p] Copiere fișiere cu diverse protocoale: durată și consum de resurse
- 5. [10p] Trafic criptat și necriptat
- 6. [10p] Blocare servicii necriptate
- 7. [10p] Blocare SSH
- 8. [10p] Permitere trafic SSH
- 9. [10p] Ștergere reguli adăugate
- 10. [10p] Capturi de trafic
- 11. [BONUS - 10p] Blocare acces din Internet la rețeaua locală
- 12. [BONUS - 10p] Transfer sincronizat de fișiere folosind "rsync" peste SSH

Curs

Informații generale RL

- Regulament
  - Notare Seria CA
  - Notare Seriile CB/CC
- Resurse
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

Table of Contents

- Laborator 8. Securizarea unui server
  - Cunoștințe și abilități ce vor fi dobândite
  - Cheat sheet
  - Pregătire infrastructură de laborator
  - Topologie
  - Navigare
  - Exerciții
    - 1. [10p] Conectare SSH folosind cheie publică
    - 2. [10p] Generare cheie publică și autentificare
    - 3. [10p] Download și upload de director folosind "scp"
    - 4. [10p] Copiere fișiere cu diverse protocoale: durată și consum de resurse
    - 5. [10p] Trafic criptat și necriptat
    - 6. [10p] Blocare servicii necriptate
    - 7. [10p] Blocare SSH
    - 8. [10p] Permitere trafic SSH
    - 9. [10p] Ștergere reguli adăugate
    - 10. [10p] Capturi de trafic
    - 11. [BONUS - 10p] Blocare acces din Internet la rețeaua locală
    - 12. [BONUS - 10p] Transfer sincronizat de fișiere folosind "rsync" peste SSH
  - Navigare

BY-SA CHIMERIC DE W3C CSS DOKUWIKI GET FIREFOX RSS XML FEED W3C XHTML 1.0

Laborator 9. Translatarea de adrese

Cunoștințe și abilități ce vor fi dobândite

- Cunoștințe generale legate de translatarea de adrese
- Translatarea de adrese pe Linux
- Translatarea directă (SNAT, MASQUERADE) și port forwarding (DNAT)
- Tunelare SSH

Cheat sheet

- Cheat Sheet

Pregătire infrastructură de laborator

- **Reminder**: avem nevoie de o masina virtuală a laboratorului. Vă rugăm urmăriți pagina aceasta pentru instrucțiuni, apoi reveniți.
- Schimbati utilizatorul curent ca `root` folosind comanda

```
student@host:~$ sudo su
```

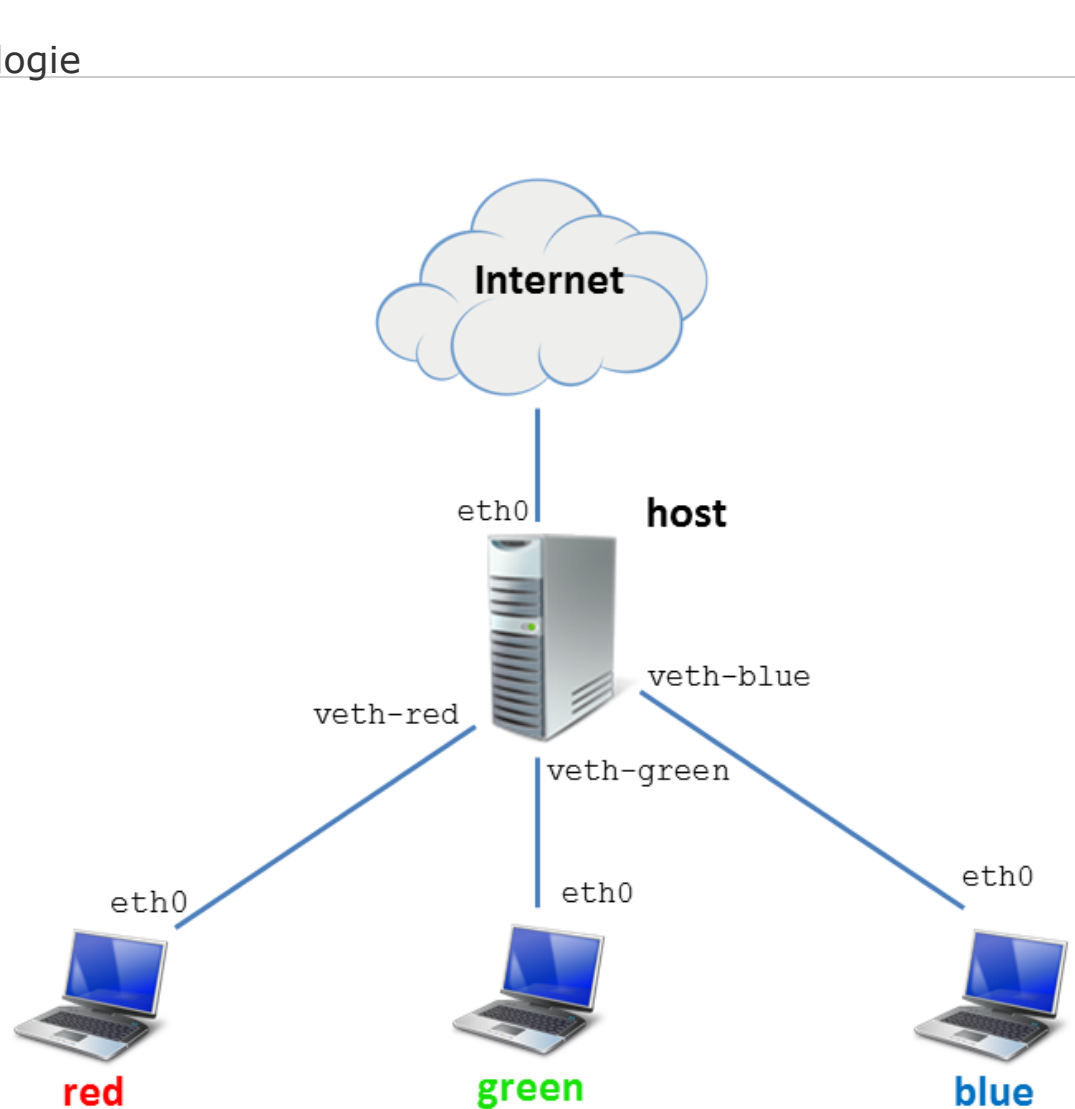
- Pentru a pregăti configurația de laborator, pe mașina virtuală (stația `host`) folosiți comenzile următoare din contul utilizatorului `root` de pe stația `host` (puteți da copy/paste la comenzi în terminal):

```
root@host:~# update_lab --force
root@host:~# start_lab lab9
```

- Deschideți trei noi tab-uri în terminal (folosiți combinația de taste `Ctrl+Shift+t`), și conectați-vă, din nou, la mașina virtuală folosind comanda `ssh` de mai sus.
  - De pe cele trei noi tab-uri, conectați-vă la cele trei containere (`red`, `green` și `blue`).
  - Pentru o conectare mai usoara puteti folosi aliasul `go` (ex. `go red`)

În mod implicit folosiți contul `root` pentru conectare pe toate stațiile. Aveți nevoie de drepturi privilegiate pentru configurare. Folosiți contul `student` doar unde vi se cere explicit.

Topologie



Navigare

Laboratorul 9

- 01. [10p] Configurare translatare de adrese (MASQUERADE)
- 02. [10p] Format de pachete la translatare
- 03. [10p] Format de pachete TCP la translatare
- 04. [10p] Configurare incorectă a translatării
- 05. [10p] Port forwarding
- 06. [10p] Extindere port forwarding
- 07. [10p] Format de pachete la port forwarding
- 08. [10p] Port forwarding pentru telnet
- 09. [10p] Configurare persistentă rutare și NAT
- 10. [10p] Tunel SSH invers
- 11. [BONUS - 10p] Translatare selectivă de adrese

Exerciții

Pentru a primi întregul punctaj va trebui ca la finalul laboratorului să ștergeți mașina virtuală pornită și să îi arătați asistentului listarea instanțelor din OpenStack.

În cadrul exercițiilor din laboratoarele de Linux vom folosi topologia de mai sus.

01. [10p] Configurare translatare de adrese (MASQUERADE)

Epuizarea adreselor IPv4 în cadrul rețelelor curente a condus la folosirea de adrese IP din clase private (de ex. `192.168.0.0/24`). Pe lângă comunicația dintre stațiile unei rețele, dorim și accesul la Internet al acestora. De aceea s-a introdus conceptul de translatare de adrese (NAT: *Network Address Translation*) prin care mai multe stații dispun de acces la Internet folosind aceeași adresă IP rutabilă: adresa gateway-ului. Activarea translatării de adrese (NAT) pe gateway conduce la înlocuirea perechii <adresă IP sursă, port sursă> (aparținând stației) cu perechea <adresa IP gateway, port disponibil>.

Configurarea NAT pe Linux se realizează tot prin intermediul comenzii `iptables`, la fel ca în cazul configurării firewall-ului. Dacă pentru configurarea firewall-ului foloseam tabela `filter` (tabela implicită a `iptables`), pentru configurarea translatării de adrese vom folosi tabela `nat`.

Astfel, pentru a activa NAT pe un server Linux executăm comanda

```
root@host:~# iptables -t nat -A POSTROUTING -j MASQUERADE
```

În comanda de mai sus:

- `-t` specifică tabela pe care se aplică regula, în cazul nostru tabela `nat`.
- `-A` înseamnă adăugarea unei reguli la sfârșitul listei de reguli.
- `POSTROUTING` se referă la momentul când va fi realizat procesul de translatare de adrese: după rutare.
  - În nomenclatura `iptables` acesta se numește și **lanț (chain)**.
  - Exemple de alte lanțuri: `INPUT`, `OUTPUT`, `FORWARD`, `PREROUTING`.
- `-j` este acțiunea ce va fi luată, iar în acest caz este `MASQUERADE` (acțiune simplă de translatare de adrese).

Pentru a verifica și valida regula, afișăm intrările din lanțul `POSTROUTING` din tabela `nat` folosind comanda

```
root@host:~# iptables -t nat -L POSTROUTING -n -v
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  all  --  *      *       0.0.0.0/0            0.0.0.0/0
```

Vrem să verificăm configurarea corectă a NAT. Pentru acesta vom trimite de pe stația `red` un pachet către `8.8.8.8`. Pachetul va trece prin gateway (adică stația `host`) și va fi translatat. Pe stația `red` rulăm comanda

```
root@red:~# ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
From 192.168.1.2 icmp_seq=1 Destination Host Unreachable
From 192.168.1.2 icmp_seq=2 Destination Host Unreachable

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 0 received, +2 errors, 100% packet loss, time 999ms
```

Observăm că nu există conectivitate de la stația `red` către adresa IP `8.8.8.8`. Consultăm întreaga tabelă `nat`:

```
root@host:~# iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 2 packets, 168 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  all  --  *      *       0.0.0.0/0            0.0.0.0/0
```

Observăm că pachetele ajung în lanțul `PREROUTING` (înainte de rutare), dar nu ajung în lanțul `POSTROUTING` (după rutare). Ne gândim că este posibil să fie o problemă cu rutarea pe gateway. Verificăm dacă rutarea este activată:

```
root@host:~# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
```

Într-adevăr, rutarea nu este activată. Pentru a activa rutarea pe stația `host` rulăm comanda

```
root@host:~# sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

Intrăm din nou pe stația `red` și folosim `ping` pentru a testa conectivitatea la adresa IP `8.8.8.8`:

```
root@red:~# ping -c 2 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_req=1 ttl=61 time=92.9 ms
64 bytes from 8.8.8.8: icmp_req=2 ttl=61 time=81.2 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 81.272/87.094/92.917/5.829 ms
```

Acum există conectivitate, lucru certificat și de prezența unor pachete în lista prelucrată pe lanțul `POSTROUTING`:

```
root@host:~# iptables -t nat -L POSTROUTING -n -v
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    2   168 MASQUERADE  all  --  *      *       0.0.0.0/0            0.0.0.0/0
```

02. [10p] Format de pachete la translatare

Ne propunem să analizăm antetul IP al pachetelor ce sunt generate de stațiile `red`, `green` și `blue` și au ca destinație o rețea din Internet. Pentru acest lucru vom folosi utilitarul de captură `tcpdump`.

Pe stația `red` pornim comanda `ping` către `8.8.8.8.`:

```
root@red:~# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_req=1 ttl=127 time=42.0 ms
[...]
```

Atunci când rulăm comanda `tcpdump` secvența de urmat este:

1. Se rulează comanda `tcpdump` cu opțiunile aferente într-un terminal, activând astfel captura de pachete. Utilitarul `tcpdump` așteaptă acum transmiterea de pachete pe interfețele pe care ascultă.
2. Într-un alt terminal se rulează o comandă specifică unui client de rețea care generează trafic.
3. Se revine în terminalul în care rulează comanda `tcpdump` și se urmăresc pachetele capturate.
4. Când nu mai este nevoie de utilitarul `tcpdump` se întrerupe captura de pachete folosind combinația de taste `Ctrl+c`.

Pentru a urmări traficul, pe stația `host` rulăm comanda

```
root@host:~# tcpdump -i eth0 ip dst host 8.8.8.8
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
12:59:20.976707 IP host.local > google-public-dns-a.google.com: ICMP echo request, id 625, seq
12:59:21.977708 IP host.local > google-public-dns-a.google.com: ICMP echo request, id 625, seq
```

(click pentru expandare) Detalii rulare masina virtuala VMware local

Observăm că adresa IP sursă este `host.local` chiar dacă stația `red` este cea care execută comanda `ping` și generează pachetele de tip `ICMP echo request`.

Pentru a vedea pachetele așa cum sunt generate inițial, rulăm comandă `tcpdump` pe interfața `veth-red` în loc de `eth0`:

```
root@host:~# tcpdump -i veth-red ip dst host 8.8.8.8
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on veth-red, link-type EN10MB (Ethernet), capture size 65535 bytes
13:01:12.557692 IP red > google-public-dns-a.google.com: ICMP echo request, id 626, seq 6, lengt
13:01:13.559726 IP red > google-public-dns-a.google.com: ICMP echo request, id 626, seq 7, lengt
```

Observăm că pe interfața `veth-red` adresa IP sursă este adresa stației `red`, adică așa cum este generat de la început pachetul. Motivul este reprezentat de faptul că pachetele ce intră pe intefața `veth-red` sunt capturate înainte de rutare, iar procesul NAT este aplicat **după rutare** (vezi 01. [10p] Configurare translatare de adrese (MASQUERADE)).

Pentru a vedea modul în care se translatează traficul capturăm traficul pe toate interfețele (cele de interes sunt `veth-red` și `eth0`)

```
root@host:~# tcpdump -i any ip dst host 8.8.8.8
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on any, link-type LINUX_SLL (Linux cooked), capture size 65535 bytes
10:23:07.632412 IP red > google-public-dns-a.google.com: ICMP echo request, id 707, seq 237, len
10:23:07.632430 IP host.local > google-public-dns-a.google.com: ICMP echo request, id 707, seq
10:23:08.633936 IP red > google-public-dns-a.google.com: ICMP echo request, id 707, seq 238, len
10:23:08.633954 IP host.local > google-public-dns-a.google.com: ICMP echo request, id 707, seq
```

În lista de mai sus observăm atât pachetele care sunt captuarate pe interfața `veth-red` (generate de stația `red`) cât și cele capturate pe interfața `eth0` (translatate de stația `host`).

Capturați și pachetele de reply, care au ca **sursă** adresa `8.8.8.8`. Folosiți șirul de argumente `ip src host 8.8.8.8` pentru `tcpdump`.

Repetați testele de mai sus pentru stația `green`.

Întrucât latența DNS poate fi semnificativă, recomandăm folosirea opțiunii `-n` a utilitarului `tcpdump`, opțiune care dezactivează rezolvarea DNS. În exemplele de mai sus, nu am folosit opțiunea `-n` a utilitarului `tcpdump` pentru a identifica mai ușor numele stațiilor implicate în conversație.

03. [10p] Format de pachete TCP la translatare

Dorim să urmărim conținutul pachetelor TCP înainte și după translatarea de adrese. În plus față de exercițiul de mai sus, vrem să urmărim și porturile. Pentru aceasta vom genera pachete TCP (HTTP) folosind `wget` de pe stația `red` și vom captura pachetele folosind `tcpdump` pe stația `host`.

Pe stația `host` capturați pe toate interfețele pachetele HTTP care au ca adresă destinație numele `cs.pub.ro`.

Va trebui să folosiți argumentele `ip dst host cs.pub.ro` și `tcp dst port 80` la `tcpdump`. Legați argumentele prin șirul `and`. Urmăriți și exemplele de aici.

Folosiți opțiunea `-n` a `tcpdump` ca să afișați în format rezultatele în format numeric (adresă IP și port). Vă va fi mai ușor să urmăriți pachetele și translatarea de adrese.

Pe stația `red` folosiți `wget` pentru a obține pagina de la `cs.pub.ro`.

Urmăriți translatarea adresei IP sursă și a portului sursă în cadrul capturii de pachete realizată cu `tcpdump` pe `host`.

O să existe o latență (2-3 secunde) la afișarea pachetelor capturate cu `tcpdump` datorată rezolvării DNS. Puteți folosi opțiunea `-n` a `tcpdump` pentru a dezactivarea rezolvarea DNS și pentru a elimina latența.

Observați în cadrul capturii că portul sursă, ales de stația `red` este același cu cel folosit în urma translatării de stația `host`. În general implementările de NAT vor păstra portul în urma translatării. În cazul rar în care acel port este ocupat pe stația `host` (posibil datorită unei alte translatării) se va aloca un alt port.

Pe stația `host` capturați pe toate interfețele pachetele HTTP care au ca adresă numele `cs.pub.ro`; nu contează dacă este sursă sau destinație, capturăm și pachetele de interogare și cele de răspuns; la fel, portul poate fi sursă sau destinație; folosiți `ip host` în loc de `ip dst host` și `tcp port` în loc de `tcp dst port`. Pe stația `red` folosiți, din nou, `wget` pentru a obține pagina de la `cs.pub.ro`. Urmăriți translatarea adresei IP sursă și a portului sursă pentru pachetele de interogare și translatarea adresei IP destinație și a portului destinație pentru pachetele de răspuns.

04. [10p] Configurare incorectă a translatării

Comanda folosită mai sus pentru translatare, la 01. [10p] Configurare translatare de adrese (MASQUERADE), are neajunsuri pe care le vom identifica mai jos.

Pe stația `green` capturăm traficul pe interfața `green-eth0` folosind comanda `tcpdump`:

```
root@green:~# tcpdump -i green-eth0
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on green-eth0, link-type EN10MB (Ethernet), capture size 65535 bytes
```

Pe stația `red` executăm comanda `ping` către stația `green`:

```
root@red:~# ping -c 2 green
PING green (192.168.2.2) 56(84) bytes of data.
64 bytes from green (192.168.2.2): icmp_req=1 ttl=63 time=0.155 ms
64 bytes from green (192.168.2.2): icmp_req=2 ttl=63 time=0.086 ms
[...]
```

În acest moment, pe stația `green` avem captura pachetelor în output-ul comenzii `tcpdump`:

```
11:18:54.535064 IP host > green: ICMP echo request, id 633, seq 17, length 64
11:18:54.535092 IP green > host: ICMP echo reply, id 633, seq 17, length 64
```

Observăm că adresa sursă a pachetelor de tip `ICMP echo request` este stația `host`, nu stația `red`. Acest lucru se întâmplă din cauză că atunci când am activat NAT nu am specificat pentru ce tip de trafic să aplice politica de translatare. Astfel stația `host` aplică politica de NAT pentru tot traficul care o tranzitează. Dorim să aplicăm politica doar pentru pachetele ce **ies** în Internet, adică cele ale căror **interfață de ieșire** este interfața `eth0`. Vom șterge vechea regulă de NAT și vom adăuga o regulă corectă.

Pentru ștergerea vechii reguli de NAT rulăm pe stația `host` comanda

```
root@host:~# iptables -t nat -D POSTROUTING -j MASQUERADE
```

Verificăm faptul că nu mai există comanda

```
root@host:~# iptables -t nat -L POSTROUTING -n -v
Chain POSTROUTING (policy ACCEPT 1 packets, 328 bytes)
 pkts bytes target     prot opt in     out     source               destination
```

Adăugăm din nou regula de NAT, dar de data aceasta vom specifica să aplice politica de NAT pachetelor ce ies prin interfața `eth0` a stației `host`:

```
root@host:~# iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

Verificați că regula a fost inserată:

```
root@host:~# iptables -t nat -L POSTROUTING -n -v
Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  all  --  *      eth0    0.0.0.0/0            0.0.0.0/0
```

Observăm prezența interfeței `eth0` în coloana `out`. Înainte apărea caracterul `*`, adică orice interfață.

Repetați testul de la începutul exercițiului și arătați că pachetele trimise de la `red` către `green` au adresele IP sursă/destinație nemodificate (nu mai fac trecerea prin NAT), în vreme ce pachetele ce ies în Internet sunt translatate.

05. [10p] Port forwarding

La exercițiile de până acum am folosit NAT pentru a permite stațiilor cu adrese private dintr-o rețea locală să acceseze Internetul. NAT poate fi folosit și pentru a permite unei stații din rețeaua locală să fie accesată din Internet, adică să se inițieze conexiuni din Internet. Acest proces poartă numele de *port forwarding*.

Dorim să putem accesa prin SSH, din Internet, stația `red`. Acest lucru nu este posibil în mod implicit întrucât stația `red` are adresă IP privată. Soluția este "deschiderea unui port" pe gateway (adică stația `host`) și redirectarea acestui port (*port forwarding*) către portul aferent serviciului SSH (portul TCP `22`) de pe stația `red`.

Vom aplica pe stația `host` o regulă prin care redirectăm traficul ce vine către `host` pe portul `10022` către portul `22` (SSH) al stației `red` (adresa IP `192.168.1.2`):

```
root@host:~# iptables -t nat -A PREROUTING -p tcp --dport 10022 -j DNAT --to-destination 192.168
```

Verificăm aplicarea regulii prin consultarea lanțului `PREROUTING` din tabela NAT:

```
root@host:~# iptables -t nat -L PREROUTING -n -v
Chain PREROUTING (policy ACCEPT 1 packets, 474 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DNAT       tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt
```

Pentru a verifica rezultatul de mai sus, de pe `fep.grid.pub.ro` deschidem un nou terminal și ne conectăm prin SSH folosind portul `10022` la stația `host`:

```
mihai.carabas@fep:~$ ssh -l student $ADRESA_IP_MV -p 10022
root@192.168.138.129's password:
[...]
root@red:~#
```

unde `$ADRESA_IP_MV` este adresa IP a masinii virtuale obținută din dashboard-ul OpenStack.

Pentru rulare masina virtuala VMware local

Observăm că în urma autentificării ne găsim pe stația `red`. *Port forwarding* a funcționat.

Folosim comanda de conectare SSH la stația `host` de pe stația `green`:

```
root@green:~# ssh -l student host -p 10022
[...]
root@red:~#
```

Observăm că și de pe stația `green` am accesat stația `red` prin *port forwarding*. Dorim să limităm *port forwarding* doar pentru conexiuni de la stațiile din Internet. Pentru aceasta trebuie să actualizăm regula de *port forwarding*.

Ștergeți regula de *port forwarding* și adăugați o regulă nouă care să permită **doar** stațiilor din Internet să acceseze prin *port forwarding* stația `red`.

Aplicați regula doar pentru pachetele ce **sosesc** pe interfața `eth0`. Adică `eth0` este interfața de **intrare**. Folosiți opțiunea `-i` a `iptables` pentru precizarea interfeței de **intrare**.

Realizați apoi din nou conectare SSH pe portul `10022` al stației `host` de pe `fep.grid.pub.ro` și de pe stația `green`. Dacă ați configurat corect, nu va merge conexiunea SSH de pe stația `green` dar va merge în continuare de pe `fep.grid.pub.ro`.

06. [10p] Extindere port forwarding

Dorim să accesăm din Internet/exterior stațiile `green` și `blue` prin SSH folosindu-ne de stația `host`. Vom folosi:

- portul `20022` de pe stația `host` pentru a face *port forwarding* pe portul 22 al stației `green`;
- portul `30022` de pe stația `host` pentru a face *port forwarding* pe portul 22 al stației `blue`.

Pe modelul exercițiului anterior, realizați configurarea necesară pentru această extindere de port forwarding. Verificați prin conectare de pe `fep.grid.pub.ro` (echivalentul Internet-ului/exteriorului) pe porturile respectiv `20022` și `30022` de pe stația `host`.

Pentru a afla adresele IP ale stațiilor `green` și `blue` rulați comanda de afișare a configurării interfeței `eth0` pe fiecare stație. Verificați, pentru siguranța, faptul că există conectivitate între ele și stația `host`.

07. [10p] Format de pachete la port forwarding

La exercițiul 03. [10p] Format de pachete TCP la translatare am observat cum se modifică adresa IP sursă și portul sursă în cazul translatării de adrese. La port forwarding vom observa cum se modifică adresa IP destinație (de aici și numele DNAT: `Destination NAT`).

Vom captura traficul SSH inițiat din exterior către stația `red`, prin intermediul portului `10022` al stației `host`. Este vorba de traficul înainte de *port forwarding*. Pentru aceasta, pe stația `host` vom folosi comanda

```
root@host:~# tcpdump -n -i eth0 tcp dst port 10022 -w /home/student/portfwd_eth0_output.pcap
```

Pe un alt terminal, tot pe stația `host` capturăm traficul **după** *port forwarding*, pe interfața `veth-red` către portul SSH (22) al stației red. Pe stația `host` folosim comanda

```
root@host:~# tcpdump -n -i veth-red tcp dst port 22 -w /home/student/portfwd_veth-red_output.pca
```

Pentru a genera trafic, de pe `fep.grid.pub.ro` realizăm o conexiune SSH la stația `host` pe portul `10022`, conexiune ce va fi redirectată la portul `22` al stației `red`:

```
mihai.carabas@fep:~$ ssh -l student $ADRESA_IP_VM -p 10022
root@host.local's password:
[...]
root@red:~#
```

Pentru a vizualiza capturile salvate în cele două fișiere copiați-le la voi local (pe calculatorul vostru) folosind `scp` și deschideți-le cu utilitarul `wireshark`.

(click pentru expandare) Copiere fișiere din VM pe local

Exemplu instalare și pornire `Wireshark` (pe sisteme Debian):

```
student@mjolnir:~$ apt-get install wireshark
student@mjolnir:~$ wireshark portfwd_eth0_output.pcap &
student@mjolnir:~$ wireshark portfwd_veth-red_output.pcap &
```

În capturile realizate de comenzile de mai sus, observăm cum se translatează adresa IP și portul destinație din perechea <$ADRESA_IP_VM, 10022> în perechea <192.168.1.2, 22>.

08. [10p] Port forwarding pentru telnet

În cadrul exercițiilor anterioare am activat *port forwarding* pentru serviciul SSH. Dorim ca stațiile `red`, `green` și `blue` să fie accesibile și prin `telnet` din Internet astfel:

- `red` să poată fi accesat folosind portul 10023
- `green` să poată fi accesat folosind portul 20023
- `blue` să poată fi accesat folosind portul 30023

Realizați configurațiile necesare pentru a permite *port forwarding* pentru `telnet` așa cum este descris mai sus.

Serviciul `telnet` ascultă în mod implicit conexiuni pe portul 23. Puteți observa acest lucru cu ajutorul uneia dintre comenzile de mai jos:

```
user@host:~$ grep -w 'telnet' /etc/services
telnet          23/tcp
tfido           60177/tcp                       # fidonet EMSI over telnet

user@host:~$ getent services telnet
telnet                23/tcp
```

Exercițiul este academic. Nu este indicată folosirea `telnet` cu atât mai puțin *port forwarding* peste `telnet` din rațiuni de securitate.

Testați de pe `fep.grid.pub.ro` folosind comanda `telnet`:

```
mihai.carabas@fep:~# telnet <adresa-ip-vm> <port-redirectat>
```

unde `<adresa-ip-vm>` este adresa IP a mașinii virtuale vizibilă de pe fep iar `<port-redirectat>` este portul care face redirectarea pe stația `host` (`10023` sau `20023` sau `30023`).

Există o latență de circa 10-15 secunde din momentul conectării până la apariția promptului `telnet`. Așteptați să apară promptul și apoi autentificați-vă cu username `student` și parolă `student` pentru a valida conectivitatea.

09. [10p] Configurare persistentă rutare și NAT

Configurările realizate până în acest moment sunt temporare. La repornirea stației `host` regulile de NAT se pierd. Dorim să configurăm în mod persistent rutarea și regulile NAT.

Pentru a configura în mod persistent rutarea pe stația `host`, edităm fișierul `/etc/sysctl.conf` și decomentăm linia

```
net.ipv4.ip_forward=1
```

Astfel la fiecarea pornire a sistemului, rutarea va fi activată.

Rutarea nu se va activa în mod automat după ce decomentați linia din fișier. Trebuie să resetați sistemul sau să rulați comanda

```
sysctl -p
```

pentru a aplica modificările din fișierul de configurare `/etc/sysctl.conf`.

Pentru a salva toate regulile `iptables` introduse în sistem se poate folosi comanda `iptables-save`. În mod implicit aceste reguli sunt afișate pe consolă. Noi le vom salva în `/etc/iptables-rules` folosind comanda:

```
root@host:~# iptables-save > /etc/iptables-rules
```

Regulile pot fi restaurate folosind comanda `iptables-restore < /etc/iptables-rules`. Așadar trebuie să configurăm sistemul să execute această comanda la pornire. Modul cel mai uzual pentru a face acest lucru este adăugarea acestei comenzi în fișierul de configurare al interfeței:

```
root@host:~# cat /etc/network/interfaces
[...]
# The primary network interface
auto eth0
iface eth0 inet dhcp
        up iptables-restore < /etc/iptables-rules
```

După ce ați făcut toate configurările, reporniți stația `host`:

```
Dashboard -> Drop Down in dreptul masinii virtuale -> Soft Reboot Instance
```

În urma repornirii stației `host`, va trebui sa rerulam scripturile de pregatire a infrastructurii pentru a recrea containerele. Vom vedea aplicată rutarea și regulile de NAT: (in cazul in care rutarea nu este activata, inseamna ca a fost suprascrisa de catre scripturile de pregatire a infrastructurii si va trebui s-o activati din nou)

```
root@host:~# sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
root@host:~# iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 30 packets, 5694 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DNAT       tcp  --  eth0   *       0.0.0.0/0            0.0.0.0/0            tcp dpt
    0     0 DNAT       tcp  --  eth0   *       0.0.0.0/0            0.0.0.0/0            tcp dpt
    0     0 DNAT       tcp  --  eth0   *       0.0.0.0/0            0.0.0.0/0            tcp dpt
    0     0 DNAT       tcp  --  eth0   *       0.0.0.0/0            0.0.0.0/0            tcp dpt
    0     0 DNAT       tcp  --  eth0   *       0.0.0.0/0            0.0.0.0/0            tcp dpt
    0     0 DNAT       tcp  --  eth0   *       0.0.0.0/0            0.0.0.0/0            tcp dpt

Chain INPUT (policy ACCEPT 2 packets, 534 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 16 packets, 958 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain POSTROUTING (policy ACCEPT 15 packets, 918 bytes)
 pkts bytes target     prot opt in     out     source               destination
    1    40 MASQUERADE  all  --  *      eth0    0.0.0.0/0            0.0.0.0/0
```

10. [10p] Tunel SSH invers

Sunt situații în care dorim ca o stație să fie accesibilă din exterior, dar este dificil să obținem un port în gateway (pentru *port forwarding*): fie din rațiuni administrative, fie din motive de securitate. Pentru a permite totuși o conexiune din exterior, se poate crea un tunel SSH invers. Tunel SSH invers înseamnă următorii pași:

- se creează o conexiune SSH de la o stație cu adresă IP privată la un server extern cu adresă IP publică,
- se deschide un port pe serverul cu adresă IP publică
- traficul către acel port este tunelat prin conexiunea SSH către stația cu adresă IP privată

Dorim să permitem conectarea la stația `red` din Internet pe SSH. Vom crea o conexiune SSH "directă" de la stația `red` la contul vostru de pe `fep.grid.pub.ro`. Prin această conexiune veți tunela invers traficul SSH către `red` permițând conexiunea din exterior.

Pentru început, de pe stația `red` creăm o conexiune SSH directă către contul de pe `fep.grid.pub.ro` (identic cu cel de pe `cs.curs.pub.ro`):

```
root@red:~# ssh -l $USERNAME fep.grid.pub.ro -R 100XY:localhost:22
[...]
$USERNAME@fep.grid.pub.ro's password:
Last login: Mon Sep  9 09:55:37 2013 from 141.85.225.214
[$USERNAME@fep-62-2 ~]$ netstat -tln | grep 100XY
tcp        0      0 127.0.0.1:100XY             0.0.0.0:*                   LISTEN
tcp        0      0 ::1:100XY                   :::*                        LISTEN
```

unde `$USERNAME` este numele contului de pe `fep.grid.pub.ro/cs.curs.pub.ro`, iar `XY` este un identificator numeric unic al stației din laborator (dacă nu aveți pe monitorul stației din laborator un identificator discutați cu asistentul). Introduceți parola corespunzătoare contului vostru de pe `fep.grid.pub.ro/cs.curs.pub.ro`.

Observăm că se deschide portul `100XY` pe `fep.grid.pub.ro`. Acest port va fi folosit pentru tunelul invers care ne va duce pe stația `red`.

Argumentul `-R 100XY:localhost:22` transmis comenzii `ssh` are următoarea semnificație:

- `100XY` - portul de pe server pe care se vor asculta conexiuni care vor fi tunelate invers;
- `localhost` - stația către care vor fi trimise pachetele primite prin tunel (chiar stația locală);
- `22` - portul către care vor fi trimise pachetele primite prin tunel (adică portul SSH local).

După ce alți colegi rulează comenzile similare vă veți putea conecta pe stația `red` a acestora folosind portul deschis de ei; discutați cu alți colegi ca să vă spună ce port au deschis. Pentru aceasta, veți folosi stația fizică și vă veți conecta conecta pe contul de pe `fep.grid.pub.ro`.

Pe `fep.grid.pub.ro` rulează un load balancer pentru SSH. Din acest motiv este posibil să intrați pe "celălalt" sistem. Verificați că numele de host din prompt corespunde cu cel pe care l-ați folosit inițial la crearea tunelului: adică ambele să fie `fep-62-2` sau `fep-62-1`. Pentru siguranță folosim și comanda `netstat`:

```
[$USERNAME_COLEG@fep-62-2 ~]$ netstat -tln | grep 100ZT
tcp        0      0 127.0.0.1:100ZT             0.0.0.0:*                   LISTEN
tcp        0      0 ::1:100ZT                   :::*                        LISTEN
```

În output-ul comenzii de mai sus `100ZT` este portul deschis de coleg.

După conectarea colegului la `fep`, realizăm o conexiune prin tunelul SSH pe portul `100ZT` deschis de coleg:

```
[$USERNAME_COLEG@fep-62-2 ~]$ ssh -l student localhost -p 100ZT
Warning: Permanently added '[localhost]:100ZT' (RSA) to the list of known hosts.
root@localhost's password:
[...]
root@red:~#
```

În cadrul conexiunii ni se cere parola utilizatorului `root` de pe stația `red` (adică parola `student`).

În final ați accesat stația `red` a colegului, o stația cu adresă IP privată inaccesibilă din exterior. Pentru aceasta ați folosit o altă stație (care ar putea fi din altă rețea) pentru a se conecta la `fep.grid.pub.ro` și de acolo la `red` prin intermediul tunelului invers creat inițial.

Creați un fișier pe stația `red` proaspăt accesată, iar colegul poate verifica faptul că, într-adevăr, ați accesat container-ul său prin tunelul SSH invers pe care l-a creat anterior.

11. [BONUS - 10p] Translatare selectivă de adrese

Pentru situația în care adresa IP exterioară a gateway-ului este adresă statică (nu dinamică) se recomandă folosirea acțiunii `SNAT` în loc de `MASQUERADE` la translatarea de adrese. Pe lângă aceasta, SNAT are avantajul precizării unui spațiu de adrese care să fie noile adrese (cele substituite) și a porturilor.

Informații despre folosirea `SNAT` și diferența dintre `SNAT` și `MASQUERADE` găsiți aici.

Ștergeți regula anterioară de tip `MASQUERADE` și folosiți reguli de tip `SNAT` pentru a face următoarele translatări:

- conexiunile TCP către Internet de la stația `red` să iasă prin porturi din spațiul `45000-50000`;
- conexiunile TCP către Internet de la stația `green` să iasă prin porturi din spațiul `50000-55000`;
- conexiunile TCP către Internet de la stația `blue` să iasă prin porturi din spațiul `55000-60000`;
- toate celelalte conexiuni să fie realizate în mod obișnuit, având ca adresă sursă adresa IP de exterior a stației `host` (această regulă trebuie să fie ultima).

Verificați translatarea corectă inițiind conexiuni HTTP (folosind `wget`) către `cs.pub.ro` de pe fiecare dintre cele trei stații de tip container (`red`, `green` și `blue`) și folosind `tcpdump` pe stația `host` pentru a captura traficul aferent.

Navigare

Laboratorul 9

- 01. [10p] Configurare translatare de adrese (MASQUERADE)
- 02. [10p] Format de pachete la translatare
- 03. [10p] Format de pachete TCP la translatare
- 04. [10p] Configurare incorectă a translatării
- 05. [10p] Port forwarding
- 06. [10p] Extindere port forwarding
- 07. [10p] Format de pachete la port forwarding
- 08. [10p] Port forwarding pentru telnet
- 09. [10p] Configurare persistentă rutare și NAT
- 10. [10p] Tunel SSH invers
- 11. [BONUS - 10p] Translatare selectivă de adrese

Laborator
Laboratorul 9
- 01. [10p] Configurare translatare de adrese (MASQUERADE)
- 02. [10p] Format de pachete la translatare
- 03. [10p] Format de pachete TCP la translatare
- 04. [10p] Configurare incorectă a translatării
- 05. [10p] Port forwarding
- 06. [10p] Extindere port forwarding
- 07. [10p] Format de pachete la port forwarding
- 08. [10p] Port forwarding pentru telnet
- 09. [10p] Configurare persistentă rutare și NAT
- 10. [10p] Tunel SSH invers
- 11. [BONUS - 10p] Translatare selectivă de adrese

Curs
Informații generale RL
- Regulament
  - Notare Seria CA
  - Notare Seriile CB/CC
- Resurse
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri
- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare
- Laborator 1. Introducere în Rețele de Calculatoare
- Laborator 2. Comutarea pachetelor
- Laborator 3. Adresarea IP
- Laborator 4. VLAN (Virtual LAN)
- Laborator 5. Rutare
- Laborator 6. Virtualizare în Docker
- Laborator 7. Adresare IP și rutare în Linux
- Laborator 8. Securizarea unui server
- Laborator 9. Translatarea de adrese
- Laborator 10. Clienți de rețea
- Laborator 11. Securitatea Rețelelor Locale
- Laborator 12. Amazon Web Services
- Laborator extra - Virtualizare în Networking
- Laborator Recapitulare

Teme
- Tema1
- Tema2

Laborator 10. Clienți de rețea

Cunoștințe și abilități ce vor fi dobândite

- Identificarea porturilor deschise pe un sistem
- Folosirea clienților de rețea în linia de comandă în Linux (`netcat`, `wget`, `curl`, `mail`)
- Accesarea serviciilor de web, transfer de fișiere (FTP) și e-mail

Cheat sheet

- Cheat Sheet

Pregătire infrastructură de laborator

- **Reminder**: avem nevoie de o masina virtuală a laboratorului. Vă rugăm urmăriți pagina aceasta pentru instrucțiuni, apoi reveniți.
- Schimbati utilizatorul curent ca `root` folosind comanda

```
student@host:~$ sudo su
```

- Pentru a pregăti configurația de laborator, pe mașina virtuală (stația `host`) folosiți comenzile următoare din contul utilizatorului `root` de pe stația `host` (puteți da copy/paste la comenzi în terminal):

```
root@host:~# update_lab --force
root@host:~# start_lab lab10
```

- Deschideți trei noi tab-uri în terminal (folosiți combinația de taste `Ctrl+Shift+T`), și conectați-vă, din nou, la mașina virtuală folosind comanda `ssh` de mai sus.
- De pe cele trei noi tab-uri, conectați-vă la cele trei containere (`red`, `green` și `blue`).
- Pentru o conectare mai usoara puteti folosi aliasul `go` (ex. `go red`)

În mod implicit folosiți contul `root` pentru conectare pe toate stațiile. Aveți nevoie de drepturi privilegiate pentru configurare. Folosiți contul `student` doar unde vi se cere explicit.

Navigare

Laboratorul 10

- 01. [15p] Observare porturi deschise pe o stație
- 02. [10p] Port forwarding pentru accesare serviciu web
- 03. [10p] Accesare serviciu web folosind wget
- 04. [15p] Descărcare recursivă de resurse web
- 05. [5p] Accesare URL conținând caractere speciale
- 06. [5p] Accesare de pagini web folosind curl
- 07. [5p] Download FTP folosind curl (fără autentificare)
- 08. [10p] Download FTP folosind curl (cu autentificare)
- 09. [10p] Trimitere e-mail din linia de comandă
- 10. [15p] Trimitere e-mail cu adresă personalizată din linia de comandă
- 11. [BONUS - 10p] Upload prin FTP
- 12. [BONUS - 10p] Trimitere e-mail cu atașament din linia de comandă

Exerciții

În cadrul exercițiilor din laboratoarele de Linux vom folosi topologia de mai sus.

01. [15p] Observare porturi deschise pe o stație

Utilitarul `netcat` este folosit pentru a crea conexiuni server-client. Acesta poate îndeplini atât rolul de server, cât și de client.

Pentru a crea un server care ascultă conexiuni pe portul TCP `2024` rulăm comanda

```
root@host:~# netcat -l 2024
```

Parametrul `-l` înseamnă că `netcat` este în modul *listen* (*server*), iar `2024` este portul pe care ascultă: `netcat` va crea un socket de tip server și va aștepta conexiuni pe portul `2024`. Terminalul va rămâne în așteptare, pentru ca un client să se conecteze serverul deschis cu `netcat`. **NU** închideți terminalul.

Dorim să verificăm faptul că pe stația `host` un serviciu ascultă pe portul `2024`. Deschidem un nou terminal către stația `host` și rulăm comanda

```
root@host:~# netstat -tlnp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
[...]
tcp        0      0 0.0.0.0:2024            0.0.0.0:*               LISTEN      4958/netcat
[...]
```

Comanda `netstat` afișează serviciile TCP care așteaptă conexiuni pe stația locală. Observăm că programul `netcat` ascultă pe portul `2024`. Opțiunile comenzii `netstat` sunt:

- `t` - afișează conexiuni TCP
- `l` - afișează serviciile care **așteaptă** (*listen*) conexiuni
- `n` - dezactivează rezolvare de nume pentru adrese IP și porturi; de obicei dezactivăm rezolvarea numelor pentru a elimina latența procesului de rezolvare;
- `p` - afișează numele și PID-ul procesului de tip serviciu de rețea care așteaptă conexiuni pe portul dat.

În afișare construcția `0.0.0.0:2024` din coloana `Local Address` înseamnă că serviciul ascultă conexiuni pe toate adresele IP/interfețele disponibile sistemului pe portul `2024`.

În terminalul nou deschis către stația `host` conectați-vă la serviciul proaspăt creat. Folosiți pentru aceasta tot comanda `netcat` în modul client:

```
root@host:~# netcat localhost 2024
```

Comanda a creat un socket de tip client și s-a conectat la serviciul de mai sus (conexiune TCP pe portul `2024`). În continuare comanda așteaptă intrare de la utilizator. Mesajele pe care le scriem vor fi transmise serverului care le va afișa la ieșirea standard. Scrieți câteva linii și observați afișarea acestora pe partea serverului. Închideți folosind combinația de taste `Ctrl+d`. Se închide și serverul.

Pe stația `host`, realizați pașii de mai sus pentru UDP:

- creați un server care ascultă pachete UDP pe portul 2024;
- listați serviciile care ascultă pachete UDP pe stația `host`;
- porniți un client UDP și comunicați cu serverul de mai sus.

Pentru a afla ce opțiune trebuie transmisă comenzii `netcat` pentru a porni un server sau client UDP, consultați pagina de manual a comenzii și căutați șirul `UDP`.

La fel, pentru a afla ce opțiune trebuie transmisă comenzii `netstat` pentru a afișa socketii UDP, consultați pagina de manual a comenzii și căutați șirul `udp`.

02. [10p] Port forwarding pentru accesare serviciu web

Pe stația `red` este instalat un server de HTTP pe care dorim să îl accesăm din browser-ul stației `fep.grid.pub.ro`. Pentru acest lucru trebuie trebuie să redirectați portul `8080` de pe stația `host` spre portul `80` de pe stația `red` folosind DNAT (vezi 05. [10p] Port forwarding).

Pentru testare, deschideți browser-ul web de pe `fep.grid.pub.ro` și accesați adresa IP a stației `host` pe portul `8080`. Vă va apărea o pagină web cu mesajul *Laboratorul 10 - pe red*. Pentru a porni browser-ul pe stația `fep.grid.pub.ro` conectați-vă cu parametrul `-X` al comenzii `ssh`:

```
student@mjolnir:~$ ssh -X mihai.carabas@fep.grid.pub.ro
mihai.carabas@fep.grid.pub.ro's password:

[mihai.carabas@fep-62-2 ~]$ firefox
```

Din cadrul browser-ului pornit pe stația `fep.grid.pub.ro` vrem să accesăm serviciul web de pe stația `red` prin intermediul stației `host` pe care facem NAT; adică pachetele HTTP trimise de pe stația `fep.grid.pub.ro` către portul `8080` al stației `host` să ajungă pe stația `red` pe portul `80`. Pentru aceasta, în address bar-ul din browser-ul Firefox deschis pe stația `fep.grid.pub.ro` scriem URL-ul `http://<adresa-ip-mv>:8080/`, unde `<adresa-ip-vm>` este adresa IP publică a mașinii, adresă pe care o putem obține din dashboard-ul OpenStack.

03. [10p] Accesare serviciu web folosind wget

Sunt situații în care:

- vrem doar să descărcăm o resursă, nu să accesăm o pagină web și să o parcurgem;
- vrem să testăm serviciul web;
- nu avem acces la o interfață grafică (pe un server sau pe o mașină virtuală) pentru accesarea web;
- vrem să automatizăm, într-un script, procesul de acces al unor resurse în web.

În aceste situații, este util să folosim un client web în linie de comandă. Astfel de clienți web în linie de comandă sunt `wget` sau `curl`. În general aceste utilitare sunt folosite pentru descărcarea de fișiere sau alte resurse disponibile în web.

Pentru a descărca fișierul HTML accesat anterior (la 02. [10p] Port forwarding pentru accesare serviciu web) executați, pe stația `fep.grid.pub.ro`, respectiv pe stația `host` (urmăriți **promptul** pentru a determina stația), comenzile de mai jos (`<adresa-ip-mv>` o puteți obține din dashboard-ul OpenStack):

```
[mihai.carabas@fep-62-2 ~]$ wget http://<adresa-ip-mv>:8080
--2013-11-26 18:20:10--  http://<adresa-ip-mv>:8080/
Connecting to <adresa-ip-mv>:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 31 [text/html]
Saving to: ‘index.html’

100%[======================================>] 31          --.-K/s   in 0s

2013-11-26 18:20:10 (4.92 MB/s) - ‘index.html’ saved [31/31]

[mihai.carabas@fep-62-2 ~]$ cat index.html
<h1>Laborator 10 - pe red</h1>

root@host:~# wget -q http://red/
root@host:~# ls
PacketTracer5   index.html
root@host:~# cat index.html
<h1>Laborator 10 - pe red</h1>
```

Avem același conținut afișat în browser-ul web grafic: șirul *Laboratorul 10 - pe red* formatat *heading 1*.

În a doua comandă am folosit opțiunea `-q` a `wget` pentru a dezactiva afișarea informații legate de transfer a `wget`. Opțiunea trebuie folosită cu grijă. Informațiile de eroare furnizate de `wget` sunt utile pentru depanare și sunt dezactivate la folosirea opțiunii `-q`.

La adresa `http://red/file.dat` se află un fișier cu dimensiunea de `1 KB`. Descărcați fișierul pe stația `host` folosind `wget`. Folosiți comanda `ls -l` pentru a verifica descărcarea corectă și dimensiunea sa.

Serverul web este configurat să folosească și conexiuni sigure (protocolul HTTPS). Descărcăm fișierul de mai sus (`file.dat`) pe stația `host` folosind protocolul HTTPS:

```
root@host:~# wget https://red/file.dat
--2013-11-23 17:50:52--  https://red/file.dat
Resolving red (red)... 192.168.1.2
Connecting to red (red)|192.168.1.2|:443... connected.
ERROR: cannot verify red's certificate, issued by `/CN=red':
  Self-signed certificate encountered.
To connect to red insecurely, use `--no-check-certificate'.
```

Observăm că ni s-a afișat o eroare că nu poate fi verificat certificatul. Această verificare poate fi eliminată folosind opțiunea `--no-check-certificate`:

```
root@host:~# wget --no-check-certificate https://red/file.dat
--2013-11-23 17:52:02--  https://red/file.dat
Resolving red (red)... 192.168.1.2
Connecting to red (red)|192.168.1.2|:443... connected.
WARNING: cannot verify red's certificate, issued by `/CN=red':
  Self-signed certificate encountered.
HTTP request sent, awaiting response... 200 OK
Length: 1024 (1.0K)
Saving to: `file.dat'

100%[==========================================================================================

2013-11-23 17:52:02 (18.2 MB/s) - `file.dat' saved [1024/1024]
```

04. [15p] Descărcare recursivă de resurse web

La adresa `http://red/folder` se află o structură de directoare și fișiere. Pe stația `host`, descărcați întreaga structura folosind o singură comandă `wget` cu parametrul corespunzător.

Pentru a identifica opțiunea corespunzătoare, căutați în pagina de manual a `wget` după șirul `recursive`.

Instalați utilitarul `tree` și folosiți-l petru a lista conținutul directorului descărcat. Observați prezența mai multor fișiere cu nume de forma `index.html?C=S;O=D`. Aceste fișiere sunt fișiere de tip directory listing generate de serverul web și nu avem nevoie de ele. Ștergeți structura de directoare pe care tocmai ați descărcat-o pe stația `host` și descărcați-o din nou ignorând aceste fișiere.

Urmăriți discuția de aici. Să aveți în vedere și partea din discuția care conține șirul *"without the hostname"*.

Structura de directoare trebuie să fie similară cu:

```
root@host:~# tree folder/
folder/
├── embedded
│   ├── Qualcomm
│   │   ├── Krait
│   │   │   └── info.txt
│   │   └── Scorpion
│   │       └── info.txt
│   └── TI
│       ├── OMAP3
│       │   └── info.txt
│       └── OMAP4
│           └── info.txt
└── mobile
    ├── Apple
    │   ├── iPadAir
    │   │   └── info.txt
    │   └── iPhone5S
    │       └── info.txt
    └── Google
        ├── Nexus4
        │   └── info.txt
        └── Nexus7
            └── info.txt

14 directories, 8 files
```

05. [5p] Accesare URL conținând caractere speciale

Unele URL-uri pot avea caractere interpretate special în shell rezultând într-un comportament ciudat la descărcare.

Pentru a evidenția acest lucru, pe stația `host` folosiți pentru a descărca resursa de la link-ul `http://localhost/login.php?name=Lab10&email=rl@upb.ro`:

```
root@host:~#  wget http://localhost/login.php?name=Lab10&email=rl@upb.ro
[1] 3284
root@host:~# --2013-12-05 09:02:10--  http://localhost/login.php?name=Lab10
Resolving localhost (localhost)... 127.0.0.1
Connecting to localhost (localhost)|127.0.0.1|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 72 [text/html]
Saving to: `login.php?name=Lab10'

100%[======================================>] 72          --.-K/s   in 0s

2013-12-05 09:02:10 (13.3 MB/s) - `login.php?name=Lab10' saved [72/72]


[1]+  Done                    wget http://localhost/login.php?name=Lab10
root@host:~#
```

După output-ul comenzii de mai, pentru a vă reapărea prompt-ul consolei, apăsați pe `Enter` ca să vă reapară prompt-ul.

Observați că procesul intră în background (linia cu `[1] 3284` înseamnă că a fost pornit un job). Acest lucru se întâmplă întrucât simbolul `&` (*ampersand*) nu este escapat și se folosește rolul său special din shell: pornirea unui proces în background. Se observă din output că este separată comanda înainte de `&`, adică rezultă comanda `wget` `http://localhost/login.php?name=Lab10`.

Ștergeți fișierul descărcat prin comanda de mai sus:

```
root@host:~# ls
PacketTracer5  folder  login.php?name=Lab10
root@host:~# rm login.php\?name\=Lab10
root@host:~# ls
PacketTracer5  folder
```

Folosiți escaping în shell pentru a folosi în mod corect URL-ul și pentru a descărca resursa disponibilă la acel link.

Fișierul final descărcat trebuie să arate în acest fel (să conțină *Lab10* și *rl@upb.ro*). Folosiți `cat log<TAB>` pentru rula comanda de mai jos, ca să obțineți *autocompletion* din partea shell-ului (`<TAB>` înseamnă apăsarea tastei `TAB`):

```
root@host:~# cat login.php\?name\=Lab10\&email\=rl@upb.ro
<html>
<body>

Welcome Lab10<br>
Your email address is: rl@upb.ro
</body>
</html>
```

06. [5p] Accesare de pagini web folosind curl

Utilitarul `wget` este folosit pentru a descărca fișiere prin intermediul web. Utilitarul `curl` îndeplinește un rol similar, doar că afișează în mod implicit conținutul resurselor la consolă. Este indicat pentru afișarea de pagini web (format HTML) sau alte fișiere în format text.

Spre exemplu, dacă vrem să afișăm conținutul paginii de la adresa `http://red/index.html` vom folosi comanda:

```
root@host:~# curl http://red/index.html
<h1>Laborator 10 - pe red</h1>
```

Observăm că output-ul comenzii `curl` este conținutul fișierului `index.html` descărcat anterior cu `wget`.

Un exemplu în mod particular util este aflarea adresei IP (externe) prin interogarea site-ului `icanhazip.com`:

```
root@host:~# curl icanhazip.com
141.85.225.204
```

Observăm că în urma interogării site-ului `icanhazip.com`, output-ul comenzii `curl` este adresa noastră IP externă (rutabilă).

În general output-ul întors de `curl` poate fi prelucrat folosind filtre de text (`grep`, `cut`, `tr`, `sort`, `awk`, `sed` etc.) pentru a obține informațiile de interes.

07. [5p] Download FTP folosind curl (fără autentificare)

Utilitarele `curl` și `wget` sunt folosite atât pentru transfer HTTP cât și pentru transfer FTP sau alte protocoale. Avantajul folosirii acestor utilitare în fața altora este că sunt neinteractive și pot fi adăugate în scripturi.

Vom folosi `curl` pentru a descărca un fișier prin FTP. Pe stația `red` rulează un server FTP și pe stația `host` vom folosi comanda `curl` pentru a descărca fișierul localizat la `/download/file-10M.dat`:

```
root@host:~# curl ftp://red/download/file-10M.dat
[...]
```

Observăm că apar multe caractere ciudate pe ecran. Anulați folosind combinația de taste `Ctrl+c`.

Pentru a salva într-un fișier conținutul de la distanță folosim opțiunea `-o` a comenzii `curl`:

```
root@host:~# curl ftp://red/download/file-10M.dat -o file-10M.dat
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 10.0M  100 10.0M    0     0  86.8M      0 --:--:-- --:--:-- --:--:-- 91.7M
root@host:~# ls -l  file-10M.dat
-rw-r--r-- 1 root root 10485760 Nov 26 19:54 file-10M.dat
```

Mai sus utilitarul `curl` a afișat un sumar al transferului și apoi am validat, folosind comanda `ls -l` lungimea fișierului descărcat. URL-ul folosit începe cu `ftp://` pentru a semnaliza folosirea protocolului FTP pentru transfer.

Același efect ca mai sus poate fi obținut prin rularea comenzii `wget`:

```
root@host:~# wget ftp://red/download/file-10M.dat
--2013-11-26 19:55:49--  ftp://red/download/file-10M.dat
           => `file-10M.dat'
Resolving red (red)... 192.168.1.2
Connecting to red (red)|192.168.1.2|:21... connected.
Logging in as anonymous ... Logged in!
==> SYST ... done.    ==> PWD ... done.
==> TYPE I ... done.  ==> CWD (1) /download ... done.
==> SIZE file-10M.dat ... 10485760
==> PASV ... done.    ==> RETR file-10M.dat ... done.
Length: 10485760 (10M) (unauthoritative)

100%[======================================>] 10,485,760  --.-K/s   in 0.07s

2013-11-26 19:55:49 (138 MB/s) - `file-10M.dat' saved [10485760]
```

08. [10p] Download FTP folosind curl (cu autentificare)

Ne propunem să folosim curl pentru descărcarea unui fișier FTP însă folosind autentificare. Fișierul căutat se găsește în directorul home al utilizatorului `ana` de pe red și se numește `ana-ftp-file-5M.dat`. Utilizatorul `ana` are parola `student`.

Pe stația `host` folosiți `curl` pentru a descărca, prin FTP, fișierul `ana-ftp-file-5M.dat` din directorul home al utilizatorului `ana`.

URL-ul folosit va fi `ftp://red/ana-ftp-file-5M.dat`. Va fi relativ la directorul home al utilizatorului autentificat.

Pentru autentificarea ca utilizatorul `ana` în `curl`, folosiți opțiunea `--user`. Vedeți informații în pagina de manual a utilitarului `curl`. Comanda trebuie să fie neinteractivă: trebuie să furnizați în linia de comandă și utilizatorul și parola.

09. [10p] Trimitere e-mail din linia de comandă

Pentru activități de automatizare (transmiterea de notificări și anunțuri) este util să putem transmite e-mail-uri din linia de comandă în mod neinteractiv. Trimiterea din linia de comandă presupune scrierea a trei componente așa cum se întâmplă și într-un client de e-mail:

- destinatarul
- subiectul mesajului
- conținutul mesajului

Pentru transmiterea unui mesaj în linia de comandă folosim comanda `mail`. Pentru a transmite un mesaj de la utilizatorul `bogdan` către utilizatorul `corina` (ambii pe stația `host`), ne autentificăm ca utilizatorul `bogdan`:

```
root@host:~# su - bogdan
bogdan@host:~$
```

și folosim comanda (apăsați `ENTER` acolo unde vă cere `Cc`; nu vrem să introducem adresă de tipul *Carbon Copy*)

```
bogdan@host:~$ mail -s "Invitatie la film" corina
Cc:
Buna, Corina!

Vii la filmul "Ender's Game"?

Bogdan
^D
```

Mesajul de mai sus a fost închis folosind combinația de taste `Ctrl+d`, însemnând *End of File* (EOF).

Pentru a verifica faptul că mesajul e transmis, ne autentificăm ca utilizatorul `corina` și rulăm comanda

```
corina@host:~$ mail
"/var/mail/corina": 1 message 1 new
>N   1 bogdan@host        Tue Nov 26 21:07  17/419   Invitatie la film
? 1
[...]
Subject: Invitatie la film
To: <corina@host>
[...]
From: bogdan@host

Buna, Corina!

Vii la filmul "Ender's Game"?

Bogdan
? q
Saved 1 message in /home/corina/mbox
Held 0 messages in /var/mail/corina
```

Comanda `mail`, fără argumente citește căsuța poștală implicită a utilizatorului (din `/var/mail/<username>`). Promptul comenzii este `?` și permite introducerea de comenzii. Mai sus am folosit comanda `1` pentru a citi primul mesaj și apoi comanda `q` pentru a părăsi utilitarul. În conținutul mesajului apare și destinatarul (câmpul `To`) și subiectul mesajului (câmpul `Subject`).

Mesajul nu este obligatoriu să fie scris la intrarea standard ci poate fi redirectat dintr-un fișier sau o altă comandă. Un mesaj simplu poate fi trimis și folosind comanda:

```
bogdan@host:~$ echo 'Hello, Corina' | mail -s "Salutari" corina
```

Verificăm mesajul folosind comanda `mail` în contul utilizatorului `corina`:

```
corina@host:~$ mail
"/var/mail/corina": 1 message 1 new
>N   1 bogdan@host        Tue Nov 26 21:13  13/371   Salutari
? 1
[...]
Subject: Salutari
To: <corina@host>
[...]
From: bogdan@host

Hello, Corina
? q
Saved 1 message in /home/corina/mbox
Held 0 messages in /var/mail/corina
```

Trimiteți un mesaj simplu din contul utilizatorului `bogdan` către o adresă a voastră de e-mail (de preferat GMail). Verificați și în Spam/Junk primirea mesajului.

10. [15p] Trimitere e-mail cu adresă personalizată din linia de comandă

La exercițiul anterior, mesajul primit apare ca sosind de la `bogdan@host`. Nu este foarte elegant și am vrea să-i schimbăm adresa expeditorului.

Pentru aceasta vom folosi comanda `mail` disponibilă în pachetul `s-nail` care permite această opțiune. Pentru început instalăm cele necesare:

```
root@host:~# apt-get install s-nail
Reading package lists... Done
Building dependency tree
[...]
```

Acum comanda `mail` va fi furnizată de către pachetul `s-nail`.

Trimiteți un mesaj la adresa voastră de e-mail (de preferat GMail) astfel încât expeditorul mesajului să fie `bogdan.georgescu@rl.cs.pub.ro`.

Comanda `mail` din pachetul `s-nail` are opțiunea `-r` cu ajutorul căreia puteți preciza adresa de e-mail sursă care să fie folosită pentru trimiterea e-mail-ului.

În forma sa simplă, protocolul SMTP nu face verificarea adresei sursă a utilizatorului. Dacă utilizatorul se conectează la *Mail Transfer Agent* de pe o stație considerată sigură (de exemplu chiar de pe stația locală – *localhost*), atunci acesta poate folosi orice adresă. Pentru autentificare se folosește SMTP cu suport SASL, care obligă autentificarea utilizatorului înainte de a folosi serviciul SMTP.

Detalii despre fenomen și prevenirea *e-mail forging/spoofing* găsiți aici.

11. [BONUS - 10p] Upload prin FTP

Dorim ca pe lângă download, să uploadăm fișiere prin FTP. Din rațiuni de securitate, nu se recomandă upload anonim, ci doar cu autentificare.

În directorul home al utilizatorului `bogdan` de pe stația `host` se găsește fișierul `bogdan-ftp-data-3M.dat`. Folosiți `curl` pentru a uploada acest fișier în directorul home al utilizatorului `ana` de pe stația `red`.

Căutați, în pagina de manual a comenzii `curl`, șirul `upload-file`.

După ce ați realizat upload-ul, descărcați prin FTP, din directorul home al utilizatorului `ana`, fișierul `bogdan-ftp-data-3M.dat` în directorul home al utilizatorului `corina` de pe stația `host`.

12. [BONUS - 10p] Trimitere e-mail cu atașament din linia de comandă

Dorim să trimitem din linia de comandă mesaje cu atașament.

În contul utilizatorului `bogdan` creați o arhivă (în ce format doriți) a directorului `/etc/init.d/`. Trimiteți arhiva ca atașament într-un e-mail trimis în linia de comandă către adresa voastră.

Pentru trimiterea atașamentului, va trebui să folosiți utilitarul `uuencode`. Pentru aceasta trebuie să instalați pachetul `sharutils`:

```
root@host:~# apt-get install sharutils
Reading package lists... Done
Building dependency tree
[...]
Setting up sharutils (1:4.11-1) ...
```

Pentru indicații legate de trimiterea de atașamente ca e-mail consultați această pagină.

În modul uzual de folosire comanda `uuencode` primește două argumente cu aceeași valoare (de exemplu `store.zip store.zip`). Aceasta întrucât primul argument este numele fișierului care este codificat, iar al doilea argument este numele arhivei așa cum apare ea în atașament. Teoretic, este posibil să codifici un fișier și să aibă alt nume în atașament; în practică se folosește același nume, motiv pentru care apar două argumente cu aceeași valoare.

Folosiți opțiunea `-s "<subiect>"` la comanda `mail` pentru a preciza subiectul mesajului; înlocuiți `<subiect>` cu subiectul mesajului. Eticheta în Internet dictează să folosiți subiecte pentru e-mail-uri.

Navigare

Laboratorul 10

- 01. [15p] Observare porturi deschise pe o stație
- 02. [10p] Port forwarding pentru accesare serviciu web
- 03. [10p] Accesare serviciu web folosind wget
- 04. [15p] Descărcare recursivă de resurse web
- 05. [5p] Accesare URL conținând caractere speciale
- 06. [5p] Accesare de pagini web folosind curl
- 07. [5p] Download FTP folosind curl (fără autentificare)
- 08. [10p] Download FTP folosind curl (cu autentificare)
- 09. [10p] Trimitere e-mail din linia de comandă
- 10. [15p] Trimitere e-mail cu adresă personalizată din linia de comandă
- 11. [BONUS - 10p] Upload prin FTP
- 12. [BONUS - 10p] Trimitere e-mail cu atașament din linia de comandă

Laborator

Laboratorul 10

- 01. [15p] Observare porturi deschise pe o stație
- 02. [10p] Port forwarding pentru accesare serviciu web
- 03. [10p] Accesare serviciu web folosind wget
- 04. [15p] Descărcare recursivă de resurse web
- 05. [5p] Accesare URL conținând caractere speciale
- 06. [5p] Accesare de pagini web folosind curl
- 07. [5p] Download FTP folosind curl (fără autentificare)
- 08. [10p] Download FTP folosind curl (cu autentificare)
- 09. [10p] Trimitere e-mail din linia de comandă
- 10. [15p] Trimitere e-mail cu adresă personalizată din linia de comandă
- 11. [BONUS - 10p] Upload prin FTP
- 12. [BONUS - 10p] Trimitere e-mail cu atașament din linia de comandă

Curs

Informații generale RL

- **Regulament**
  - Notare Seria CA
  - Notare Seriile CB/CC
- **Resurse**
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

BY-SA CHIMERIC DE W3C CSS DOKUWIKI GET FIREFOX RSS XML FEED W3C XHTML 1.0

Laborator 11. Securitatea Rețelelor Locale

- Feedback CA
- Feedback CB
- Feedback CC

Cunoștințe și abilități ce vor fi dobândite

- Descoperirea de informații despre o anumită rețea, entitate
- Folosirea utilitarelor în linia de comandă în Linux (whois, nmap, wireshark)
- Descoperirea vulnerabilităților
- Înțelegerea tipului de atac MiTM (Man in the Middle) folosind ARP spoofing și DNS spoofing

Cheat sheet

- Cheat Sheet

Pregătire infrastructură de laborator

- **Reminder**: avem nevoie de o masina virtuală a laboratorului. Vă rugăm urmăriți pagina aceasta pentru instrucțiuni, apoi reveniți.
- Schimbati utilizatorul curent ca `root` folosind comanda

```
student@host:~$ sudo su
```

- Pentru a pregăti configurația de laborator, pe mașina virtuală (stația `host`) folosiți comenzile următoare din contul utilizatorului `root` de pe stația `host` (puteți da copy/paste la comenzi în terminal):

```
root@host:~# update_lab --force
root@host:~# start_lab lab11
```

- Deschideți trei noi tab-uri în terminal (folosiți combinația de taste `Ctrl+Shift+t`), și conectați-vă, din nou, la mașina virtuală folosind comanda `ssh` de mai sus.
- De pe cele trei noi tab-uri, conectați-vă la cele trei containere (`red`, `green` și `blue`).
- Pentru o conectare mai usoara puteti folosi aliasul `go` (ex. `go red`)

În mod implicit folosiți contul `root` pentru conectare pe toate stațiile. Aveți nevoie de drepturi privilegiate pentru configurare. Folosiți contul `student` doar unde vi se cere explicit.

Navigare

Laboratorul 11

- 00. Completare formular de feedback
- 01. [10p] Aflare adresă IP publică
- 02. [10p] Aflare informații despre adresa IP publică
- 03. [10p] Descoperire adrese IP online
- 04. [10p] Descoperirea porturilor TCP deschise
- 05. [10p] Descoperire porturi UDP deschise
- 06. [10p] Determinare versiuni aplicații descoperite
- 07. [10p] Scanare porturi frecvent utilizate
- 08. [10p] Detecție sistem de operare
- 09. [10p] Descoperire vulnerabilități
- 10. [10p] Rulare Denial of Service pe un server Web
- 11. [Bonus - 20p] Rulare Man in The Middle

Exerciții

00. Completare formular de feedback

Vă invităm să evaluați activitatea echipei de RL și să precizați punctele tari și punctele slabe și sugestiile voastre de îmbunătățire a materiei. Feedback-ul vostru este foarte important pentru noi să creștem calitatea materiei în anii următori și să îmbunătățim materiile pe care le veți face în continuare.

Găsiți formularul de feedback în partea dreaptă a paginii principale de RL de pe curs.pub.ro într-un frame numit "FEEDBACK".

- Feedback Moodle

Vă mulțumim!

01. [10p] Aflare adresă IP publică

Pentru început, vrem să aflăm adresa IP publică cu ajutorul căreia ieșim în internet de pe mașina virtuală. Sistemul de pe care lucrați în cadrul acestui laborator se află în spatele unui rețele cloud private și poate accesa internetul datorită unei configurații NAT (Network Address Translation) și are un IP privat alocat de controller-ul de rețea cloud. Cum aflăm adresa noastră publică ?

Reamintiți-vă care sunt clasele de adrese IP private și care sunt cele pentru adrese IP publice.

02. [10p] Aflare informații despre adresa IP publică

Mai departe vrem să aflăm din ce subnet face parte adresa publică, cine se ocupă de aceasta, cărei organizații ii aparține și cărei țări a fost alocată.

Whois este un protocol cerere răspuns care este folosit pentru a interoga informații despre resurse din Internet precum: numele de domeniu, subnet-uri de adrese IP sau sistemul autonom (autonomous system) din care face parte. Vom folosi utilitarul **whois** pentru a afla acest informații despre adresa IP descoperită la exercițiul anterior:

```
root@host: whois <ip_ex_1>
```

Ce informații putem observa în output-ul obținut în urma rulării acestei comenzi ? Care este subnet-ul din care face parte adresa noastră publică ?

03. [10p] Descoperire adrese IP online

Procesul de scanare a rețelelor, în afara spațiului de adrese menționat în cadrul laboratorului, poate fi semnalat ca atac cibernetic de autoritățile abilitate și poate cădea sub incidența legii, în situații particulare. Folosiți astfel de unelte cu precauție.

Vrem să vedem ce adrese IP sunt online pentru ca ulterior să analizăm ce servicii rulează pe acestea. Din motive legale, nu putem realiza o scanare activa cu `nmap` in cadrul retelelor ce tin de domeniul `pub.ro`. In consecinta, strict in cadrul acestui exercitiu, vom scana subnetul din care face parte masina voastra virtuala de pe OpenStack.

In general, nu vrem să stăm foarte mult timp pentru a scana un subnet cu foarte multe adrese IP (in cazul nostru subnet-ul are masca /16) astfel încât la acest exercițiu vom scana doar primele 4094 adrese IP. Ce mască vom utiliza? Pentru a realiza acest exercițiu vom folosi utilitarul `nmap`.

Căutați în manual după `Ping Scan`.

Puteți găsi aici un Nmap Cheat Sheet având informații utile legate de nmap.

Redirectionati output-ul într-un fișier sau folosiți less.

Scanarea va dura aproximativ un minut.

04. [10p] Descoperirea porturilor TCP deschise

Mai departe, ne propunem să aflăm ce porturi sunt deschise pe anumite servere pentru a vedea serviciile ce rulează și versiunile acestora ce pot avea vulnerabilități.

În cadrul acestui exercitiu (precum si a exercitiilor urmatoare), vom folosi urmatoarele host-uri:

- `hermes.codacloud.net`
- `midgard.codacloud.net`

Scanați aceste adrese doar pe conexiuni TCP, utilizând porturile următoare: `21, 22, 23, 25, 53, 80, 138, 443, 8000 și 8080` (am ales aceste porturi întrucât sunt cele mai uzuale porturi pentru expunerea serviciilor/aplicațiilor în rețea).

Veți observa porturi ce se află în starea `open`, `closed`, `filtered` (port filtrat de un firewall) și alte tipuri pe care le puteți găsi descrise aici.

05. [10p] Descoperire porturi UDP deschise

De asemenea ne interesează și porturile UDP expuse, nu doar cele TCP.

Scanați aceste adrese pe UDP pe porturile `53, 123, 139, 161, 444, 500, 4567, 5353, 45320, 51413 și 60202`.

06. [10p] Determinare versiuni aplicații descoperite

Pe lângă porturile expuse, descoperite anterior, am dori să aflăm și versiunile aplicațiilor și serviciilor ce rulează pe acele porturi.

Acestea sunt foarte importante când vine vorba de securitate, pentru că anumite vulnerabilități se găsesc doar pe unele versiuni.

Dorim să determinăm versiunile serviciilor ce rulează peste protocoalele TCP și UDP, întâlnite la exercițiile anterioare.

Căutați în manual după `Version detection`.

Procesul de detectare de versiuni îngreunează scanarea. Astfel, recomandăm folosirea parametrului de `timing template` cu valoarea maximă 5 (`-T5`). Acesta ne spune ca scanarea va fi foarte agresivă cu ținta (default-ul este `-T3` normal mode).

Folosind această valoarea (insane mode), se pot depista mult mai ușor sistemele de securitate implementate în infrastructura țintă. Tot pentru a economisi timp, putem folosi parametrul `-n` pentru a nu face rezolvare de DNS.

Pentru a fi mai greu detectabili de către sistemele de securitate, este recomandat să se evite trimiterea pachetelor de ICMP (fără ping) folosind opțiunea `-Pn`

Pentru ca utilitarul nmap să ofere mai multe informații despre ceea ce a scanat, puteți utiliza noțiunea de `verbosity`.

Mai exact, puteți crește nivelul de informații afișate prin adăugarea parametrului `-v`. Puteți, de asemenea, crește nivelul de abundență a informației prin utilizarea unui număr mai mare de v-uri.

07. [10p] Scanare porturi frecvent utilizate

În general, se scanează toate cele `65535` de porturi, deoarece pot exista anumite servicii obscure ce rulează pe porturi cu valori mari, nestandard, care pot prezenta vulnerabilități.

Pentru a micșora timpul de rulare, există posibilitatea de a scana un anumit număr de porturi ce sunt asociate celor mai folosite servicii. Scanați primele 100 de astfel de porturi TCP pe serverul `hermes.codacloud.net`.

Căutați în manual după `-top-ports` (sunt porturile pe care le găsim listate în fișierul `/etc/services`)

08. [10p] Detecție sistem de operare

Putem folosi `nmap` și pentru a detecta sistemul de operare ce rulează pe un anumit server.

În outputul oferit veți observa mai multe variante posibile. Cele cu un procentaj mai mare sunt, în general, mai aproape de adevăr. În general, `nmap` te va informa dacă sistemul de operare e Linux, Windows sau MacOS, dar există situații în care va fi capabil să menționeze atât distribuția, cât și versiunea acesteia. Scanați adresa IP asociată domeniului `hermes.codacloud.net`.

Căutați în manual după `Enable OS detection` și `Guess OS`.

`Nmap` nu oferă întotdeauna informații relevante și, astfel, pot apărea rezultate false positive.

09. [10p] Descoperire vulnerabilități

În procesele ce implică analiza securității unei infrastructuri se folosesc tool-uri de descoperire a vulnerabilităților, precum: Nessus, OpenVAS, Nexpose, etc. La acest laborator nu vom folosit un tool anume, ci vom căuta vulnerabilități pe anumite platforme cunoscute.

În urma rezolvării exercițiului 7 ați obținut pentru serverul `hermes.codacloud.net` o listă de aplicații și versiunile lor ce rulează peste TCP.

Căutați pe platforma https://www.cvedetails.com/ vulnerabilități pentru aplicațiile găsite și versiunile lor, incluzând și patch number. Găsiți-le pe cele mai importante și discutați cu asistentul și colegii despre ele.

Observați formatul vulnerabilităților: CVE-2017-15710 (prefixul CVE, anul și indicele vulnerabilității din anul respectiv). Fiecare dintre acestea are un anumit scor în funcție de cât de mare este impactul pe care îl poate avea asupra serviciului respectiv:

- none pentru 0.0
- low între 0.1 și 3.9
- medium între 4.0 și 6.9
- high între 7.0 și 8.9
- critical între 9.0 și 10.0

De asemenea se pot găsi informații despre CVE-uri și pe https://nvd.nist.gov/ sau https://cve.mitre.org/.

În urma determinării CVE-urilor, începe procesul de căutare de exploit-uri fie pe diverse site-uri (exemplu: https://www.exploit-db.com), fie utilizând Metasploit ce conține o serie de astfel de exploit-uri. Un exploit este o secvență de cod, un set de comenzi sau o succesiune de date ce profită de o defecțiune sau de o vulnerabilitate în scopul de a determina un comportament nedorit sau neprevăzut al software-ului, al hardware-ului de pe un calculator sau de pe alt dispozitiv electronic.

1. Ce informații ați obținut pentru porturile 22 si 80?
2. Ce fel de exploit-uri ați găsit pentru serviciile de mai sus?

10. [10p] Rulare Denial of Service pe un server Web

La acest exercițiu ne propunem să provocăm un comportament distructiv pe un anumit serviciu. Astfel vom realiza un Denial of Service asupra unui server web. Pe `red` există un server web (apache2) deja instalat. Verificați dacă acesta este pornit.

```
root@red:~# /etc/init.d/apache2 status  # Verificați că serverul web este pornit
```

Pe sisteme aveți instalat un text web browser. Dați comanda `elinks` și accesați adresa `http://192.168.1.2/`. Ar trebui să obțineți pagina default a serverului web Apache2 Ubuntu.

În continuare vom folosit un utilitar de Denial of Service numit `slowloris` pe care il avem deja instalat pe `green`. Urmăriți README-ul de la această adresă.

Porniți un nou terminal pe sistemul `host`. Rulati `tcpdump` pe interfața de rețea `veth-red` urmărind pachetele primite. Acum puteți porni tool-ul asupra webserver-ului de pe `red`. Încercați să accesați din nou serverul web. Mai funcționează ?

```
root@student:~# tcpdump -i veth-red -n
```

- Dacă server-ul web tot vă răspunde, folosiți combinatia de taste `CTRL+R` pentru a da refresh la pagina din `elinks`.
- Dacă încă funcționează serverul, încercați să rulați tool-ul din mai multe terminale până nu mai răspunde.

11. [Bonus - 20p] Rulare Man in The Middle

În cadrul acestui exercițiu ne propunem să simulăm un atac de timp Man-in-the-middle. Pentru aceasta, vom considera statia `red` ca fiind sistemul de pe care se inițiază atacul și stația `green` sistemul victimă. Vom încerca să capturăm traficul către `https://curs.upb.ro/` și să îl redirecționăm către un server web malițios aflat pe stația `red`. Într-un scenariu real, un atacator ar putea să creeze o replică a unui site web cu scopul de a captura comportamentul victimei sau informații cu caracter personal (parole, adrese, opțiuni etc.)

Folosind ARP spoofing, DNS spoofing, vom crea un scenariu de MiTM (Man in The Middle).

Inainte de a porni la atac, va trebui sa pregatim infrastructura de laborator pentru acest exercitiu:

```
root@host:~# stop_lab
root@host:~# start_lab lab11 ex12
```

În primă instanță vom porni arpspoof:

```
root@red:~# arpspoof -i <interfață (red-eth0)> -t <ip_victimă 192.168.2.2> <ip_gateway 192.168.
root@red:~# ip a s    # pentru a vedea adresa MAC
root@green:~# arp -n    # comparați adresa MAC a statiei ''red'' cu cea înregistrată pentru gat
```

Deoarece am pornit mai sus ARP spoofing, pachetele victimei trec pe la atacator, inclusiv cererile de tip DNS. Astfel sistemul atacator (MiTM) va răspunde la aceste cereri în locul serverelor dedicate și vor trimite drept rezoluție de nume pentru domeniul `curs.upb.ro` adresa IP scrisă într-un fișier `hosts` pe care va trebui sa il cream. În cazul de față aceasta adresa este adresa IP a atacatorului unde rulează o copie malițioasă a site-ului http://curs.upb.ro/.

```
root@red:~# cat hosts
<adresa_mea_ip>    curs.upb.ro
(folosiți TAB!)
root@red:~# dnsspoof -f hosts
```

Pentru a valida succesul atacului, de pe stația `green` rulați browser-ul text `elinks` si accesați pagina `curs.upb.ro`. Veți observa faptul că, în locul paginii clasice, vă va apărea o pagină web diferită (pagina default Apache).

Resurse:

- http://www.irongeek.com/i.php?page=security/arpspoof
- http://techgenix.com/Understanding-Man-in-the-Middle-Attacks-ARP-Part

Navigare

Laboratorul 11

- 00. Completare formular de feedback
- 01. [10p] Aflare adresă IP publică
- 02. [10p] Aflare informații despre adresa IP publică
- 03. [10p] Descoperire adrese IP online
- 04. [10p] Descoperirea porturilor TCP deschise
- 05. [10p] Descoperire porturi UDP deschise
- 06. [10p] Determinare versiuni aplicații descoperite
- 07. [10p] Scanare porturi frecvent utilizate
- 08. [10p] Detecție sistem de operare
- 09. [10p] Descoperire vulnerabilități
- 10. [10p] Rulare Denial of Service pe un server Web
- 11. [Bonus - 20p] Rulare Man in The Middle

Laborator

Laboratorul 11

- 00. Completare formular de feedback
- 01. [10p] Aflare adresă IP publică
- 02. [10p] Aflare informații despre adresa IP publică
- 03. [10p] Descoperire adrese IP online
- 04. [10p] Descoperirea porturilor TCP deschise
- 05. [10p] Descoperire porturi UDP deschise
- 06. [10p] Determinare versiuni aplicații descoperite
- 07. [10p] Scanare porturi frecvent utilizate
- 08. [10p] Detecție sistem de operare
- 09. [10p] Descoperire vulnerabilități
- 10. [10p] Rulare Denial of Service pe un server Web
- 11. [Bonus - 20p] Rulare Man in The Middle

Curs

Informații generale RL

- **Regulament**
  - Notare Seria CA
  - Notare Seriile CB/CC
- **Resurse**
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri

- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare

- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme

- Tema1
- Tema2

(cc) BY-SA CHIMERIC DE W3C CSS DOKUWIKI GET FIREFOX RSS XML FEED W3C XHTML 1.0

Laborator 12. Amazon Web Services

- Feedback CA
- Feedback CB
- Feedback CC

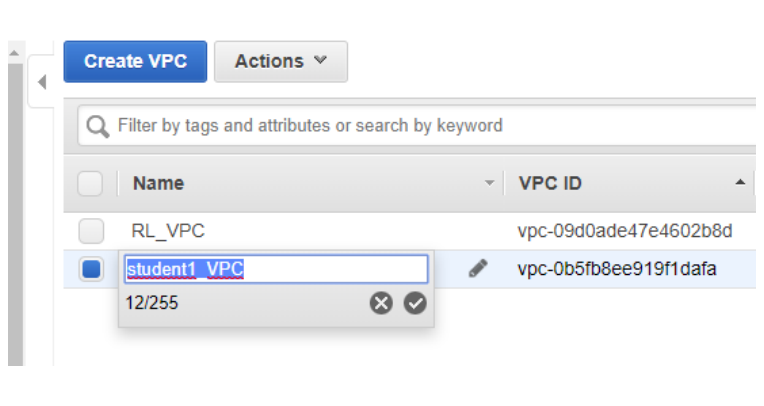
Cunoștințe și abilități ce vor fi dobândite

- Administrarea resurselor cloud folosind consola grafică.
- Securitate în cloud.
- Configurarea rutării în cloud.
- Accesarea serviciilor de transfer de fișiere (FTP).

Pregătire infrastructură de laborator

- Vom crea o infrastructură de rețea în cloud-ul Amazon.
- Fiecare student va folosi aplicația laboratorului. Pentru a vă loga, veți primi parola laboratorului din partea asistentului și veți avea astfel acces la credențialele pentru cloud.

Fiecare student își va denumi resursele astfel *student<X>*_[nume_resursa]. De exemplu *student1*_VPC.



	Name	VPC ID
☐	RL_VPC	vpc-09d0ade47e4602b8d
☑	student1_VPC 12/255	vpc-0b5fb8ee919f1dafa

Prezentare concepte

Servicii de cloud existente: Amazon Web Services, Google Cloud Platform și Microsoft Azure.

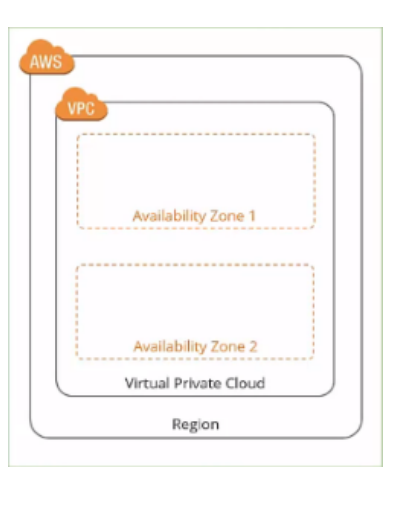
Mai departe discutăm despre AWS; serviciile pe care le vom folosi în cadrul laboratorului:

- Compute: EC2
- Networking and Content Delivery: VPC

Alte servicii în cloud-ul Amazon:

- Storage - Amazon Simple Storage Service (Amazon S3)
- Database
- Machine Learning
- Analytics
- AR&VR
- Mobile

EC2 (Amazon Elastic Compute Cloud) oferă capacitate dinamică computațională în cloud. De asemenea, pentru a dezvolta și lansa aplicații nu mai ai nevoie să îți achiziționezi hardware-ul, acest serviciu punându-ți la dispoziție: servere virtuale, configurarea securității și networking, ajustarea spațiului de stocare.



Virtual Private Cloud te ajută să pornești resursele Amazon într-o rețea virtuală definită de tine. Această rețea seamănă cu o rețea tradițională unde poți configura spații de adrese, tabele de rutare, gateways și setări de securitate, dar are în plus beneficiile utilizării unui infrastructuri scalabile precum AWS. Fiecare VPC creat este izolat logic de oricare alt VPC din cloud.

Fiecare cont Amazon vine cu un VPC prestabilit pentru a putea să folosești resursele Amazon fără să îți construiești infrastructura de la zero. O diagrama simplă pentru un VPC avem în alăturată.

Două sau mai multe VPC-uri pot comunica atâta timp cât se află în aceeași regiune. În plus VPC-urile ale căror blocuri de adrese IP se suprapun nu pot comunica.

3 tipuri de adrese IP:

- adrese IP private
- adrese IP publice. O adresă publică ce a fost atribuită unei instanțe pe care am închis-o va reveni în pool-ul inițial de adrese. Când vom reporni instanța, aceasta va avea o altă adresă IP publică.
- adrese IP publice statice sunt adrese IP publice care rămân asociate contului tău Amazon până în momentul în care alegi să renunți la ele.

Care este diferența între adrese IP private și adresele IP publice?

Subnet - interval de adrese IP într-un VPC. Poți să lansezi resursele Amazon într-un subnet pe care îl creezi, la alegere.

În Amazon avem două tipuri de **subnets**:

- **Publice** (atunci când resursele trebuie să aibă acces direct în Internet, de exemplu serverele web). Un subnet este făcut public atunci când există o rută în tabela de rutare care trimite traficul în Internet printr-un Internet Gateway.
- **Private** (atunci când resursele nu trebuie să aibă acces direct în Internet, de exemplu bazele de date)

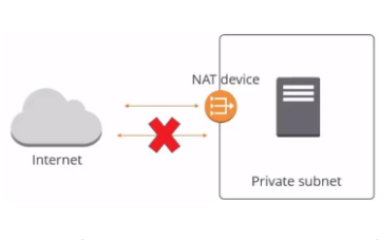
În fiecare subnet, primele 4 adrese IP și ultima adresă IP nu pot fi folosite de utilizatori:

- Prima este adresă de rețea
- A doua este rezervată pentru ruter-ul VPC-ului
- A treia este rezervată de AWS pentru serverul DNS (în general adresa de rețea a subnet-ului plus doi)
- A patra este rezervată de AWS pentru viitoare utilizări
- Ultima este adresa de broadcast

Internet Gateway. IGW este o componentă a VPC-ului care permite comunicarea între instanțele din VPC și Internet. Pentru a comunica cu o instanță din subnet-ul public, este necesar ca instanța să aibă alocată o adresă IP publică statică.

Poți crea un singur obiect IGW per VPC.

Tabela de rutare. Fiecare subnet dintr-un VPC trebuie să aibă asociată o tabelă de rutare. Un subnet poate fi asociat numai cu o singură tabelă de rutare la un moment dat, dar o tabelă de rutare poate fi asociată cu mai multe subnet-uri.



Pentru ca instanțele create într-un subnet privat să acceseze Internetul (dar să nu fie accesate din Internet) putem folosi un obiect AWS numit NAT Device (Network Address Translation Device). (exemplu: web serverul este în subnet-ul public și baza de date în subnet-ul privat. Baza de date s-ar putea să aibă nevoie de conexiune la Internet)

NAT Gateway - serviciu AWS de management al traficului. Folosim un NAT Gateway pentru a permite instanțelor dintr-un subnet privat să aibă acces la Internet sau la alte servicii AWS și în același timp pentru a fi inițiate conexiuni din Internet către stațiile din acel subnet.

Instances
Security group

Un **Security Group** acționează ca un firewall virtual care controlează traficul pentru una sau mai multe instanțe. Putem adăuga reguli pentru fiecare Security Group care să permită traficul către/de la instanțele asociate.

Prestabilit, fiecare Security Groups permite tot traficul de ieșire. Regulile unui Security Group sunt întotdeauna permisive (nu putem adăuga o regulă de „deny"); Un Security Group este stateful (dacă o instanță face o cerere, răspunsul va ajunge la instanță indiferent de regulile de intrare din Security Group). Modificările făcute asupra regulilor dintr-un Security Group se aplică în timp real.

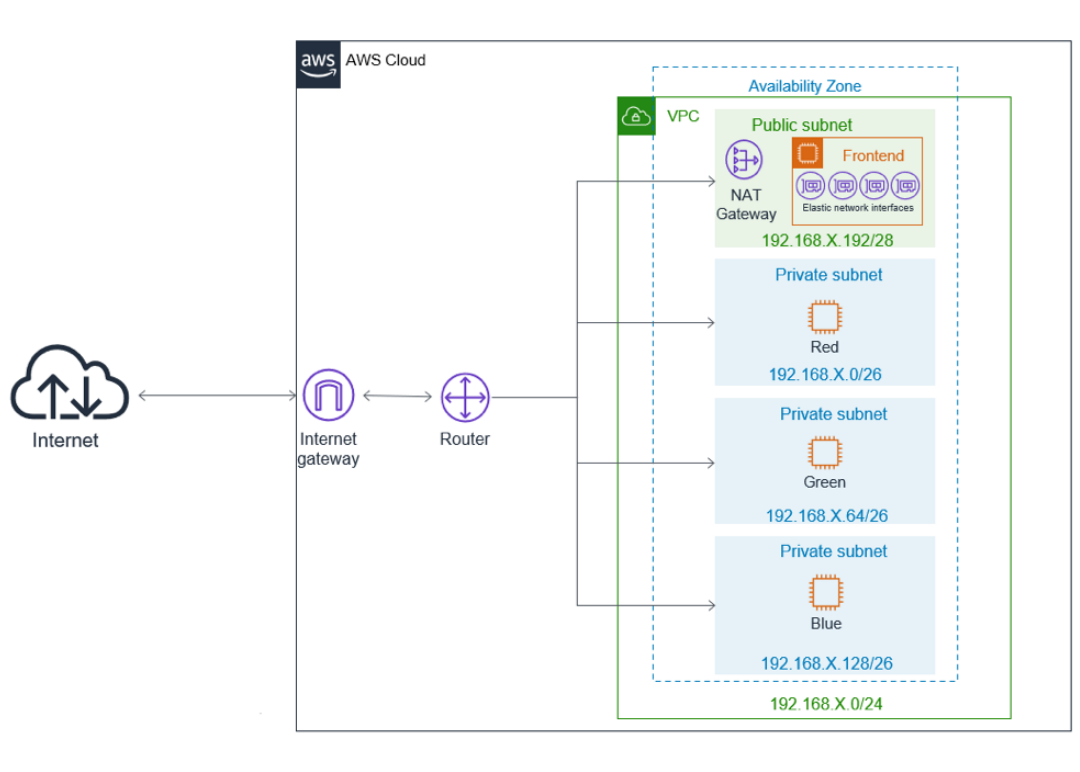
Un **Network ACL** (Access Control List) oferă un nivel opțional de securitate pentru VPC și acționează ca un firewall pentru controlul traficului de intrare și de ieșire pentru unul sau mai multe subnet-uri.

Pentru mai multe detalii despre funcționarea VPC-urilor și resursele AWS, urmăriți tutorialul.

Documentație AWS:

VPC Networking Components
VPC and Subnets
Elastic IP Addresses
Amazon EC2

Topologie



Navigare

Laboratorul 12

- 01. [10p] Creare VPC
- 02. [10p] Creare adresă IP publică
- 03. [10p] VPC Subnets
- 04. [10p] Internet Gateway
- 05. [15p] Tabela rutare
- 06. [15p] NAT Gateway
- 07. [30p] Instanțe EC2
- 08. [BONUS - 5p] ICMP
- 09. [BONUS - 10p] Download FTP folosind curl
- 10. [BONUS - 5p] Blocare FTP

Exerciții

Dorim să ne creăm propriul VPC și să obținem o rețea funcțională conform topologiei de mai sus. Următoarele exerciții constituie pașii de parcurgere pentru ca la final să putem folosi rețeaua.

Deși fiecare student primește câte un username separat, organizația AWS folosită (sponsorizată de către Bitdefender) este numai una, iar aceasta definește namespace-ul tuturor resurselor. De aici rezultă **principala limitare**, faptul că **o să puteți vedea și interacționa cu TOATE obiectele create de către fiecare student** (i.e., resursele sunt partajate între utilizatori).

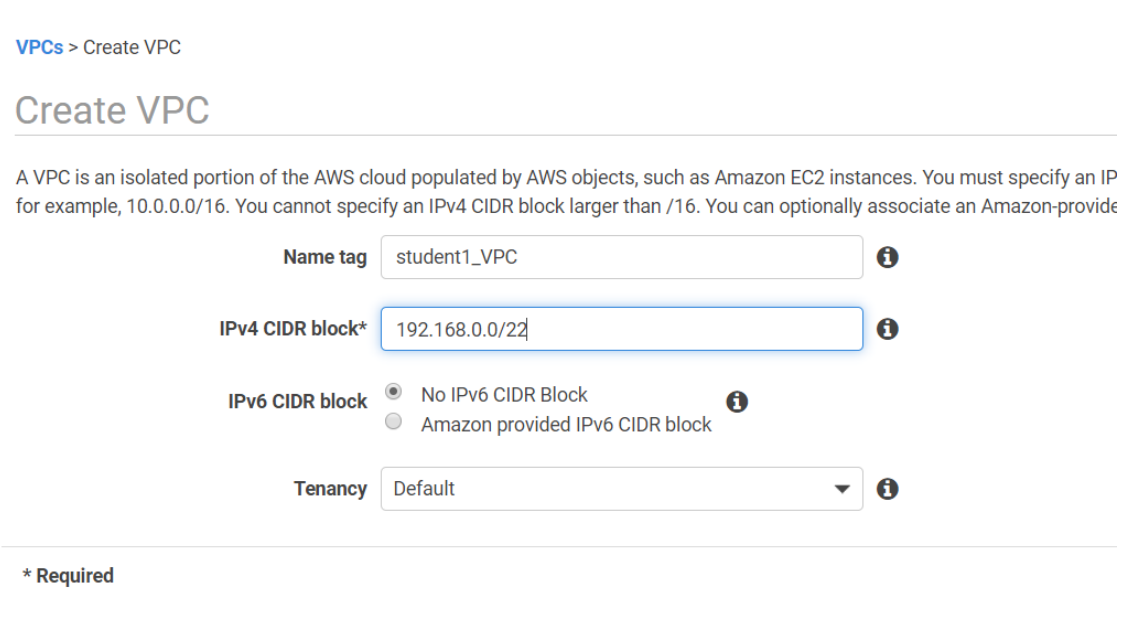
Mai mult, veți putea vedea și niște resurse prefixate cu `admin_`. Acestea sunt folosite pentru găzduirea aplicației de gestiune a conturilor, încercați să nu vă atingeți de ele (deși am implementat restricții de acces, dacă cineva reușește să șteargă / facă aplicația nefuncțională well… o să ne mai vedem și la toamnă / anul :P).

Permisiuni atribuite utilizatorilor student: doar de creare resurse, modificat unde este necesar, taguit doar cu prefixul `student<X>_`, fără drept de ștergere și alte acțiuni ne-necesare laboratorului.

01. [10p] Creare VPC

Din stanga sus, din *Services*, în secțiunea *Networking & Content Delivery* (dati scroll mai jos) alegem VPC. Există 2 moduri de a crea un VPC în contul tău Amazon: folosind un utilitar automat (care face majoritatea acțiunilor fără ca utilizatorul să controleze) sau alegând specificațiile manual.

În *VPC Dashboard*, alegem secțiunea *Your VPCs* și opțiunea **Create VPC**.



VPCs > Create VPC

Create VPC

A VPC is an isolated portion of the AWS cloud populated by AWS objects, such as Amazon EC2 instances. You must specify an IP
for example, 10.0.0.0/16. You cannot specify an IPv4 CIDR block larger than /16. You can optionally associate an Amazon-provide

Name tag: student1_VPC
IPv4 CIDR block*: 192.168.0.0/22
IPv6 CIDR block: ● No IPv6 CIDR Block ○ Amazon provided IPv6 CIDR block
Tenancy: Default

* Required

Numele VPC-ului va fi *student<X>*_VPC. Mai departe, vom alege un bloc de adrese IP, de preferat 192.168.X.0/24, unde X - id-ul studentului. Opțiunea **Tenancy** va fi setată ca *Default*.

02. [10p] Creare adresă IP publică

Dorim să ne creăm propria adresă IP publică statică pentru a o asocia mai târziu unei instanțe din topologia noastră. În *VPC Dashboard*, alegem secțiunea *Elastic IPs* și opțiunea **Allocate New Address** (păstrăm setările default), dăm un tag numit **Name** cu valoarea `studentX_public` și apăsăm **Allocate**.

Este un cost pentru fiecare adresă IP publică statică care nu este alocată unei instanțe ("totul în viață se plătește").

Acum avem în lista de adrese IP publice statice din contul nostru, adresa IP alocată mai devreme.

03. [10p] VPC Subnets

Pentru a lansa instanțe într-un VPC trebuie să avem cel puțin un subnet. În *VPC Dashboard*, alegem secțiunea *Subnets* și opțiunea **Create Subnet**.

Este de preferat să alocăm subnet-uri în mai multe "Zone de disponibilitate"/Availability Zones pentru redundanță. În laboratorul de astăzi, vom crea toate obiectele în același Availability Zone.

Mai departe, vom crea 4 subnets, unul public (pentru instanța Frontend) și 3 private (pentru instanțele Red, Green, Blue).

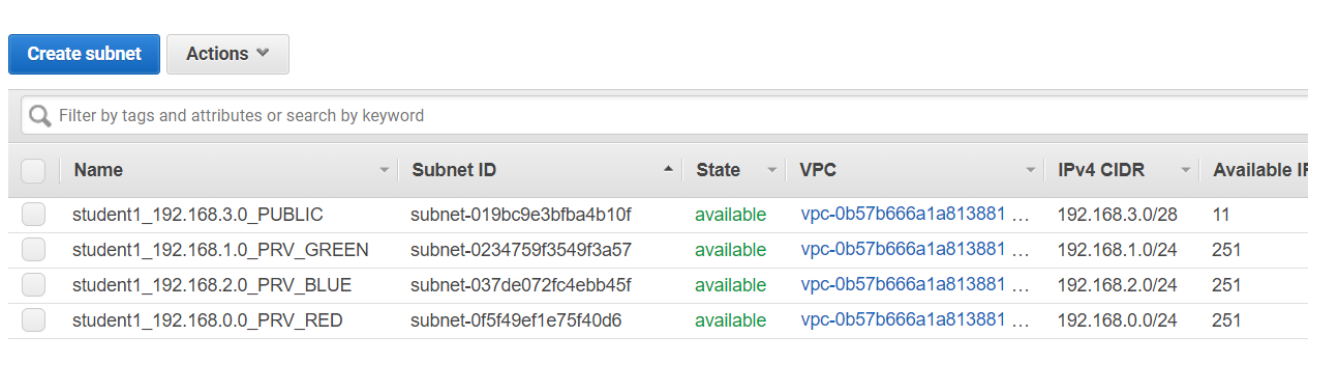
Alegem VPC-ul creat la Exercițiul 1 și alegem **același Availability Zone pentru toate subnet-urile** (*de exemplu, eu-west-1a*). Mai departe, alegem masca de rețea corespunzătoare (/28 pentru subnet-ul public și /26 pentru subnet-urile private):

- 192.168.X.0/26
- 192.168.X.64/26
- 192.168.X.128/26
- 192.168.X.192/28, unde X - id-ul studentului

Recomandăm ca numele pentu subnet-uri să fie sugestive. Pentru exercițiul nostru:

- *student<X>*_RED, unde X - id-ul studentului
- *student<X>*_GREEN
- *student<X>*_BLUE
- *student<X>*_PUBLIC (aici vom adăuga instanța *Frontend*)

Observăm în lista de subnet-uri cele 4 subnet-uri create mai devreme:



	Name	Subnet ID	State	VPC	IPv4 CIDR	Available IF
☐	student1_192.168.3.0_PUBLIC	subnet-019bc9e3bfba4b10f	available	vpc-0b57b666a1a813881 …	192.168.3.0/28	11
☐	student1_192.168.1.0_PRV_GREEN	subnet-0234759f3549f3a57	available	vpc-0b57b666a1a813881 …	192.168.1.0/24	251
☐	student1_192.168.2.0_PRV_BLUE	subnet-037de072fc4ebb45f	available	vpc-0b57b666a1a813881 …	192.168.2.0/24	251
☐	student1_192.168.0.0_PRV_RED	subnet-0f5f49ef1e75f40d6	available	vpc-0b57b666a1a813881 …	192.168.0.0/24	251

De asemenea, putem vedea și tabelele de rutare pentru fiecare subnet.

04. [10p] Internet Gateway

În *VPC Dashboard*, alegem secțiunea *Internet Gateways* și opțiunea **Create Internet Gateway**.

Numele pe care îl alocăm IGW nou creat este *student<X>*_IGW. Trebuie să îl alocăm VPC-ului creat la Exercițiul 1(**Actions** → **Attach to VPC** → Alegem din listă *student<X>*_VPC → **Attach**).

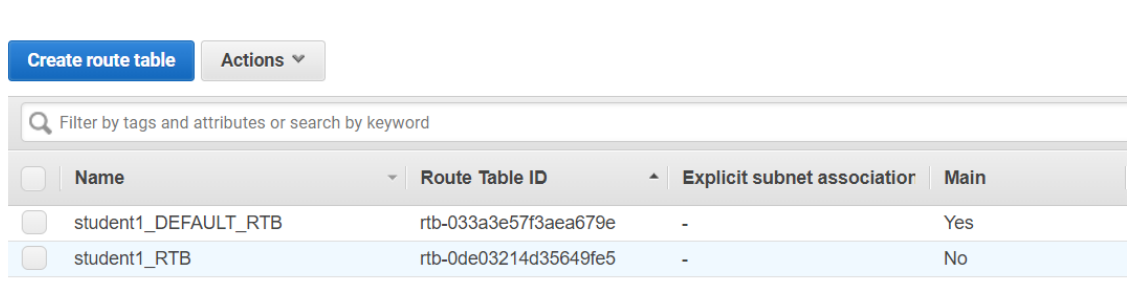
Am discutat mai devreme despre public subnets. Subnet-ul 192.168.X.192/28 va fi subnet-ul public, care va putea accesa direct Internetul prin intermediul IGW. Pentru asta, trebuie să ne asigurăm că **tabela de rutare** a subnet-ului pointează către IGW-ul creat.

05. [15p] Tabela rutare

În *VPC Dashboard*, alegem secțiunea *Route Tables* și opțiunea **Create Route Table**.

Fiecare VPC are o tabela de rutare default (Dați tabelei de rutare default numele *student<X>*_DEFAULT_RTB). Deja există și trebuie redenumită. Inițial, name tag-ul ei este gol.

Dorim să creăm o altă tabelă de rutare. Denumim tabela de rutare: *student<X>*_RTB și alegem VPC-ul creat la Exercițiul 1.



	Name	Route Table ID	Explicit subnet association	Main
☐	student1_DEFAULT_RTB	rtb-033a3e57f3aea679e	-	Yes
☐	student1_RTB	rtb-0de03214d35649fe5	-	No

Mai departe vom modifica această tabelă de rutare pentru a pointa către IGW. Din partea de jos a paginii, alegem secțiunea **Routes** → **Edit routes** → **Add route** (destinație 0.0.0.0/0, target *Internet Gateway* si o sa va apara o intrare de tip *student<X>*_IGW pe care o selectati) → **Save routes**.

În acest moment tabela de rutare nu este asociată niciunui subnet (verificăm în secțiunea de jos **Subnet Associations**), deci toate subnet-urile create în VPC sunt asociate aceleiași tabele de rutare (default route table).

Ne dorim ca doar instanțele din subnetul public să aibă acces la Internet prin IGW (**Edit subnet associations** → Alegem subnetul *student/<X>*_192.168.X.192_PUBLIC → **Save**).

Până aici doar instanțele create în subnet-ul public, asociat IGW vor avea acces la Internet. Ne dorim ca și instanțele pe care le vom crea în subnet-urile private (respectiv Red, Green, Blue) să aibă acces la Internet.

06. [15p] NAT Gateway

Va trebui să adăugăm un NAT Gateway în subnet-ul public (are nevoie de acces la Internet) și de asemenea, să îi alocăm o adresă IP publică statică. (creată la Exercițiul 2).

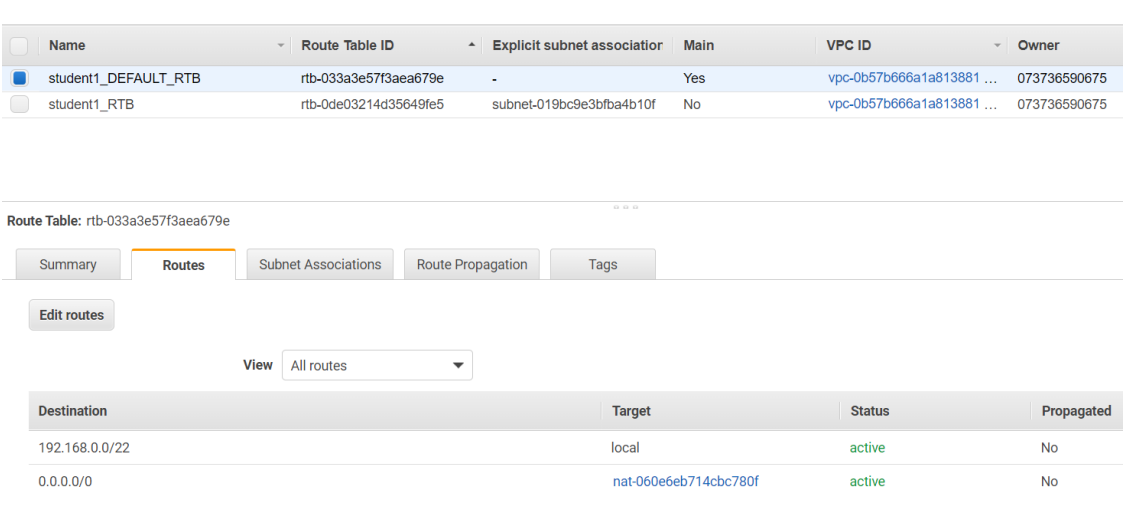
Înainte să creăm NAT GW, mergem în Secțiunea *Subnets* și copiem **Subnet ID-ul** subnet-ului public (așa cum am văzut mai devreme, NAT Gateway-ul trebuie adăugat în subnet-ul public pentru a avea conexiune la Internet).

În *VPC Dashboard*, alegem secțiunea *NAT Gateways* și opțiunea **Create NAT Gateway**.

Aici, specificăm subnet-ul (copiat mai devreme) și alegem adresa IP publică statică creată la Exercițiul 2.

Nu uitați să denumiți fiecare resursă nou-creată folosind prefixul *student<X>*_NatGtw, unde X este ID-ul username-ului vostru.

După crearea NAT GW, actualizăm tabela de rutare asociată subnet-urilor private să pointeze către NAT GW. Alegem **Edit route tables**. Adăugăm o rută în tabela de rutare default: **Routes** → **Edit routes** → **Add route** → destinație 0.0.0.0/0, target *nat*) → **Save routes**.



	Name	Route Table ID	Explicit subnet association	Main	VPC ID	Owner
☑	student1_DEFAULT_RTB	rtb-033a3e57f3aea679e	-	Yes	vpc-0b57b666a1a813881 …	073736590675
☐	student1_RTB	rtb-0de03214d35649fe5	subnet-019bc9e3bfba4b10f	No	vpc-0b57b666a1a813881 …	073736590675

Route Table: rtb-033a3e57f3aea679e

View: All routes

Destination	Target	Status	Propagated
192.168.0.0/22	local	active	No
0.0.0.0/0	nat-060e6eb714cbc780f	active	No

În acest moment, orice instanță creată într-unul din subnet-urile private va putea accesa Internetul.

07. [30p] Instanțe EC2

În EC2 Dashboard (in sus avem Services→ Compute → EC2) , alegem secțiunea *Instances* și opțiunea **Launch Instance**.

Alegem o imagine de mașină virtuală (AMI - Amazon Machine Image) din secțiunea **Choose AMI**. Folosiți imaginea **RL_AWS2**, de la My AMIs (meniul din stânga).

Pentru **Instance Type** alegem astfel:

- Pentru instanța Frontend - t3a.xlarge (avem nevoie de 4 interfețe de rețea)
- Pentru Red, Green și Blue - t2.micro (permite maxim 2 interfețe de rețea, dar noi oricum folosim doar una)

Prima dată creăm cele 3 instanțe Red, Green, Blue. Vom incepe cu instanta Red. În secțiunea **Configure Instance Details**:

- Alegem VPC-ul *student<X>*_VPC
- Alegem subnet-ul corespunzător stației Red

Selectăm secțiunea *Configure Security Group* și alegem astfel:

- pentru prima mașină creată, alegem *Create a new security group*, punem un nume descriptiv (`student<X>_sec_public`). Lăsăm regulile să permită **doar traficul TCP** de la **IP-ul public propriu** la orice destinație.

Regulile de whitelist după IP sunt o practică recomandată pentru protejarea serverelor împotriva atacatorilor externe și putem permite doar IP-urile proprii.

- Pentru celelalte instanțe alegem *Select an existing security group* și selectăm grupul creat anterior.

Selectăm apoi secțiunea **Add Tags**, label-ul *Name* și acolo dăm un nume descriptiv mașinii (`student<X>_Red/Green/Blue/Frontend`).

Ne vom ocupa mai târziu de restul de reguli.

Apăsăm **Review and Launch** → **Launch**. La acest nivel nu ne interesează setările pentru Storage și Tags.

Avem nevoie de o pereche de chei pentru a ne putea conecta la instanța pe care o creăm (alegem **Create a new key pair** și îi dăm numele *student<X>*_RL_lab. **Descărcați cheia privată**).

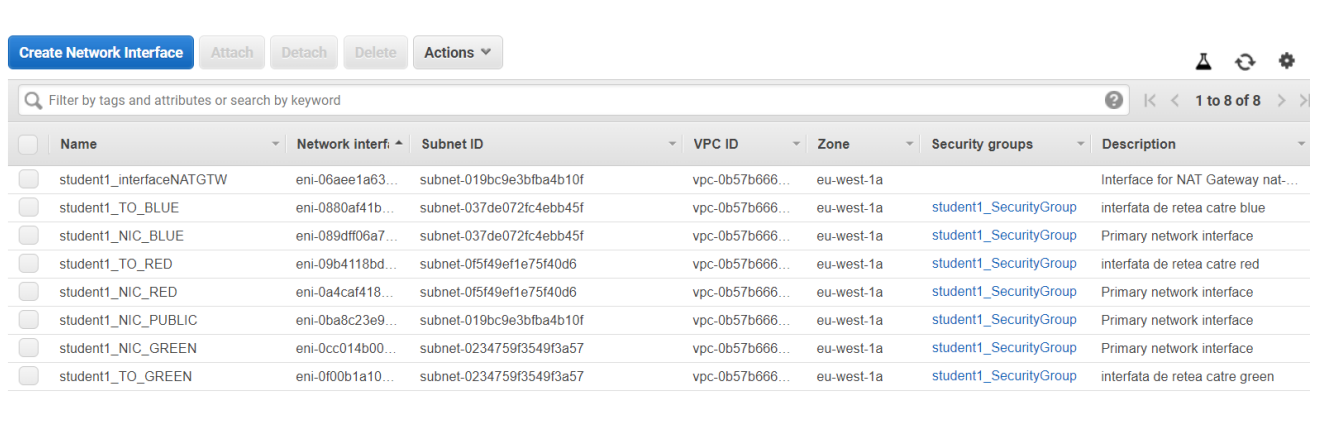
Îi dăm un nume instanței nou create (am început cu Red). Continuăm cu Green și Blue într-un mod similar.

Pentru următoarele instanțe alegem să folosim Security Group-ul si perechea de chei pe care le-am creat anterior.

Din Meniul din stânga, secțiunea *Network&Security*, alegem **Network interfaces**. Ne dorim ca instanța Frontend să aibă câte o interfață și în rețelele cu Red, Green, Blue.

Mai întâi denumim interfețele de rețea create o dată cu instanțele pentru a fi mai ușor de lucrat cu ele.

Alegem **Create Network Interface**. Adăugăm descrieri sugestive și alegem subnet-ul potrivit pentru fiecare interfață nou creată. De asemenea, alegem security group-ul creat anterior. Obținem:



	Name	Network interf.	Subnet ID	VPC ID	Zone	Security groups	Description
☐	student1_interfaceNATGTW	eni-06aee1a63…	subnet-019bc9e3bfba4b10f	vpc-0b57b666…	eu-west-1a		Interface for NAT Gateway nat-…
☐	student1_TO_BLUE	eni-0880af41b…	subnet-037de072fc4ebb45f	vpc-0b57b666…	eu-west-1a	student1_SecurityGroup	interfata de retea catre blue
☐	student1_NIC_BLUE	eni-089dff06a7…	subnet-037de072fc4ebb45f	vpc-0b57b666…	eu-west-1a	student1_SecurityGroup	Primary network interface
☐	student1_TO_RED	eni-09b4118bd…	subnet-0f5f49ef1e75f40d6	vpc-0b57b666…	eu-west-1a	student1_SecurityGroup	interfata de retea catre red
☐	student1_NIC_RED	eni-0a4caf418…	subnet-0f5f49ef1e75f40d6	vpc-0b57b666…	eu-west-1a	student1_SecurityGroup	Primary network interface
☐	student1_NIC_PUBLIC	eni-0ba8c23e9…	subnet-019bc9e3bfba4b10f	vpc-0b57b666…	eu-west-1a	student1_SecurityGroup	Primary network interface
☐	student1_NIC_GREEN	eni-0cc014b00…	subnet-0234759f3549f3a57	vpc-0b57b666…	eu-west-1a	student1_SecurityGroup	Primary network interface
☐	student1_TO_GREEN	eni-0f00b1a10…	subnet-0234759f3549f3a57	vpc-0b57b666…	eu-west-1a	student1_SecurityGroup	interfata de retea catre green

Ne întoarcem în secțiunea Instances. Instanței Frontend îi adăugăm cele trei interfețe de rețea create mai devreme (**Actions** → **Networking** → **Attach Network Interface**).

Nu uitați să denumiți fiecare resursă nou-creată folosind prefixul student<X>_, unde X este ID-ul username-ului vostru.

Fiecare instanță va avea atașată automat o interfață de rețea implicită (*Primary network interface*) cu subnet-ul pe care l-ați ales din wizard.

Din păcate, AWS nu tag-uiește numele acestei interfețe cu cel al instanței, așa că **va trebui să căutați și să dați manual numele acestei interfețe** pentru a nu le încurca între voi imediat după ce creați o instanță. Puteți s-o găsiți ușor apăsând click pe eth0 din descrierea instanței (la Instances, click pe resursă și apar jos aceste detalii).

Pentru a ne putea conecta la stația Frontend, trebuie să îi alocăm o adresă IP publică statică (așa cum am învățat la începutul laboratorului) - vezi exercițiul 2. Asociem adresa IP publică statică interfeței de rețea corespunzătoare subnet-ului public.

Subnet-ul public are conexiune la Internet direct prin intermediul Internet Gateway-ului.

Ne conectăm prin ssh la instanța Frontend folosind comanda (în cazul Linux):

```
ssh -i [private_key] ec2-user@[ip_public]
```

Schimbați permisiunile asupra cheii private (`chmod 700 RL_lab.pem`)

Pe Windows, dacă folosiți Putty, va trebui să convertiți cheia SSH într-un format acceptat urmând ghidul de aici.

Pentru a ne putea conecta prin ssh de pe Frontend și la celelalte instanțe, va trebui să copiem cheia privată de pe stația locală pe Frontend:

```
# pe Linux:
scp -i [private_key] [private_key] ec2-user@[ip_public]:~
```

(pe Windows, puteți folosi WinSCP)

Verificați conectivitatea la Internet pe cele 4 instanțe (Frontend, Red, Green, Blue).

Pentru ca tinem la feedback, va rugam sa completati acest formular de feedback anonim

08. [BONUS - 5p] ICMP

Observați că nu putem să dăm ping de pe oricare stație către oricare stație (între Red, Green și Blue).

Pentru a permite traficul de ICMP, adăugăm o nouă regulă în Security Group-ul *student<X>*_SecurityGroup prin care specificăm tipul de trafic și sursa.

Verificați conectivitatea între Red, Green și Blue.

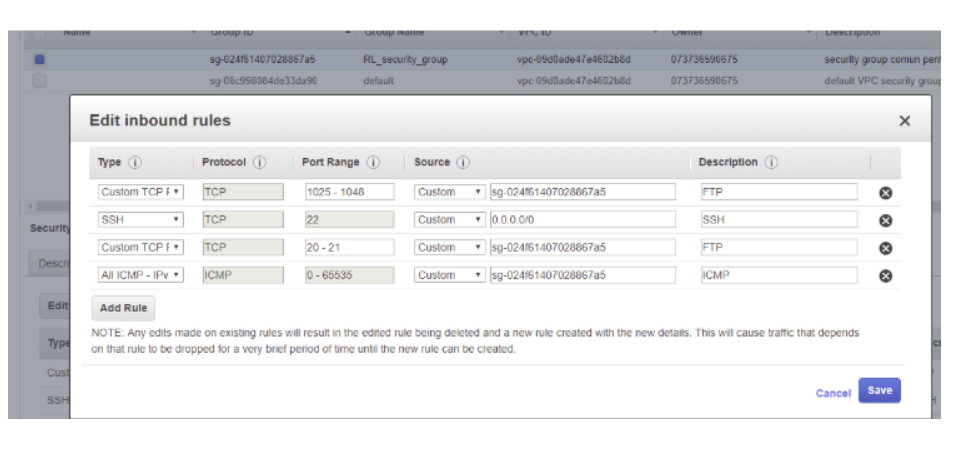
09. [BONUS - 10p] Download FTP folosind curl

Vom folosi curl pentru a descărca un fișier prin FTP. Pe stația Frontend rulează un server FTP și pe stația Red vom folosi comanda curl pentru a descărca fișierul localizat în `/home/student/big.bin` de pe frontend:

```
curl ftp://student:student@<ip_frontend>/big.bin -o big.bin
```

Este nevoie să configurăm una sau mai multe reguli în Security Group pentru a permite descărcarea de fișiere prin FTP.

Adăugăm două reguli Custom TCP Rule, una pentru port range-ul 20-21, respectiv 1025-1048 și la sursă alegem security group-ul în care se află instanțele noastre.



Edit inbound rules

Type	Protocol	Port Range	Source		Description
Custom TCP R	TCP	1025 - 1048	Custom	sg-024f61407028867a5	FTP
SSH	TCP	22	Custom	0.0.0.0/0	SSH
Custom TCP R	TCP	20 - 21	Custom	sg-024f61407028867a5	FTP
All ICMP - IPv	ICMP	0 - 65535	Custom	sg-024f61407028867a5	ICMP

NOTE: Any edits made on existing rules will result in the edited rule being deleted and a new rule created with the new details. This will cause traffic that depends on that rule to be dropped for a very brief period of time until the new rule can be created.

În acest moment, putem descărca fișierul de pe oricare din instanțele Red, Green, Blue.

10. [BONUS - 5p] Blocare FTP

Ne dorim să blocăm FTP pentru stațiile Green și Blue.

Modificăm regulile din Security Group astfel încât să permitem doar stației Red să se conecteze prin FTP la server.

Verificați că puteți descărca fișierul testFTP.dat doar de pe stația Red.

Navigare

Laboratorul 12

- 01. [10p] Creare VPC
- 02. [10p] Creare adresă IP publică
- 03. [10p] VPC Subnets
- 04. [10p] Internet Gateway
- 05. [15p] Tabela rutare
- 06. [15p] NAT Gateway
- 07. [30p] Instanțe EC2
- 08. [BONUS - 5p] ICMP
- 09. [BONUS - 10p] Download FTP folosind curl
- 10. [BONUS - 5p] Blocare FTP

Laborator
Laboratorul 12
- 01. [10p] Creare VPC
- 02. [10p] Creare adresă IP publică
- 03. [10p] VPC Subnets
- 04. [10p] Internet Gateway
- 05. [15p] Tabela rutare
- 06. [15p] NAT Gateway
- 07. [30p] Instanțe EC2
- 08. [BONUS - 5p] ICMP
- 09. [BONUS - 10p] Download FTP folosind curl
- 10. [BONUS - 5p] Blocare FTP

Curs
Informații generale RL
- Regulament
  - Notare Seria CA
  - Notare Seriile CB/CC
- Resurse
  - Cartea RL
  - Packet Tracer
  - Mașină Virtuală
- Calendar
- Catalog
- Desfășurătorul semestrului
- Echipa RL
- Feed RSS
- Orar

Cursuri
- Cursul 00 - Regulament. Prezentarea echipei.
- Cursul 01 - Introducere. Mediul fizic.
- Cursul 02 - Nivelul legătură de date
- Cursul 03 - Protocolul IP
- Cursul 04 - VLAN
- Cursul 05 - Rutare
- Cursul 06 - IPv4 și IPv6
- Cursul 07 - Securizarea Rețelei
- Cursul 08 - Translatarea Adreselor
- Cursul 09 - Virtualizarea echipamentelor și a rețelelor
- Cursul 10 - Atacuri de Rețea
- Cursul 11 - Fragmentarea Internetului
- Cursul 12 - Exemplu subiect curs examen
- Cursul 13 - Tehnologii de Containerizare - Docker

Laboratoare
- **Laborator 1. Introducere în Rețele de Calculatoare**
- **Laborator 2. Comutarea pachetelor**
- **Laborator 3. Adresarea IP**
- **Laborator 4. VLAN (Virtual LAN)**
- **Laborator 5. Rutare**
- **Laborator 6. Virtualizare în Docker**
- **Laborator 7. Adresare IP și rutare în Linux**
- **Laborator 8. Securizarea unui server**
- **Laborator 9. Translatarea de adrese**
- **Laborator 10. Clienți de rețea**
- **Laborator 11. Securitatea Rețelelor Locale**
- **Laborator 12. Amazon Web Services**
- **Laborator extra - Virtualizare în Networking**
- Laborator Recapitulare

Teme
- Tema1
- Tema2